

AI & Machine Learning Bootcamp

Sessions × 2 Hours · Foundations → GenAI 20

Interactive Roadmap

ML Core

S1 – S6

Foundations, regression, classification, trees & SVM

Deep Learning

S7

Neural networks, CNNs, RNNs, optimization

NLP

S8 – S13

NLP: fundamentals → tokenization → analysis → LM → embeddings → Seq2Seq (S8–S13)

GenAI

S14 – S16

BERT, GPT, LLMs, RAG & MLOps

Introduction to Artificial Intelligence

How Does a Computer Think? — traditional vs. AI programming

What Is AI — a simple, comprehensive definition

Three Types of AI — Narrow, General, and Super

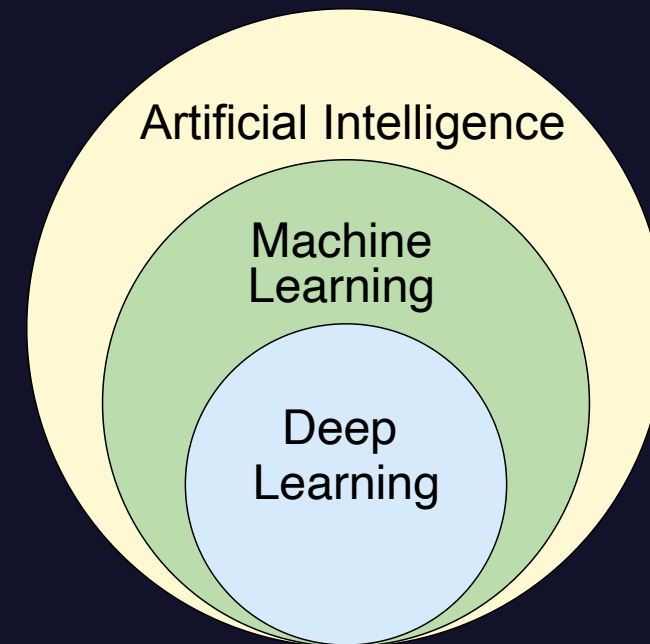
Data — the real fuel of AI

slides · ~15 min 6 ▶



A Simple, Comprehensive Definition

Artificial Intelligence (AI) refers to computer systems designed to perform tasks that typically require human intelligence — such as learning from experience, recognizing patterns, reasoning about information, understanding language, and making decisions under uncertainty



.Learning: improve performance from data and feedback (e.g., recommendation engines)

.Reasoning: draw conclusions from incomplete or noisy information

.Perception: interpret images, speech, and sensor signals (e.g., face unlock)

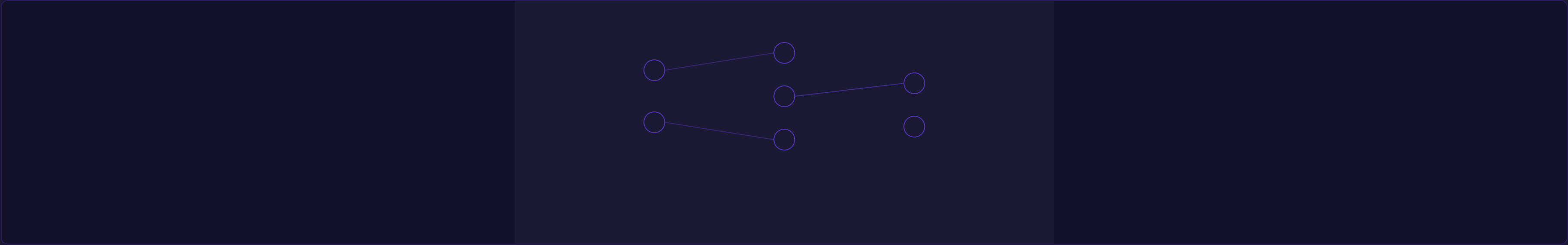
.Language: understand and generate text or speech (e.g., voice assistants, chatbots)

.Everyday examples: spam filters, navigation apps, fraud detection, product suggestions

.Key phrase: tasks that typically require human intelligence



End-to-end machine learning workflow



Traditional Programming

.Developer writes explicit rules (if/else, formulas)

.Logic is fixed before the program runs

.Output is deterministic for the same input

.Best when rules are known and stable

.System learns patterns from examples (data)

.Rules emerge from training — not hand-coded

.Output is probabilistic (confidence scores)

.Best when rules are too complex to write manually

The Core Difference

.Traditional: Input + Rules → Output

.ML: Input + Output (labels) → Learned Rules (model)

.You teach the machine by showing examples

.This bootcamp teaches you the ML path

Data is not just an input — it is the engine of artificial intelligence. A model can only learn what its training data represents. Poor, biased, or mislabeled data produces unreliable systems .regardless of algorithm choice

Clean

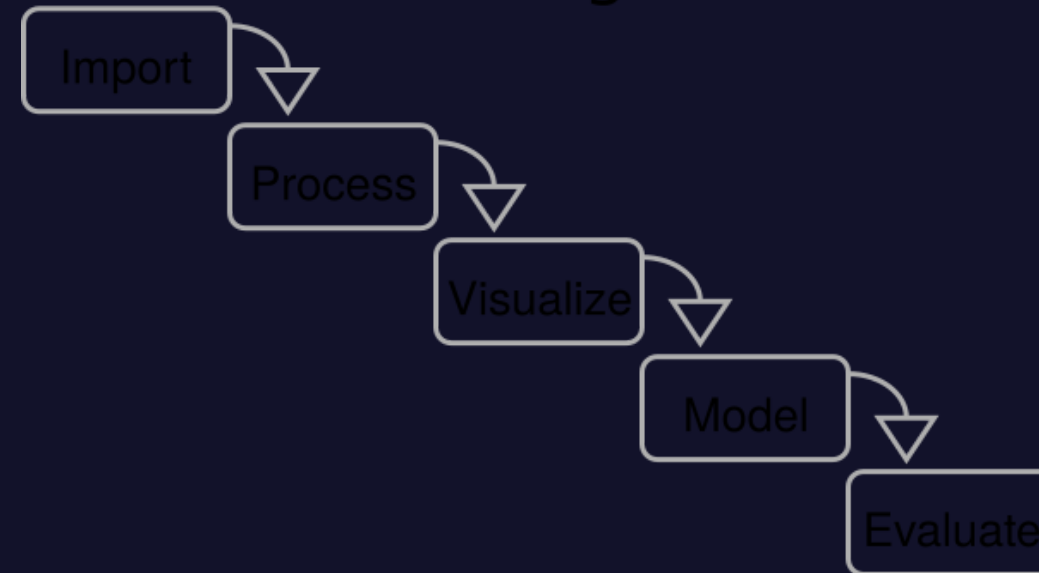
Scale

Encode

Split

Data preprocessing before modeling

Machine learning workflow



.Quality beats quantity: clean, representative data outperforms massive noisy datasets

.Labeled data: each example has a known answer (supervised learning — classification, regression)

.Unlabeled data: patterns discovered without explicit answers (clustering, representation learning)

.Features: measurable properties extracted from raw data (columns, pixels, words)

.Garbage in, garbage out (GIGO): bad preprocessing → bad predictions

.Bridge to Section 1: we start with data cleaning, scaling, and train/test splits

.Link forward: Section 1 covers the preprocessing pipeline in depth



Narrow AI (also called Weak AI or ANI) is designed and trained for one specific task or domain. It can match or exceed human performance in that narrow scope but cannot transfer its skills to unrelated problems.



What It Is

.Specialized systems optimized for a single purpose

.No general reasoning or common-sense understanding

Real-World Examples

.Chess engines (Deep Blue, Stockfish) — beat grandmasters at chess only

.Face recognition on phones — identifies faces, not general vision

.Spam filters — classify email, cannot write poetry

.ChatGPT — excels at language tasks, not physical robotics

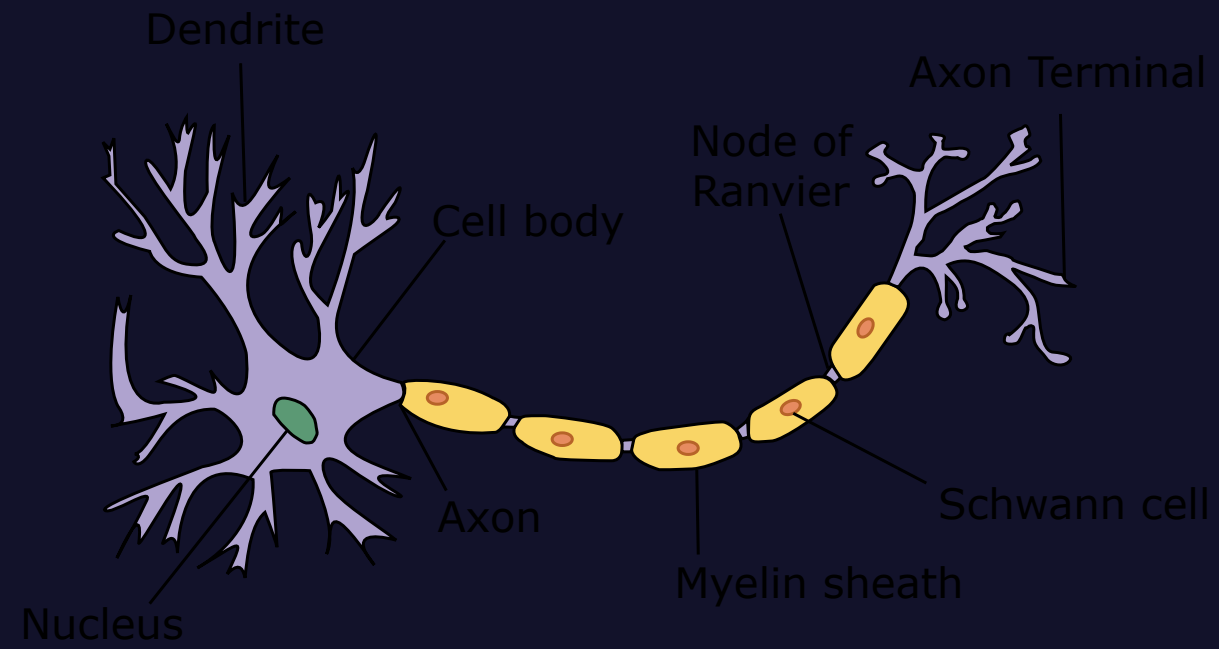
Limitations & Status

.Fails outside its training domain (no true understanding)

.Status: everywhere today — this is what industry builds and deploys

.Emphasize: ANI = practical, deployable, industry-ready

General AI (AGI) would match human-level cognitive ability across a wide range of tasks — learning, reasoning, and adapting to new domains without retraining from scratch, much like a .person who can switch from medicine to music



What It Would Be

.Human-level versatility across diverse tasks and domains

.Transfer learning: skills from one area apply to unrelated areas

.Common-sense reasoning and abstract problem solving	
Hypothetical Examples	
.A single agent that diagnoses patients, writes code, and drives a car	
.Learns a new language from a few examples, like a child	
Current Status	
.Not achieved yet — active research at OpenAI, DeepMind, Anthropic, and academia	

.Large language models show sparks of generality but lack robust reasoning

.Timeline estimates vary widely (years to decades)

.Status: research goal — not available as a product today

Super AI (ASI) refers to hypothetical systems that surpass human intelligence in virtually every field — scientific creativity, social skills, wisdom, and problem-solving — and could improve themselves at an accelerating rate

What It Would Mean

.Outperforms the best humans in all cognitive tasks

.Self-improvement loop: each upgrade makes the next upgrade faster

.Unpredictable impact on science, economy, and society	
Why It Matters (Even If Theoretical)	
.AI safety and alignment: ensuring AI goals match human values	
.Ethics frameworks guide today's Narrow AI deployment	
.Responsible AI: bias, privacy, transparency, and accountability	

Bootcamp Focus

.ASI is science fiction today — but ethics applies to Narrow AI now

.This bootcamp builds practical Narrow AI: ML, DL, NLP, and GenAI

Three types of AI — at a glance			
STATUS	SCOPE	.ABBREV	TYPE
Exists today — industry standard	One specific task	ANI	Narrow AI
Research goal — not yet achieved	Human-level, all domains	AGI	General AI
Theoretical — ethics still relevant	Beyond human, all domains	ASI	Super AI

.Bridge to Chapter 2 — History of AI

History of Artificial Intelligence

Birth of the field (1956) — Dartmouth & Alan Turing

Philosophical & mathematical roots — before computers

Expert systems boom & second AI winter (1980s)

First golden age & first AI winter (1970s)

Modern renaissance (1990s–2012) & deep learning era

slides · ~15 min 8 ▶



Before Electronic Computers

.Long before the word 'Artificial Intelligence' existed, thinkers asked whether reasoning could be mechanized — laying the intellectual foundation for modern AI

Turing machine — foundational model of computation

17th–19th Century: Machines & Logic

.Gottfried Leibniz (1646–1716): dreamed of a universal calculus of reasoning

.Charles Babbage & Ada Lovelace: Analytical Engine — first programmable machine concept

.George Boole (1847): Boolean algebra — logic as mathematics (AND, OR, NOT)

Early 20th Century: Formal Computation

.Kurt Gödel (1931): limits of formal systems — some truths are unprovable

?Alonzo Church & Alan Turing (1936): what can be computed in principle

.Turing Machine: abstract model of any computer — the blueprint for AI research

Why It Matters

.AI is not about robots — it starts with logic, math, and the limits of computation

.Every ML model today runs on Turing-equivalent hardware

.Pre-computer era — no electronic AI yet, only ideas and math

Artificial Intelligence was officially born as a research discipline at the Dartmouth Summer Research Project on Artificial Intelligence (1956), organized by John McCarthy, Marvin Minsky, .Nathaniel Rochester, and Claude Shannon



.John McCarthy coined the term 'Artificial Intelligence' at Dartmouth (1956)

'.Proposal: 'every aspect of learning or intelligence can in principle be precisely described and simulated by a machine

.Alan Turing (1912–1954): WWII code-breaker; asked 'Can machines think?' in his 1950 paper

.Turing Test: if a human cannot distinguish machine from human in conversation, the machine is intelligent

.Turing's work on computation and the Turing Test remain touchstones for AI ethics and evaluation

.official birth year of AI as a field = 1956

High Expectations & Early Programs (1950s–1960s)

.The first decade after Dartmouth was marked by enormous enthusiasm. Researchers believed human-level AI was only 20 years away — a prediction that would prove premature

IBM 7030 Stretch — mainframe era when early AI programs ran

.Logic Theorist (1956, Newell & Simon): first program to prove mathematical theorems — mimicked human problem-solving

.General Problem Solver (1957): heuristic search for any well-defined problem

.ELIZA (1966, Joseph Weizenbaum): chatbot simulating a Rogerian psychotherapist — fooled many users

.Perceptron (1958, Rosenblatt): early neural network for pattern recognition — initial excitement

'.Funding flowed from DARPA, NSF, and governments — AI was the 'next big thing

'.Hubris: Minsky (1967) — 'Within a generation ... the problem of creating AI will substantially be solved

.Golden age = symbolic AI + early neural nets + massive optimism

.By the early 1970s, AI research had failed to deliver on its boldest promises. Governments and agencies cut funding sharply — the first 'AI winter' had arrived

What Went Wrong

.Combinatorial explosion: search spaces grew faster than computers could explore

.Brittle systems: symbolic programs failed on real-world noise and ambiguity

.Minsky & Papert (1969): Perceptron book showed single-layer nets cannot solve XOR — neural net funding dropped

Key Events

.Lighthill Report (1973, UK): criticized AI research; led to major UK funding cuts

.DARPA reduced grants for undirected 'general AI' research in the mid-1970s

.Speech recognition and machine translation projects underperformed expectations

Lesson for Today

.Hype cycles are real — capability must match promises

.Narrow, practical wins survive winters; grand claims do not

First AI winter — causes vs. effects	
EFFECT	CAUSE
Loss of public and funder trust	Overpromised timelines
Could not scale symbolic search	Limited compute (1970s hardware)
Neural network research stalled ~15 years	Perceptron limitations exposed
Statistical methods lacked fuel	No big data yet

.First winter $\approx$ 1974–1980 — funding drought, not end of research

Expert Systems

.IF-THEN rule engines encoding human domain expertise

.MYCIN (1970s–80s): diagnosed blood infections — rivaled human experts

.XCON (1980): configured DEC VAX computers — saved millions annually

.Commercial boom: AI market reached ~\$1B by mid-1980s

Neural Networks Return

.Backpropagation popularized (Rumelhart, Hinton, Williams, 1986)

.Multi-layer networks could learn non-linear patterns (including XOR)

.Parallel Distributed Processing (PDP) books reignited connectionist research

.Still limited by data and compute — not yet mainstream

Global Context

.Japan Fifth Generation Computer Project (1982): \$400M push for AI supercomputers

.US and Europe responded with competing AI funding programs

.AI returned to fashion — but fragility of expert systems would soon show

The expert-system bubble burst in the late 1980s. Maintaining rule bases proved expensive, hardware could not keep pace with ambitions, and symbolic AI hit a ceiling — triggering a .second funding and market pullback

.Expert systems were costly to update — rules drifted as domains changed

.Desktop PCs replaced specialized Lisp machines — AI hardware startups failed

.Japan Fifth Generation project fell short of goals by the early 1990s

- .DARPA shifted funding toward specific applications, not general intelligence
- .AI company valuations collapsed; 'AI' became a risky label in boardrooms
- .Research continued quietly in universities — seeds for the next wave were planted

Two AI winters compared		
SECOND WINTER (LATE 1980S)	FIRST WINTER (1970S)	
Expert system market crash	Lighthill Report, perceptron limits	Trigger
Commercial AI / Lisp machines	General AI / neural nets	Main victim
Statistics, ML, backprop research	Academic AI, expert systems niche	What survived
Don't overpromise commercial ROI	Don't overpromise timelines	Lesson

.Late 1980s–early 1990s — AI rebranded as ML in many labs



Deep Blue, Big Data & the Deep Learning Spark

.AI recovered not through grand symbolic promises but through statistics, data, and faster hardware — culminating in deep learning's breakthrough on ImageNet in 2012



End-to-end machine learning workflow



Landmark Victories

.Deep Blue beats Garry Kasparov at chess (1997, IBM) — Narrow AI triumph

.IBM Watson wins Jeopardy! (2011) — NLP + search at scale

Enablers of the Renaissance

.Internet era: massive datasets became available (web text, images, logs)

.GPUs: parallel matrix ops made neural net training 10–100× faster

.Statistical ML: SVMs, random forests, boosting dominated Kaggle and industry

The 2012 Turning Point

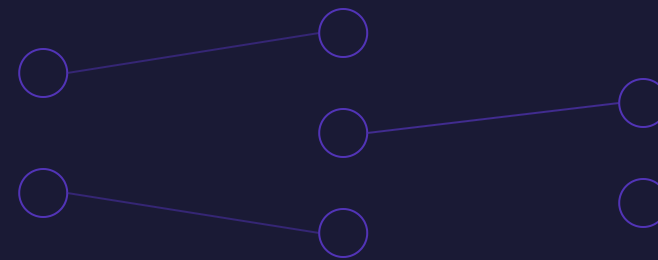
.ImageNet (2009, Fei-Fei Li): 14M labeled images — benchmark for vision

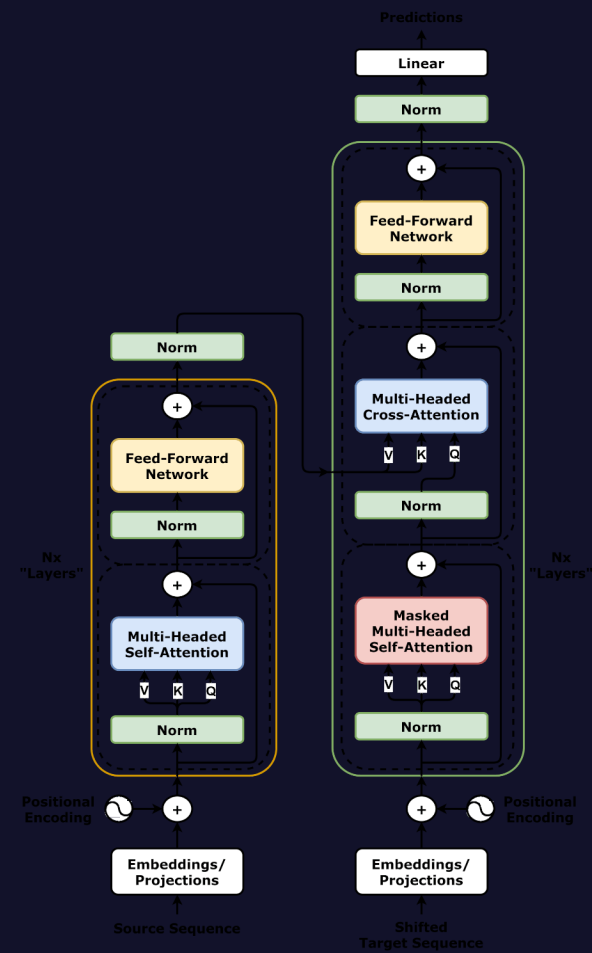
.AlexNet (2012, Krizhevsky et al.): deep CNN crushed ImageNet — error rate halved

.Deep learning era officially began — industry pivoted en masse

.Deep Blue → 2012 AlexNet = path from symbolic wins to deep learning 1997

Since 2012, AI has advanced at unprecedented speed — from real-time object detection to transformers and chatbots used by hundreds of millions. This is the era this bootcamp prepares you to build in





Computer Vision & Scale

.YOLO (2016): real-time object detection — AI in cameras, drones, autonomous vehicles

.ResNet, EfficientNet: ever-deeper and more efficient image classifiers

.AlphaGo (2016): deep RL beats world Go champion — beyond explicit search

Language & Generative AI

.Transformers (2017, 'Attention Is All You Need'): parallel sequence modeling

.BERT (2018), GPT-2/3 (2019–2020): pre-trained language models at scale

.ChatGPT (2022): 100M users in 2 months — GenAI enters mainstream

.GPT-4, Claude, Gemini, multimodal models (2023–2025): text, image, code, voice

Bootcamp Bridge

.You will learn the stack behind this era: ML → DL → NLP → GenAI → RAG → MLOps

.History teaches patience: winters follow hype — solid skills outlast trends

.Next: AI domains and branches — how the field splits into specializations

AI Domains Branches

Deep Learning — neural network simulation

Machine Learning — patterns & algorithms

Computer Vision — images, video & security

NLP — how machines understand language

Expert systems & decision making

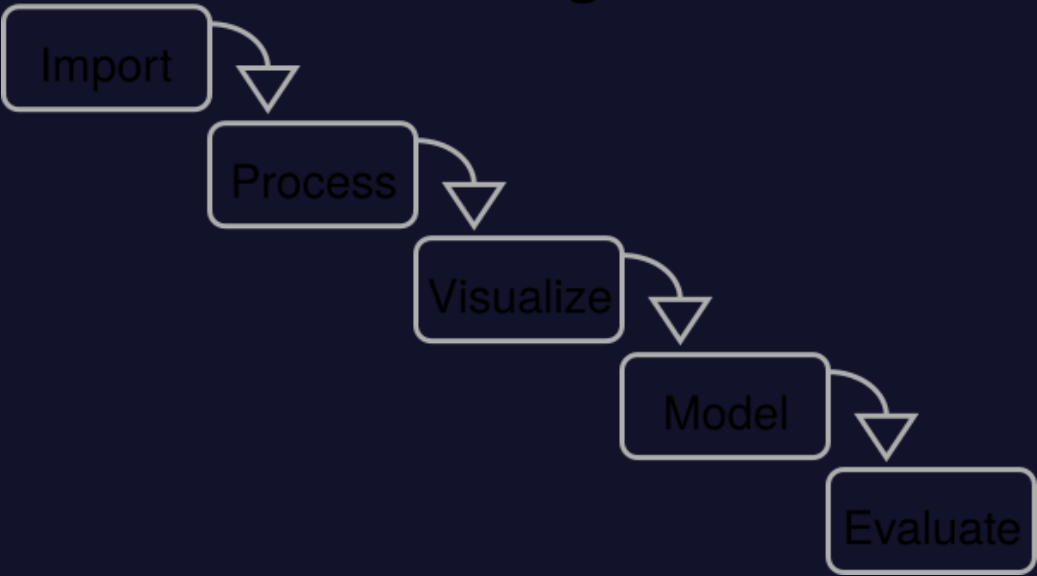
Robotics & autonomous systems

Machine Learning (ML) enables computers to learn patterns from data without being explicitly programmed for every scenario. It is the foundation of modern AI and the starting point of this bootcamp



End-to-end machine learning workflow

Machine learning workflow



Core Idea

.Learn a mapping from inputs (features) to outputs (labels or clusters)

.Generalize from training examples to unseen data

.Evaluate with held-out test sets and proper metrics

Learning Types

.Supervised: labeled data — classification & regression

.Unsupervised: no labels — clustering, dimensionality reduction

.Reinforcement: reward signals — game playing, robotics control

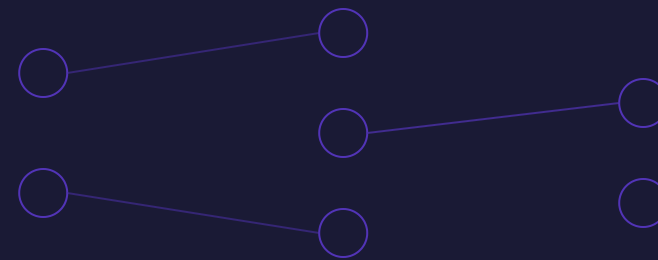
Algorithms & Bootcamp

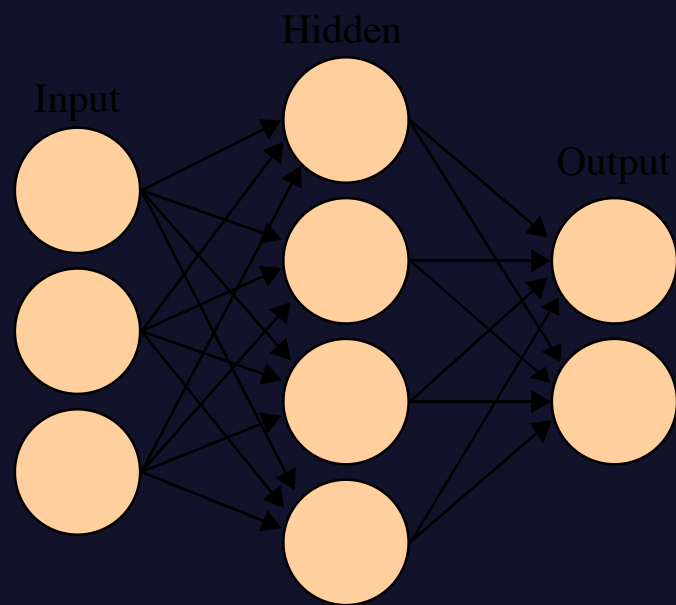
.Linear & logistic regression, decision trees, SVM, k-means, PCA

.This bootcamp: Sections S1–S6 — preprocessing through classical ML

.ML = patterns and algorithms — bootcamp Weeks/Days 1 focus

Deep Learning (DL) uses multi-layer artificial neural networks to learn hierarchical representations — from edges in images to words in sentences. It powers the breakthroughs of the .2012–today era





How It Works

.Artificial neurons: weighted inputs → activation → layered composition

.Backpropagation adjusts weights to minimize prediction error

.Deep = many layers extract features automatically (no hand-crafted rules)

Key Architectures

.CNNs: spatial patterns in images and video

.RNNs / LSTMs: sequential data (text, time series)

.Transformers: parallel attention — foundation of GPT and BERT

Requirements & Bootcamp

.Needs large datasets, GPUs, and careful regularization

.This bootcamp: Section S7 — 7 labs from MLP to CNN and RNN

.DL $\subset$ ML — neural networks with many layers



.NLP teaches computers to read, interpret, and generate human language — from tokenizing raw text to building chatbots and machine translation systems

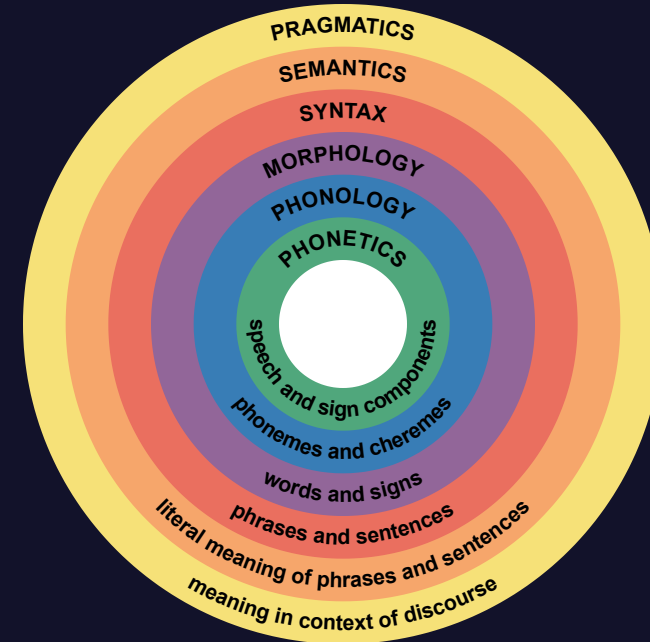
Tokenize

Embed

Encode

Predict

Typical NLP processing pipeline



Core Tasks

.Text cleaning & tokenization: raw text → structured units

.Understanding: POS tagging, NER, sentiment analysis

.Generation: summarization, translation, conversational AI

Evolution

.Classical: bag-of-words, n-grams, TF-IDF

.Neural: word embeddings (Word2Vec), RNNs, attention

.Modern: pre-trained Transformers (BERT, GPT) — transfer learning for NLP

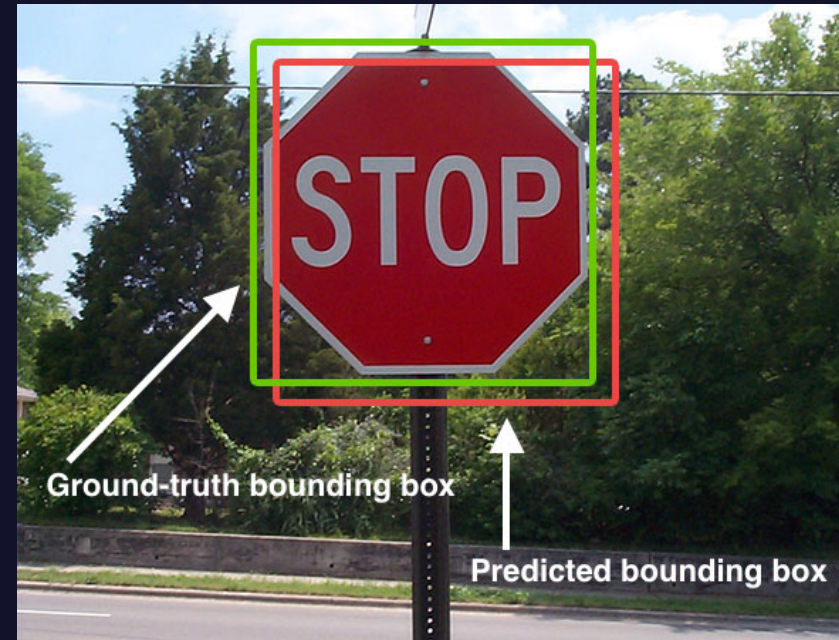
Applications & Bootcamp

.Chatbots, search, spam detection, document classification, code assistants

.This bootcamp: Sections S8–S13 — cleaning through Seq2Seq and attention

.NLP — language understanding and generation

.Computer Vision (CV) enables machines to interpret visual data — photographs, video streams, medical scans, and surveillance feeds — extracting meaning from pixels



.Image classification: what object is in this image? (ResNet, EfficientNet)

.Object detection: where are objects and what are they? (YOLO, Faster R-CNN)

.Segmentation: pixel-level boundaries for medical imaging and autonomous driving

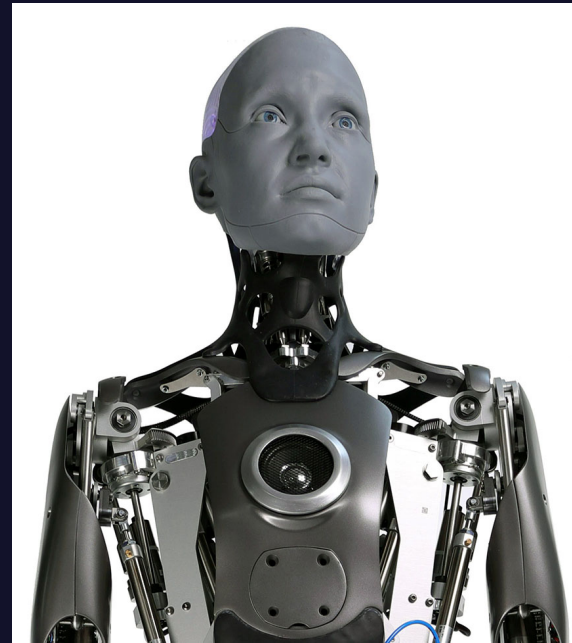
.Face recognition & biometrics: security systems, phone unlock, access control

.Video analytics: motion detection, crowd monitoring, anomaly alerts in CCTV

.Powered by CNNs and increasingly Vision Transformers (ViT)

.Bootcamp touchpoint: CNN labs and architectures in Section S7

.Robotics combines AI with mechanical systems — sensors, actuators, and real-time control — to create machines that perceive, decide, and act autonomously in dynamic environments



Core Components

.Perception: cameras, LiDAR, radar → CV + sensor fusion

.Planning: path finding, obstacle avoidance, task scheduling

.Control: motor commands executed in milliseconds

Applications

.Self-driving cars: detect lanes, pedestrians, signs — decide in real time

.Drones: aerial mapping, delivery, inspection

.Warehouse robots (Amazon, logistics): pick, pack, navigate shelves

.Surgical & industrial robots: precision tasks alongside humans

AI Domains Combined

.Robotics = CV + ML/DL + reinforcement learning + embedded systems

.Bootcamp: cross-cutting skills from S1–S7 prepare you for robotics foundations

.Robotics — perception, planning, and autonomous action

Expert Systems encode human domain knowledge as explicit IF-THEN rules — one of the first commercial AI successes. Understanding them clarifies why statistical ML and deep .learning eventually superseded pure rule-based approaches

MYCIN expert system — early rule-based medical AI

How Expert Systems Work

.Knowledge base: facts and rules written by domain experts |

.Inference engine: applies rules to reach conclusions

.Examples: MYCIN (medical diagnosis), XCON (computer configuration)

Strengths & Limits

.Strengths: interpretable, auditable, no training data required

.Limits: brittle, expensive to maintain, cannot learn from data

.1980s boom → late-1980s collapse (see AI history slides)

Today & Bootcamp

.Modern systems blend rules + ML (hybrid AI in finance, healthcare)

.Decision trees in S4 bridge symbolic and statistical approaches

.Next: Generative AI and modern applications — the current frontier

MACHINE LEARNING	EXPERT SYSTEMS	ASPECT
Learned from data	Human-written rules	Knowledge source
Retrain on new data	Manual rule updates	Maintenance
Varies (trees high, DL low)	High (explicit rules)	Interpretability
Noisy, high-dimensional data	Stable, well-defined domains	Best for

Generative AI Modern Applications

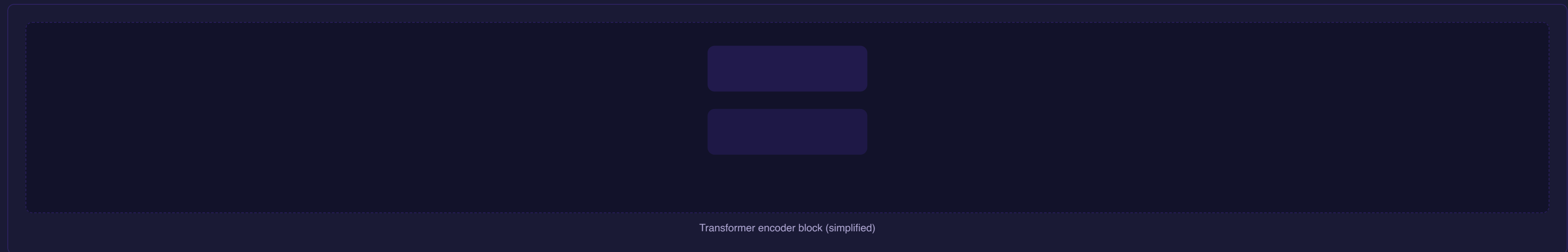
Large Language Models — conversation & context

The Generative AI revolution

Daily productivity — text, image & code generation

Industry applications — business, medicine & public security

Traditional AI answers 'What is this?' or 'What will happen?' Generative AI answers 'Create something new that matches this intent' — drafts, designs, code, and synthetic data on .demand





Discriminative vs. Generative

.Discriminative: classify spam vs. ham, predict house prices

.Generative: write emails, compose images, synthesize speech

.GenAI learns data distributions — then samples new outputs

Key Breakthroughs

.Transformers (2017): scalable sequence modeling at web scale

.Diffusion models: high-quality image generation (DALL·E, Stable Diffusion, Midjourney)

.ChatGPT (2022): conversational interface made GenAI accessible to everyone

Why It Matters Now

.Lower barrier: natural language replaces complex ML pipelines for many tasks

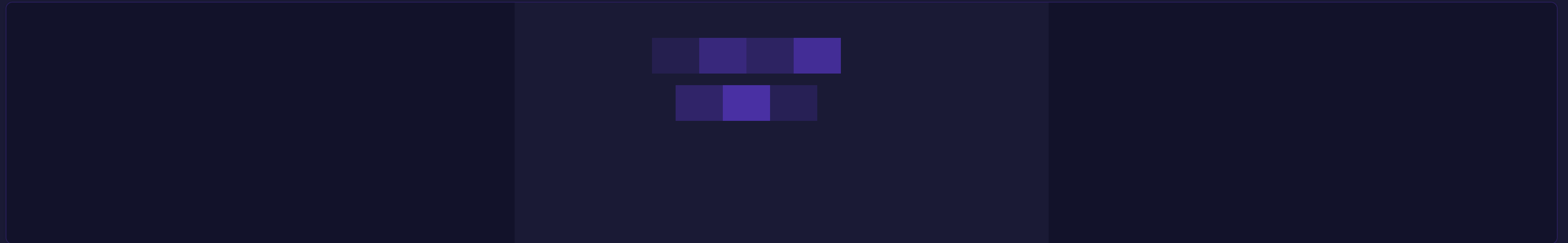
.Risks: hallucinations, bias, copyright, deepfakes — need guardrails

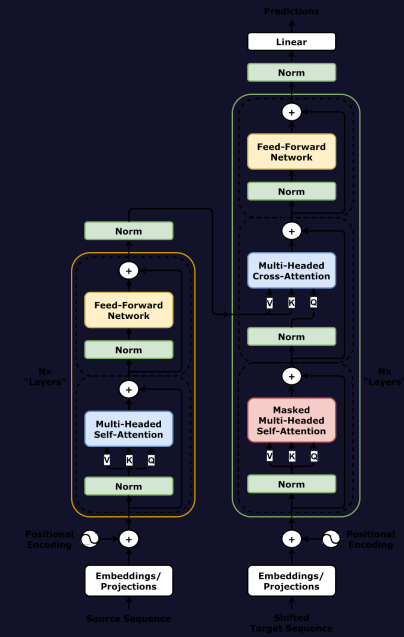
.Bootcamp: Section S14 covers BERT, GPT, attention, and LLM fundamentals

.GenAI revolution — prediction → creation paradigm shift



Large Language Models are neural networks trained on vast text corpora to predict the next token — enabling fluent conversation, reasoning over context, and task completion from .natural-language prompts





How LLMs Work

.Pre-training: learn language patterns from billions of web pages, books, code

.Self-attention: weigh every word against every other — capture long-range context

.Fine-tuning & RLHF: align outputs with human preferences (helpful, harmless, honest)

Context & Conversation

.Context window: how much prior text the model 'remembers' in one session

.Multi-turn dialogue: model tracks conversation history for coherent replies

.Prompt engineering: instructions, examples, and role-setting steer behavior

Examples & Bootcamp

.GPT-4, Claude, Gemini, Llama — general-purpose assistants and APIs

.Bootcamp path: S8–S13 (NLP foundations) → S14 (BERT, GPT, attention) → S15 (RAG)

.LLMs — conversation, context windows, and prompt steering

.Generative AI is moving from demos to production across high-impact sectors — with distinct opportunities and governance requirements in each



Retrieval-augmented generation flow

Business & Enterprise

.Customer support: AI agents resolve tickets, draft responses, summarize calls

.Marketing: personalized content, A/B copy variants, market research synthesis

.Finance: report generation, fraud narrative analysis, compliance document review

.RAG over company docs: grounded answers from internal knowledge bases (S15)

Medicine & Healthcare

.Clinical documentation: draft notes from doctor-patient conversations

.Literature review: summarize research papers and trial results

.Medical imaging assist: report generation from radiology scans (human-in-the-loop)

.Critical: HIPAA/privacy, validation, never replace clinician judgment

Public Security & Safety

.Intelligence analysis: summarize large document sets, translate multilingual sources

.Threat detection: pattern recognition in communications and surveillance metadata

.Deepfake detection: counter AI-generated misinformation and synthetic media

.Ethics: bias, civil liberties, audit trails — MLOps monitoring essential (S16)



Text, Image & Code Generation for Everyone

.GenAI is already embedded in everyday workflows — not just research labs. Knowing these tools helps trainees use AI responsibly while building their own systems in the bootcamp



Text Generation

.ChatGPT, Claude, Gemini: drafting, summarizing, translating, brainstorming

.Copilot in Word/Outlook: inline writing assistance in familiar apps

.Notion AI, Grammarly: notes, editing, and tone adjustment

Image Generation

.DALL·E, Midjourney, Stable Diffusion: marketing visuals, prototypes, storyboards

.Canva AI, Adobe Firefly: design workflows for non-designers

.Use with care: copyright, consent, and disclosure of AI-generated content

Code Generation

.GitHub Copilot, Cursor, Codeium: autocomplete, refactor, explain code

.ChatGPT/Claude for debugging, unit tests, and documentation

.Next: ethics, the future of work, and your learning roadmap

.Bridge to Chapter 5 — Ethics & the Future

Ethics the Future

Future of jobs — human–machine integration

AI ethics — bias, privacy & deepfakes

Your learning roadmap — start & grow in AI/ML

slides · ~15 min 3 ▶

.As AI systems influence hiring, healthcare, law enforcement, and public discourse, ethical design is not optional — it is a core engineering requirement

TP	FP
FN	TN

Confusion matrix layout



Bias & Fairness

.Training data reflects historical bias → models can discriminate by race, gender, age

.Examples: biased hiring filters, unequal loan approvals, skewed facial recognition

.Mitigation: diverse datasets, fairness metrics, human review, audit trails

Privacy

.Models memorize sensitive training data — risk of leakage in outputs

.Regulations: GDPR (EU), HIPAA (health), sector-specific data governance

.Best practices: anonymization, consent, data minimization, secure pipelines (S1, S16)

Deepfakes & Misinformation

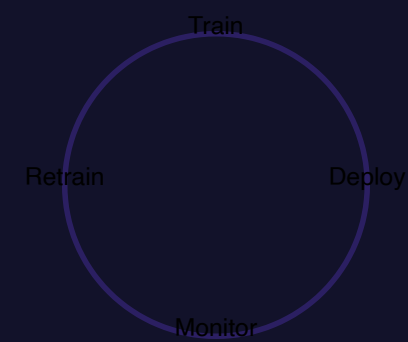
.Synthetic video/audio/text indistinguishable from real — fraud, political manipulation

.Detection tools emerging; watermarking and provenance standards in progress

.Always disclose AI-generated content; verify before sharing

.AI ethics — bias, privacy, and synthetic media risks

.AI will transform work — not by replacing every job overnight, but by reshaping tasks, roles, and the skills that matter most in a human–machine partnership



MLOps continuous delivery loop



What Changes

.Automation of repetitive tasks: data entry, basic reporting, first-draft content

.Augmentation: AI assists professionals — doctors, lawyers, engineers, analysts

.New roles: ML engineer, prompt engineer, AI ethicist, MLOps specialist, data curator

Human + Machine Model

.Humans: judgment, empathy, creativity, accountability, domain expertise

.Machines: speed, scale, pattern detection, 24/7 availability

.Best outcomes: human-in-the-loop — AI proposes, human approves

Skills That Grow in Value

.Critical thinking, communication, and interdisciplinary problem-solving

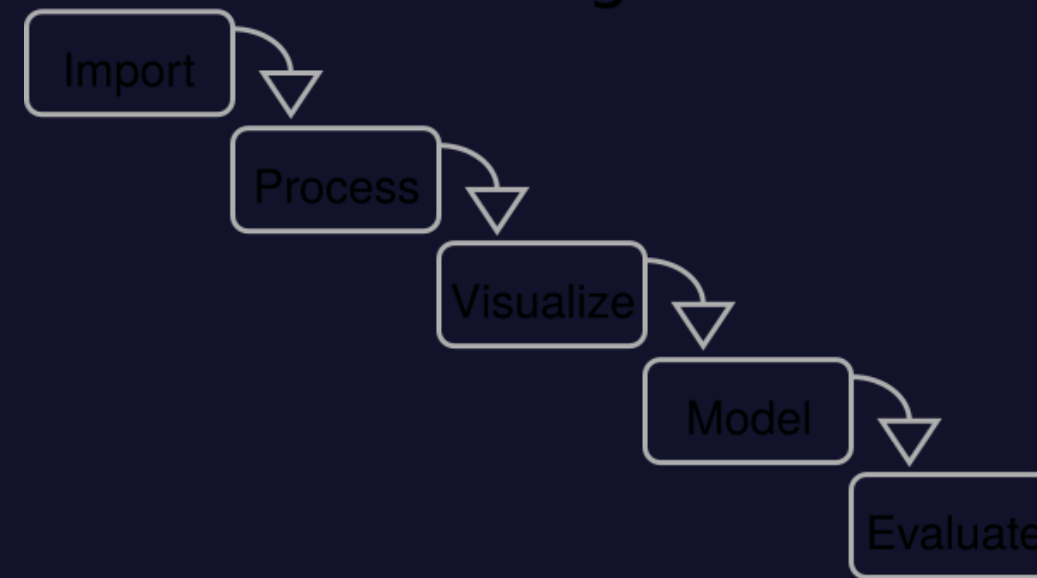
.Technical: Python, ML pipelines, data literacy — exactly what this bootcamp builds

.Adaptability: AI tools evolve fast — learn how to learn

.Future of work — augmentation over replacement

.This bootcamp is your launchpad — not your finish line. A structured path after these five days keeps momentum and builds a portfolio that employers recognize

Machine learning workflow



Phase 1 — During Bootcamp (Now)

.Follow the daily arc: S1–S6 (ML) → S7 (DL) → S8–S13 (NLP) → S14–S16 (GenAI, RAG, MLOps)

.Run every lab notebook — modify hyperparameters, break things, fix them

.Take notes on concepts you cannot explain — revisit after each section

Phase 2 — First 3 Months After

.Build 2–3 portfolio projects: classification, NLP, or a small RAG app

.Practice on Kaggle competitions and open datasets

.Read papers: start with distill.pub and Hugging Face blog posts

.Contribute to open source: scikit-learn docs, Hugging Face models

Phase 3 — Long-Term Growth

.Specialize: vision, NLP, RL, or MLOps based on interest and job market

.Stay current: follow arXiv summaries, AI newsletters, and vendor docs

.Network: local meetups, GitHub, LinkedIn — share what you build

Bootcamp → career milestones

TARGET	MILESTONE
Complete S1–S16 + core labs	Week 1 (bootcamp)
end-to-end ML project on GitHub 1	Month 1
NLP or DL project + Kaggle entry	Month 3
Specialization + internship or junior role	Month 6–12

.Final intro slide — bridge to Section 1 (S1). Total intro ~70–75 min

WEEK 1 · SESSION 1 · Foundations

1

Foundations

Data Pre-Processing

Data leakage

Encoding & imputation

Feature scaling

Train/test splits

slides · ~32 min 20 ▶

?Where Are We in the Bootcamp

.Build the end-to-end ML workflow: clean data → train → evaluate

- S1 Foundations & Preprocessing** •
- S2 Regression Models** •
- S3 Classification Basics** •
- S4 Naive Bayes & Trees** •
- S5 SVM & Kernels** •
- S6 Clustering & PCA** •
- S7 Deep Learning** •
- S8 NLP Fundamentals** •
- S9 Tokenization** •
- S10 Text Analysis & NER** •
- S11 Language Modeling** •
- S12 Embeddings & RNNs** •
- S13 Seq2Seq & NMT** •
- S14 Generative AI** •
- S15 RAG Systems** •
- S16 MLOps** •



The 3 Main Steps

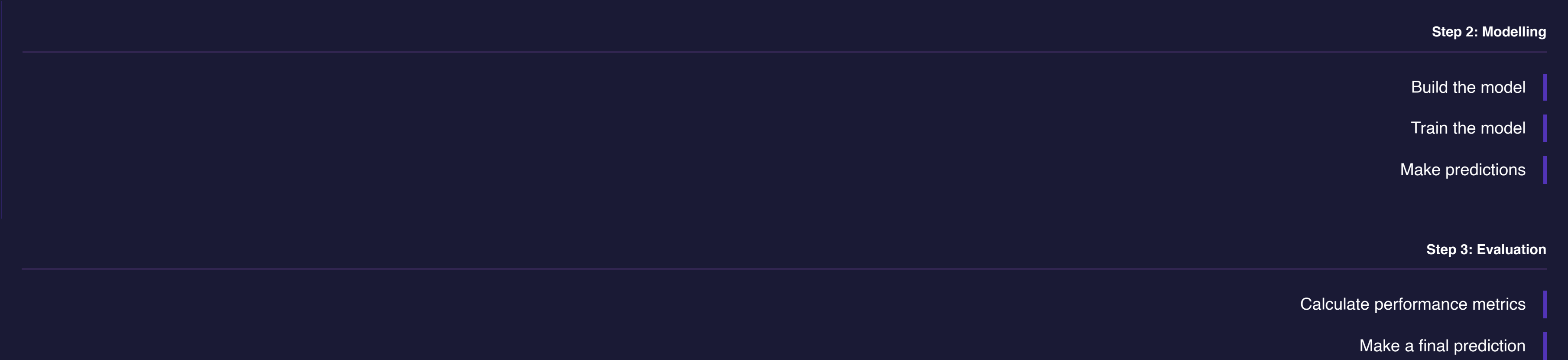
Step 1: Data Pre-Processing

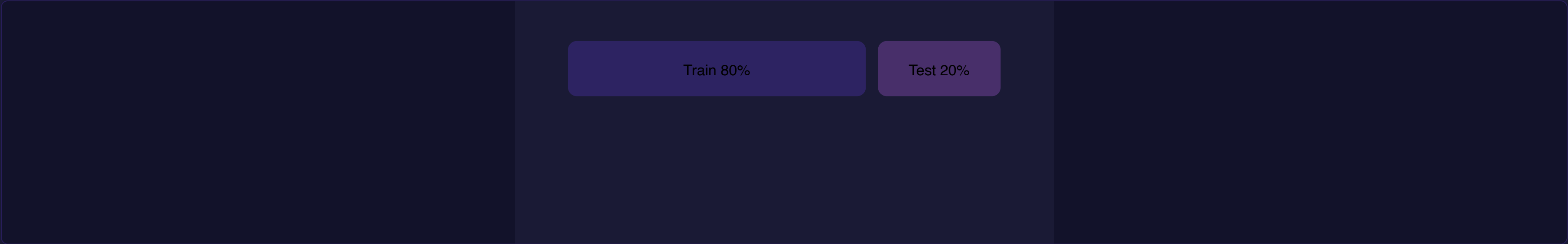
Import the data

Clean the data

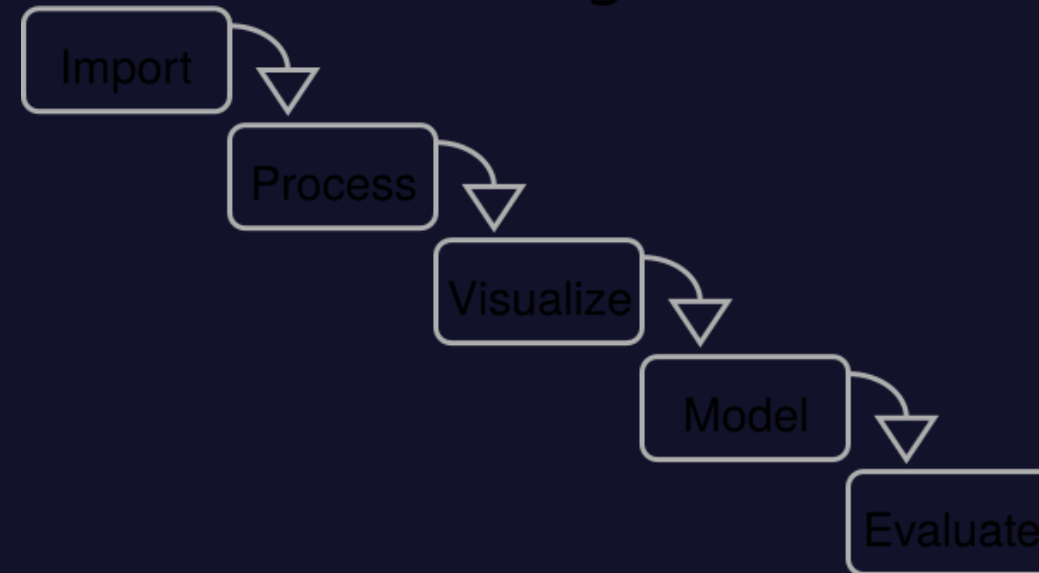
Split into training and test sets

Feature scaling





Machine learning workflow



.Training Set: Data used by the model to learn patterns

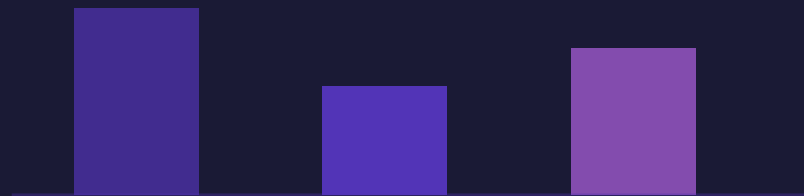
.Test Set: New, unseen data used to check model performance

.Never tune hyperparameters on the test set — it becomes part of training indirectly

.Stratified splits preserve class balance in classification tasks

.We train on one part of the data, then test on another part to make sure the model generalizes well

.Feature scaling means adjusting feature values to a similar range using methods like standardization and normalization



.It prevents large-value features from dominating smaller-value features

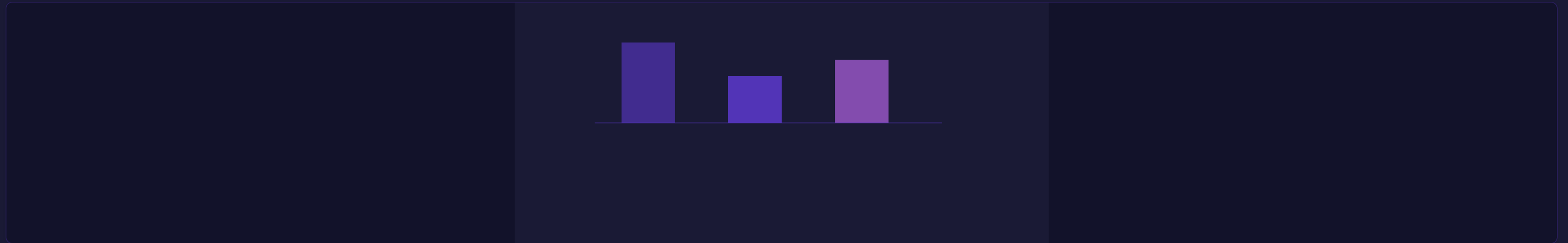
.It improves performance for distance-based and gradient-based algorithms (e.g., KNN, SVM, Gradient Descent)

.It helps training converge faster

Example (Before vs After Scaling)

WEIGHT (SCALED)	HEIGHT (SCALED)	WEIGHT (KG) - BEFORE	HEIGHT (CM) - BEFORE	STUDENT
0.22	0.25	50	150	A
0.67	0.58	70	170	B
1.00	0.83	85	185	C

.Before scaling, height and weight have different numeric ranges. After scaling, both features are on a comparable scale and contribute more fairly



.Normalization scales to [0, 1] using min–max — sensitive to outliers

.Standardization uses mean 0 and std 1 — preferred when features are roughly Gaussian

.Robust scaling uses median and IQR when outliers dominate

Normalization (Min-Max Scaling))1

.Normalization transforms each value to a range between 0 and 1

$$\frac{x - \min x}{\max x - \min x} = \text{norm} x$$

.Useful for models that depend on distances or absolute magnitudes

.Common choice for algorithms such as KNN and K-Means

Standardization (Z-score Scaling))2

.Standardization centers data around 0 with a standard deviation of 1

$$\frac{x - \mu}{\sigma} = z$$

.mu = mean of the feature, sigma = standard deviation of the feature

.Preferred when features are close to a Gaussian distribution

.Works well with Logistic Regression, SVM, and Linear Regression

.Use standardization for SVM, logistic regression, PCA, and gradient-based training

.Tree-based models are scale-invariant — scaling is optional

.When in doubt, fit the scaler on train only, then transform train and test

Normalization

.Use Normalization when feature values have very different ranges and you want all values mapped to [0, 1]

Normalization

.Typical method: Min-Max Scaling

.Good for distance-based models where magnitude differences strongly affect results

Standardization

.Use Standardization when features are expected to follow (or be close to) a Gaussian distribution, or when the dataset contains outliers

.It keeps the data centered around 0 and scales variability to 1



Given feature values: [20, 40, 60, 80, 100]



$$\frac{\min x - x}{\min x_{\max} - x} = \text{norm} x$$

$$20 = x_{min}$$

$$100 = x_{max}$$

Before vs After Normalization	
X_{NORM} (NORMALIZED VALUE	ORIGINAL VALUE (X)
0.00	20
0.25	40
0.50	60
0.75	80
1.00	100

.This table shows how all values are scaled to the range [0, 1]



Given feature values: [20, 40, 60, 80, 100]



$$\frac{x - \mu}{\sigma} = z$$

60 = μ :Mean

28.28 = σ :Standard deviation

Before vs After Standardization

STANDARDIZED VALUE (Z)	ORIGINAL VALUE (X)
1.41-	20
0.71-	40
0.00	60
0.71	80
1.41	100

.This table shows that standardized values are centered around 0 and measured in units of standard deviation

.Categorical data describes values that belong to named groups rather than continuous numbers

Category Types		
EXAMPLE	DESCRIPTION	TYPE
Education Level: High School < Bachelor's < Master's < PhD	.Categories have a meaningful order	Ordinal
Payment Method: Cash, Card, Bank Transfer	.Categories have no natural order	Nominal

Example 1: Label Encoding

ENCODED VALUE	RAW CATEGORY
0	Cash
1	Card
2	Bank Transfer

Example 2: One-Hot Encoding

BANK TRANSFER	CARD	CASH	RAW CATEGORY
0	0	1	Cash
0	1	0	Card
1	0	0	Bank Transfer

.Most machine learning algorithms work with numerical inputs only, so categorical values must be converted into numbers

How to Deal with Categorical Data (Ordinal Encoding)



[?What is Ordinal Encoding](#)

.Ordinal Encoding converts ordered categories into integers that preserve their ranking

Ordinal

One-hot

Dummy

- .Use Ordinal Encoding only when categories have a true order
- .Step 1: Identify the correct category order
- .Step 2: Assign an integer to each level based on that order
- .Step 3: Replace the original text values with encoded numbers

Example	
ENCODED VALUE	SATISFACTION LEVEL
1	Low
2	Medium
3	High

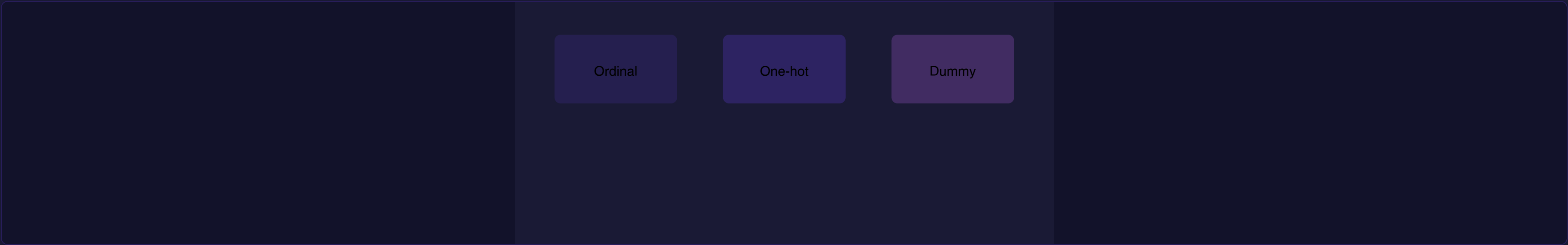
.This method keeps the order information, which is important for many machine learning models

How to Deal with Categorical Data (One-Hot Encoding)



?What is One-Hot Encoding

.One-Hot Encoding converts each category into a separate binary column (0 or 1)



- .Use One-Hot Encoding for nominal categories (no natural order)
- .Step 1: List all unique categories
- .Step 2: Create one column for each category
- .Step 3: Put 1 in the matching category column and 0 in all others

Example			
BANK TRANSFER	CARD	CASH	PAYMENT METHOD
0	0	1	Cash
0	1	0	Card
1	0	0	Bank Transfer

.This method avoids creating a false ranking between categories



?What is Dummy Encoding

.Dummy Encoding is similar to One-Hot Encoding, but it removes one category column to avoid redundancy

Ordinal

One-hot

Dummy

.It helps prevent the dummy variable trap (perfect multicollinearity), especially in linear models

.Step 1: Start with One-Hot encoded columns

.Step 2: Drop one category as a reference (baseline)

.Step 3: Keep the remaining binary columns

.Original categories: Payment Method = [Cash, Card, Bank Transfer]

.Reference category (dropped): Cash

Example

PAYMENT METHOD		
Cash	0	BANK TRANSFER
Card	1	CARD
Bank Transfer	0	1

.In this setup, Cash is represented when all remaining columns are 0

How to Deal with Categorical Data (Other Encoding Types)

.Besides Ordinal, One-Hot, and Dummy Encoding, there are other useful methods

Ordinal

One-hot

Dummy

Encoding Methods Overview		
Best Use Case	Idea	Encoding Type
Large datasets with many categories	.Replace each category with how often it appears	Frequency Encoding
Tree-based models with high-cardinality features	.Similar to frequency, but uses raw counts directly	Count Encoding
Supervised tasks (use with care to avoid leakage)	.Replace category with target mean for that category	Target Encoding
High-cardinality data with fewer columns than one-hot	.Convert category index to binary digits across columns	Binary Encoding
Very large and dynamic category sets	.Use a hash function to map categories into fixed columns	Hash Encoding

Frequency Encoding (City))1	
Frequency	City
0.50	Riyadh
0.33	Jeddah
0.17	Dammam

Count Encoding (City))2	
COUNT	CITY
3	Riyadh
2	Jeddah
1	Dammam

Target Encoding (City))3	
MEAN TARGET	CITY
0.80	Riyadh
0.40	Jeddah
0.20	Dammam

Binary Encoding (City))4		
BINARY CODE	CATEGORY ID	CITY
01	1	Riyadh
10	2	Jeddah
11	3	Dammam

CITY	HASH BUCKET
Riyadh	2
Jeddah	0
Dammam	3

.Choose the encoding method based on category order, cardinality, model type, and risk of data leakage

.Missing data refers to values that are absent or incomplete in one or more features of a dataset

.It can introduce bias into model training

.It may reduce prediction accuracy

.It can cause errors in analysis if not handled properly

Types of Missing Data

MEANING	TYPE
.Missingness happens randomly and is unrelated to any variable in the dataset	MCAR (Missing Completely at Random)
.Missingness depends on other observed variables, but not on the missing value itself	MAR (Missing at Random)
.Missingness depends on the missing value itself or unobserved factors	NMAR (Not Missing at Random)

?Handling Missing Data - Why Do We Have Missing Data

.Missing values can appear for many practical reasons during data collection and processing

Common Causes		
EXAMPLE	DESCRIPTION	CAUSE
.A user leaves the Age field empty in a form	.Data is skipped, entered incorrectly, or forgotten	Human Error
.A medical sensor stops recording heart rate temporarily	.Measurement tools fail or disconnect	Device/Sensor Failure
.One database has salary, another does not	.Missing fields appear when combining data from multiple sources	Data Integration Issues
.A customer does not provide income details	.Participants choose not to share sensitive information	Privacy or Refusal
.Network interruption causes missing rows in logs	.Data is lost during storage, transfer, or export	System/Transmission Errors
.Pregnancy_Status is empty for male patients	.Some fields are only relevant for specific groups	Conditional Missingness

.Understanding why data is missing helps us choose the right treatment method (deletion, imputation, or advanced modeling)



Missing Completely at Random (MCAR)

.MCAR means missing values occur randomly and are independent of both observed and unobserved variables

.MCAR has no systematic pattern

.In practice, MCAR is often an unrealistic assumption in real-world datasets

.Before assuming MCAR, review data collection workflow and consult domain experts

.If MCAR is reasonable, simple imputation (mean/median/mode) can work

Illustrative Data Example

SCORE	AGE	STUDENT ID
85	20	101
88	Missing	102
Missing	21	103
91	23	104
76	Missing	105
Missing	22	106

.Missing values appear across different rows and columns with no clear pattern, which is consistent with MCAR

Missing at Random (MAR)

.MAR means missingness is related to observed features in the dataset, but not to the missing feature itself

Imputation Techniques for MAR

WHY IT WORKS FOR MAR	IDEA	METHOD
.Uses observed features to find similar patterns	.Fills missing values using the nearest similar records	kNN Imputation
.Captures non-linear relationships from observed data effectively	.Uses random-forest models iteratively to predict missing values	MissForest

Example: kNN Imputation (k = 2)

.For student C, the two nearest students based on Grade and Attendance are A and B

$$7.5 = \frac{7 + 8}{2} = c_{Study\ Hours}$$

STUDENT	ATTENDANCE (%)	GRADE	STUDY HOURS
A	95	90	8
B	92	88	7
C	96	91	Missing
D	75	70	3

Example: MissForest

.MissForest trains a Random Forest model using known rows to predict the missing value for student C

$7.6 \approx \hat{c}$ Study Hours

Example (Table)

STUDENT	STUDY HOURS	GRADE (OBSERVED)
1	Missing	95
2	Missing	92
3	4	88
4	6	75
5	7	70

.MAR is common in practice because missingness can often be explained by available variables



Missing Not at Random (MNAR)

.MNAR is usually the most complex type because missingness depends on the missing value itself or unobserved variables

.Standard imputation can be unreliable without additional data or strong domain assumptions

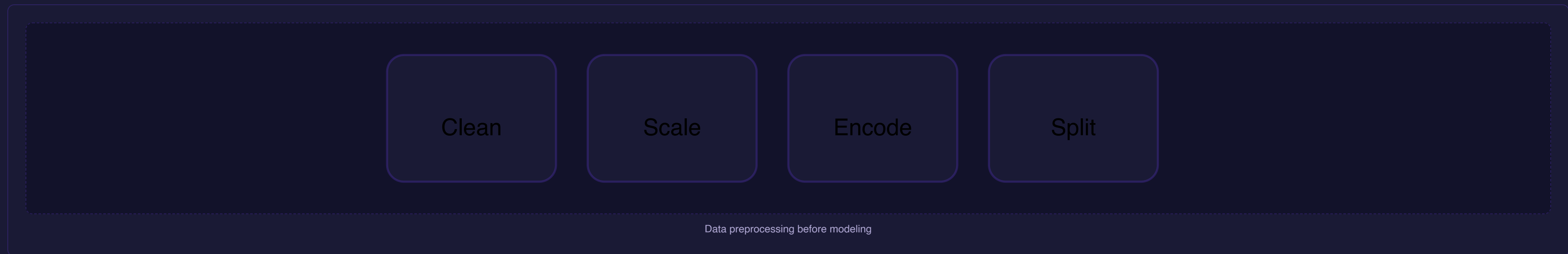
.MNAR has a systematic pattern tied to hidden information

Example (Table)

REPORTED STRESS LEVEL	PARTICIPANT
2	1
Missing	2
3	3
Missing	4
1	5

Practical Tip: Preserve Missingness Pattern

STRESS_WAS_MISSING	STRESS (IMPUTED)	PARTICIPANT
0	2.0	1
1	4.1	2
0	3.0	3
1	4.3	4
0	1.0	5



.Load the dataset

.Detect and handle missing values

.Encode categorical features into numerical form

.Split the dataset into training and test sets

.Scale numerical features when needed

.Pipeline: Raw Data -> Missing Data Treatment -> Categorical Encoding -> Train/Test Split -> Feature Scaling -> ML Model

Section Takeaways

.Always split data before touching the model — test set simulates production .1

.Feature scaling matters for distance- and gradient-based algorithms .2

.Preprocessing choices persist through the entire pipeline .3

?**Think:** Which preprocessing step would break your model if skipped

WEEK 1 · SESSION 2 · Foundations

2

Regression Models

Ridge / Lasso

Residual diagnostics

OLS & R^2

Linear regression

slides · ~59 min 37 ▶

?Where Are We in the Bootcamp

.Learn how models quantify relationships and when linear assumptions hold

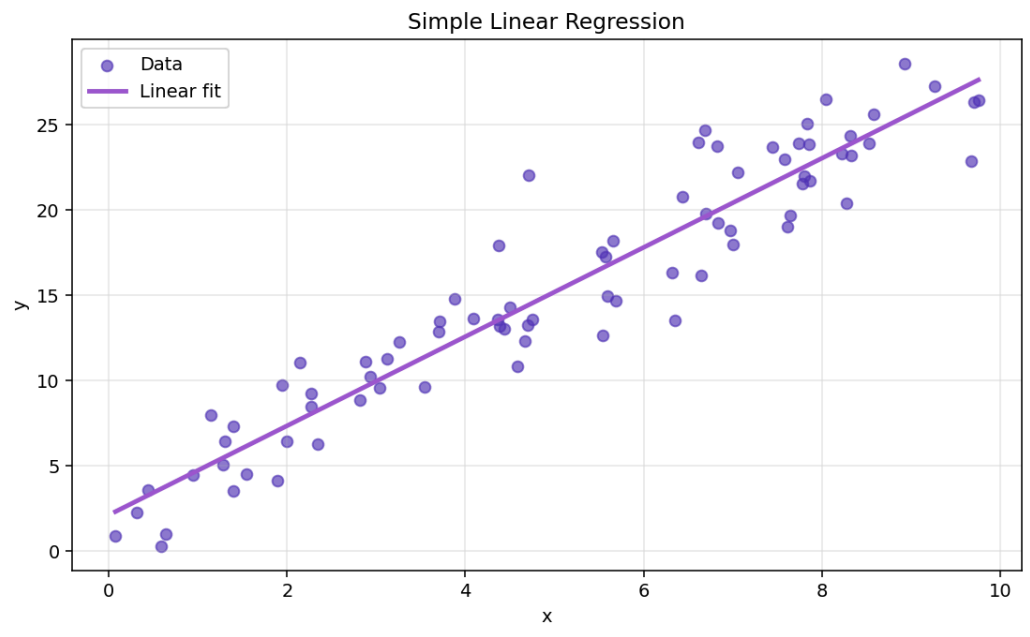
- S1 Foundations & Preprocessing •
- S2 Regression Models •
- S3 Classification Basics •
- S4 Naive Bayes & Trees •
- S5 SVM & Kernels •
- S6 Clustering & PCA •
- S7 Deep Learning •
- S8 NLP Fundamentals •
- S9 Tokenization •
- S10 Text Analysis & NER •
- S11 Language Modeling •
- S12 Embeddings & RNNs •
- S13 Seq2Seq & NMT •
- S14 Generative AI •
- S15 RAG Systems •
- S16 MLOps •



?What is Simple Linear Regression

.Simple Linear Regression models the relationship between one independent variable x and one dependent variable y using a straight line

$$b_1x + b_0 = y$$



Practical Example

.Suppose x = Hours of Study and y = Exam Score

$$5x + 40 = y$$

Practical Example

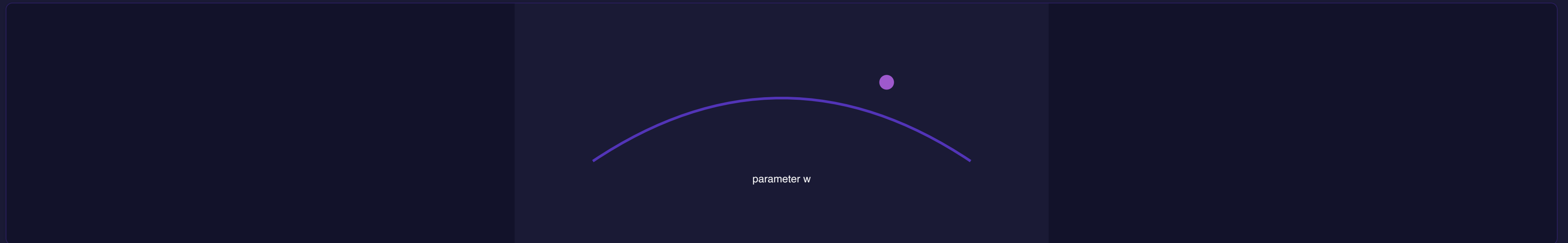
.Slope ($b_1 = 5$): each extra study hour increases predicted score by 5 points

.Intercept ($b_0 = 40$): if study hours are 0, predicted score is 40

Meaning of Each Symbol	
MEANING	SYMBOL
.Predicted output (dependent variable)	y
.Input feature (independent variable)	x
.Intercept: predicted value of y when x = 0	b_0
.Slope: expected change in y when x increases by 1 unit	b_1

.The regression line is the best-fit line that minimizes prediction errors

.Ordinary Least Squares is the most common method to fit a linear regression line by minimizing prediction errors



$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

y_i :Actual value

$\hat{y}_i$:Predicted value

$\hat{y}_i - y_i = e_i$:Residual error

.Squared errors make all errors positive, penalize large errors, and provide a clear optimization objective

.Multiple Linear Regression describes the relationship between one dependent variable y and two or more independent variables

$${}_nb_nx + \cdots + {}_2b_2x + {}_1b_1x + {}_0b = y$$

.Purpose: predict the target variable y using multiple input factors

.Compared with simple regression, it models more realistic multi-factor scenarios

Example

.Suppose we want to predict house price using house size and number of bedrooms

$$_215x + _12.5x + 50 = y$$

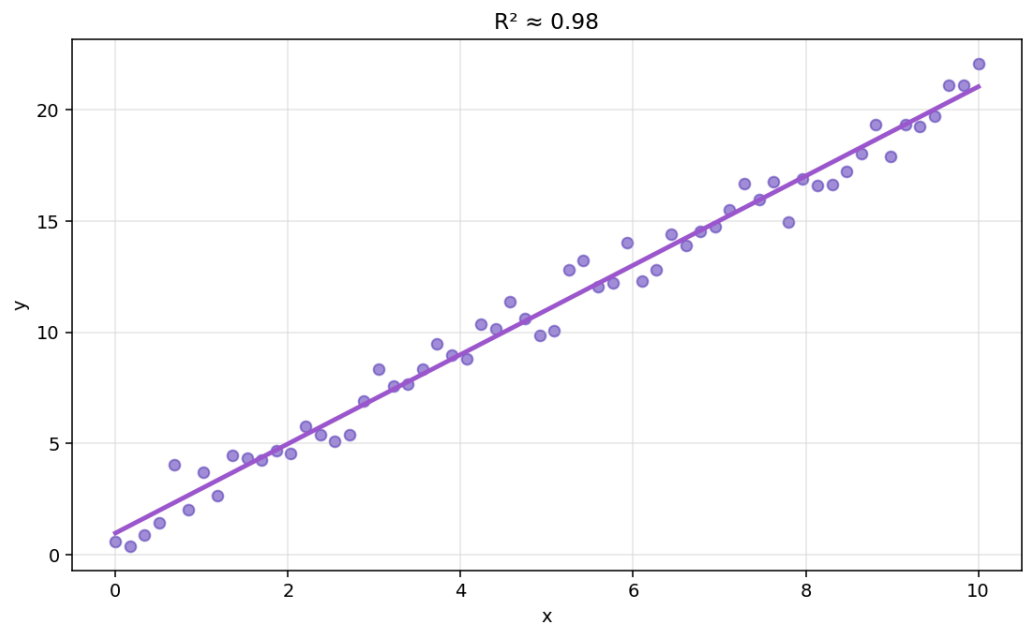
.For each additional 1 ²*m*, price increases by 2.5 (holding other variables constant)

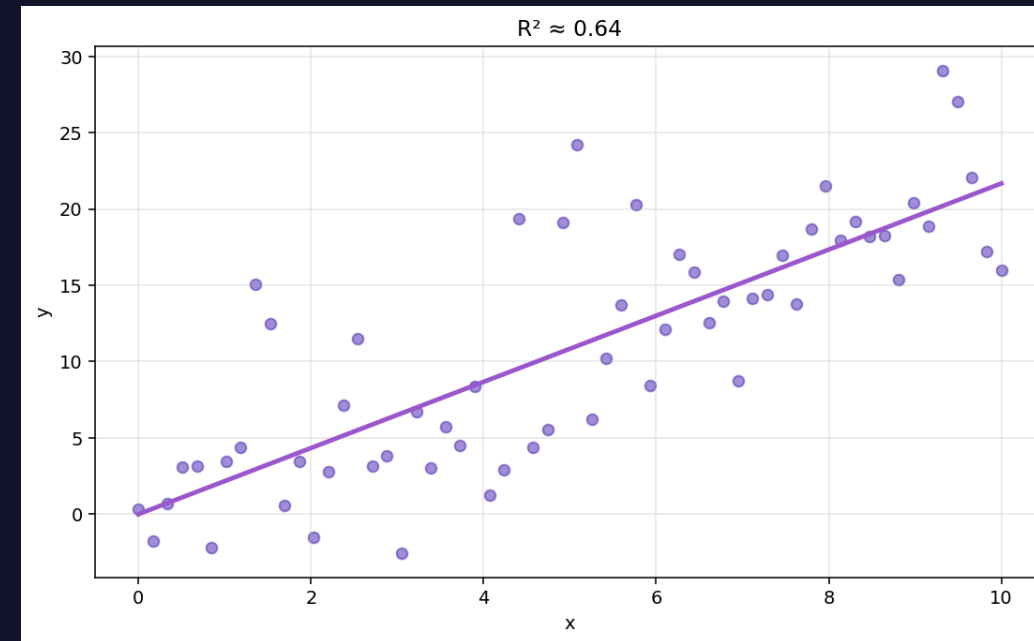
.For each additional bedroom, price increases by 15 (holding other variables constant)

Table for Clarification			
Predicted Price (Y in \$1000)	X_2 (Bedrooms)	X_1 (Size in M^2)	House
$330 = 2(15 +)100(2.5 + 50)$	2	100	A
$395 = 3(15 +)120(2.5 + 50)$	3	120	B
$265 = 1(15 +)80(2.5 + 50)$	1	80	C

.is a goodness-of-fit metric in regression. It measures how much variation in y is explained by model inputs 2R

$$\frac{resSS}{totSS} - 1 = ^2R$$





.Larger 2R means better explanatory power

.means perfect fit; $^2R = 0$ means mean-level prediction performance $1 = ^2R$

Formulas

$$\sum_{i=1}^n (\hat{y}_i - y_i)^2 = SS_{res}$$

Formulas (continued)

$$\sum_{i=1}^n (y_i - \bar{y})^2 = SS_{tot}$$

What Do These Terms Mean	
DESCRIPTION	TERM
.Residual Sum of Squares: squared distance between actual values and predictions	SS_res
.Total Sum of Squares: squared distance between actual values and the mean of y	SS_tot

.In multiple regression, adding predictors often increases 2R even when new variables are not useful

$$\frac{n-1}{n-p-1} (^2R - 1) - 1 = {}^2R$$

.Adjusted 2R penalizes unnecessary predictors

.It is usually a better metric for comparing models with different numbers of predictors

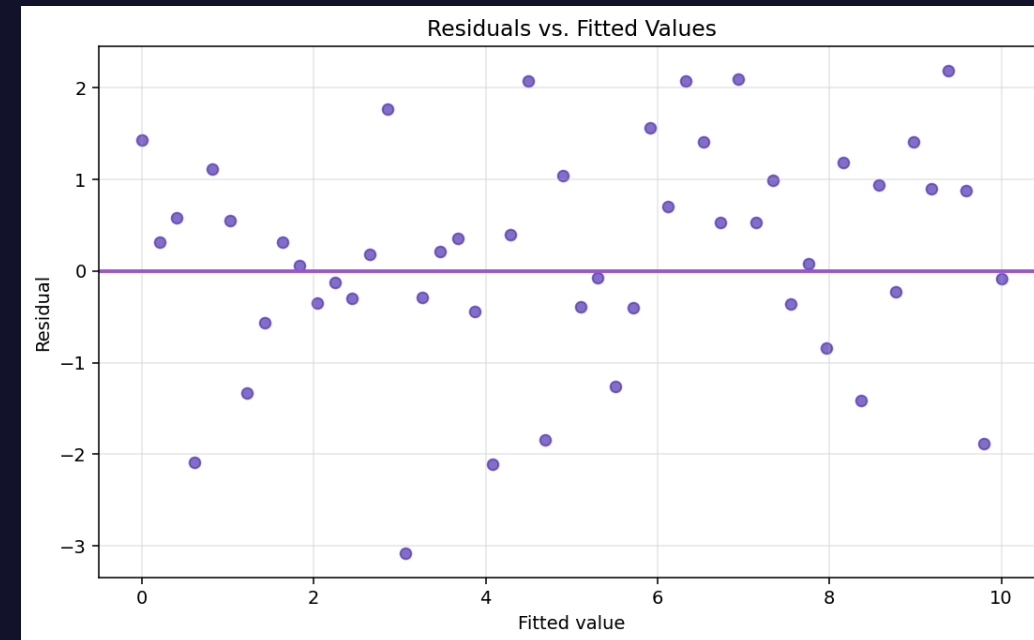
Quick Example

INTERPRETATION	2R ADJUSTED	2R	PREDICTORS	MODEL
Strong and efficient model	0.81	0.82	$^2x, ^1x$	A
Higher 2R , but worse after penalty	0.79	0.83	$^4x, ^3x, ^2x, ^1x$	B

.Model B looks better by 2R , but Adjusted 2R shows extra variables are likely not helpful

.Linear regression is most reliable when core assumptions are approximately satisfied

IF VIOLATED	MEANING	ASSUMPTION
Biased predictions	.Relationship between y and each x is roughly linear	Linearity
Unstable standard errors	.Error variance is roughly constant across fitted values	Homoscedasticity
Inference becomes less reliable	.Residuals are approximately normal	Normality of Errors
Biased significance tests	.Observations/errors are independent	Independence
Coefficients become unstable	.Predictors are not highly redundant	No Severe Multicollinearity
Distorted regression line	.Extreme points do not dominate the fit	Limited Outlier Influence



PRACTICAL CHECK	SIMPLE EXAMPLE	ASSUMPTION
Scatter plot / residual plot	.Sales increase approximately linearly with ad spend	Linearity
Residuals vs fitted values	.Prediction errors are similar for low and high sales values	Homoscedasticity
Histogram / Q-Q plot of residuals	.Most residuals are near zero, few at extremes	Normal Errors
Study design / Durbin-Watson for time data	.Customer records are independent	Independence
Correlation matrix / VIF	.TV_ads and radio_ads are not near-duplicates	Low Multicollinearity
Cook's distance / leverage diagnostics	.One extreme house price should not define the whole model	Outlier Control

.If one assumption is violated, use transformations, robust methods, feature engineering, or a different model



.Categorical variables are converted into binary columns (0/1)

Ordinal

One-hot

Dummy

Dummy Variable Trap

.If all dummy columns are included with an intercept, perfect multicollinearity appears

$$_1D - 1 = _2D$$

.Coefficients become unstable

.Interpretation becomes unreliable

.Statistical tests may be misleading

Example: State

STATE	NEW YORK ₁ D	CALIFORNIA ₂ D
New York	1	0
California	0	1



.Drop one dummy column and keep it as the baseline category

Ordinal

One-hot

Dummy

$${}_1b_4D + {}_3b_3x + {}_2b_2x + {}_1b_1x + {}_0b = \text{Profit}$$

.If we drop California, then ${}_1D = 1$ means New York and ${}_1D = 0$ means California baseline

.This keeps category information while avoiding multicollinearity

Interpretation Example		
STATE MEANING	${}_1D$	CASE
New York	1	A
California (reference)	0	B



.Model-building methods select the most relevant predictors to balance simplicity, interpretability, and predictive accuracy

BEST USE CASE	CORE IDEA	METHOD
Strong prior knowledge that all variables matter	.Use all predictors directly	All-in
Many candidate predictors	.Start full and remove least significant variables	Backward Elimination
Need a compact model from scratch	.Start empty and add most significant variables	Forward Selection
Flexible search for balanced model	.Add and remove dynamically	Bidirectional (Stepwise)

Backward Elimination)1

.Set significance level (e.g., $SL = 0.05$)

.Fit model with all predictors

.Remove highest p-value if $p > SL$

.Refit and repeat

Forward Selection)2

.Start with no predictors

.Add variable with lowest p-value below SL

.Continue one-by-one

.Stop when no variable qualifies

Bidirectional (Stepwise))3

.Add significant variables and remove non-significant ones each iteration

Mini Example

SELECTED ACTION	CANDIDATE PREDICTORS	ITERATION
Full model	x_4, x_3, x_2, x_1	Start
x_4 Remove	highest p-value: $x_4 = 0.42$	1
x_3 Remove	highest p-value: $x_3 = 0.11$	2
Keep final model	with $p < 0.05$ x_2, x_1	Final

$$\text{Reject } H_0 \Rightarrow 0.05 > 0.03$$

.If $p < \alpha$: reject H_0 (statistically significant)

. H_0 If $p \geq \alpha$: fail to reject

MEANING	CONCEPT
No effect / no relationship	H_0 (Null Hypothesis)
Probability of observing current result (or more extreme) if H_0 is true	p-value
Maximum tolerated Type I error probability	Significance Level (alpha)

.So the predictor is considered significant at the 5% level



Standard Choice

. $SL = 0.05$ is widely used because it balances sensitivity and false-positive risk

$$\text{Confidence Level} - 1 = SL$$

. $\alpha = 0.05$ implies a 5% false-positive rate under the null hypothesis

.It is a convention, not a law — adjust for domain risk (medical vs A/B tests)

.Report effect size and confidence intervals alongside p-values

STRICTNESS	CONFIDENCE LEVEL	SL
Less strict	90%	0.10
Standard	95%	0.05
More strict	99%	0.01

How to Choose in Practice

SUGGESTED SL	SCENARIO
0.01	High-stakes decisions (medicine, safety, finance)
0.05	General modeling and reporting
0.10	Exploratory analysis

.Start with SL = 0.05, then adjust based on domain risk, dataset size, and model purpose

.Polynomial Regression extends linear regression to model non-linear relationships by adding polynomial terms

$b_1x + {}_0b$

= Linear regression: y

$\dots + {}^3b_3x + {}^2b_2x + b_1x + {}_0b$

= Polynomial regression: y

.It is still linear in parameters because coefficients remain linear



.Simple linear regression fits only straight lines, but many real-world relationships are curved

Population growth over time

Disease progression

Sales and economic trends

Rainfall vs crop yield relationships

.Curved or U-shaped data is often underfit by a straight line

$$b_d x^d + \dots + b_2 x^2 + b_1 x + b_0 = y$$

DESCRIPTION	STEP
Start with input feature x	1
Create transformed terms $x^2, x^3, \dots$	2
Fit linear regression on transformed features	3
Estimate coefficients $b_0, b_1, b_2, \dots$	4
Use the fitted equation to predict y	5

.d is the polynomial degree

Choosing the Polynomial Degree

.The polynomial degree controls model complexity

.Use cross-validation

. R^2 Compare adjusted

.Compare validation/test error

EFFECT	DEGREE CHOICE
Underfitting (model too simple)	Too low
Overfitting (fits noise, weak generalization)	Too high
Better bias-variance trade-off	Balanced degree

When to Use Polynomial Regression



Good Use Cases

.Data shows clear curved trends

.You need more flexibility than a straight line

.You still want interpretable behavior

Applications

Forecasting demand and sales trends

Environmental modeling (rainfall vs yield)

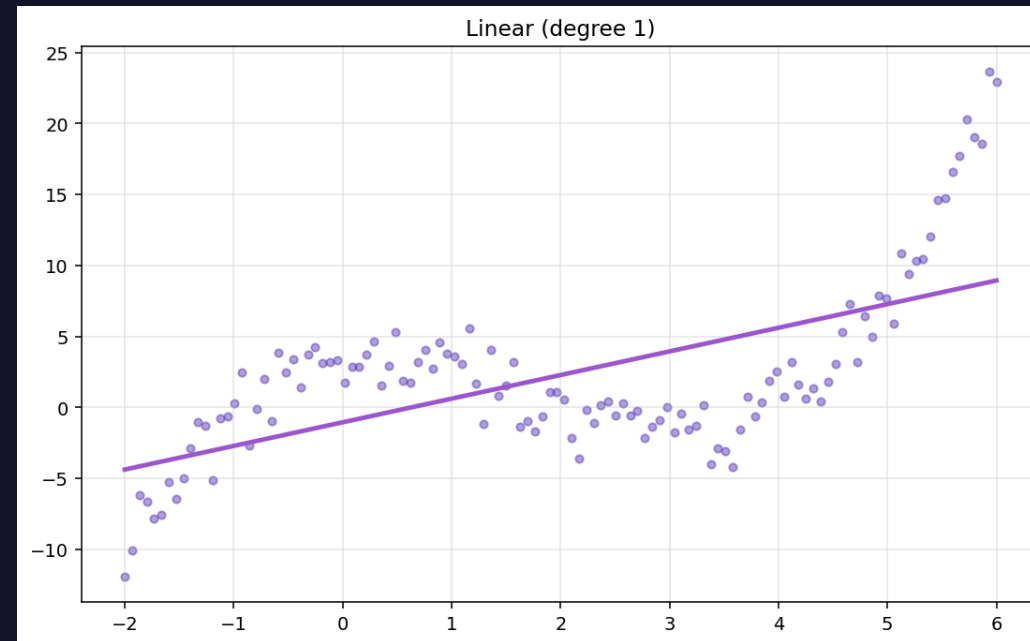
Medical progression patterns

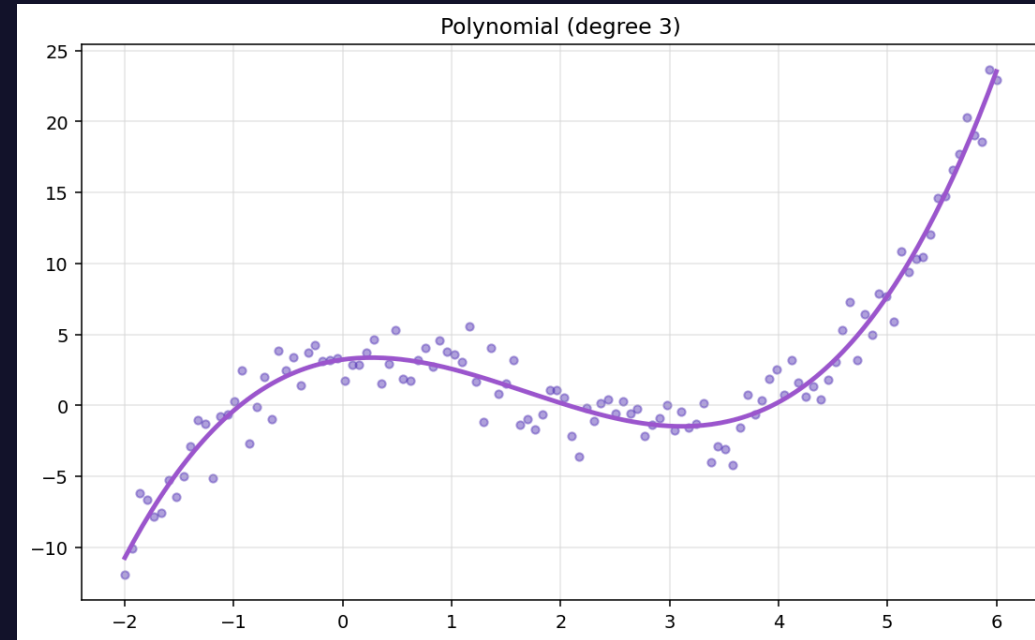
Engineering curves (stress-strain relationships)

EXPLANATION	LIMITATION
High-degree models may memorize noise	Overfitting risk
Extreme points can shift the curve strongly	Outlier sensitivity
Predictions outside training range can be unreliable	Extrapolation weakness
Some tasks are better handled by tree-based or neural models	Limited flexibility for very complex patterns

.Use polynomial regression as a baseline, then compare with alternatives as complexity grows

.This section compares linear and polynomial fits, and shows how model behavior changes with polynomial degree





.Underfitting: degree too low — high bias

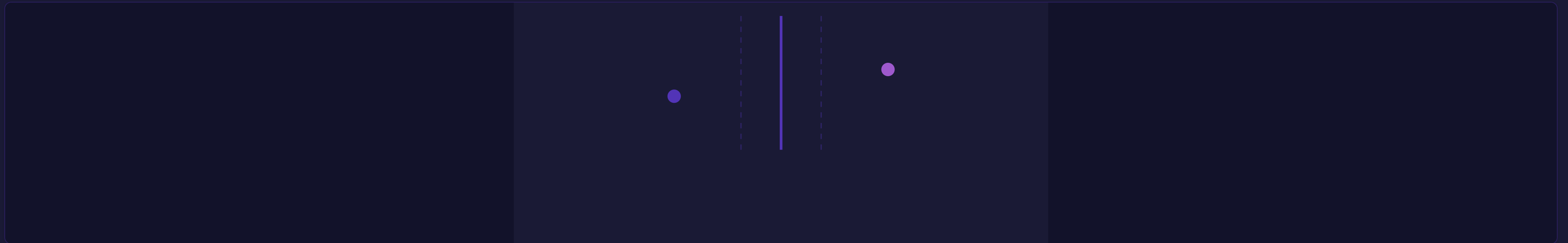
.Overfitting: degree too high — high variance

.Use validation curves to pick degree — not training error alone



?What is SVR

.Support Vector Regression (SVR) is the regression version of SVM. It predicts a continuous target with a flat function and a tolerance margin



Used in disease progression prediction

Used in engineering curves (e.g., stress-strain)

Used in demand and trend forecasting

.SVR uses an epsilon-insensitive zone where small errors are ignored

$$b + w^T \phi(x) = f(x)$$

.maps input to a (possibly) higher-dimensional space $\phi(x)$

.With kernels, we avoid computing $\phi(x)$ explicitly

Optimization Objective (Soft-Margin SVR)

$$\min_{w,b,\xi} \left(\sum_{i=1}^n C(\xi_i + \frac{1}{2} \|w\|^2) \right)$$

Constraints

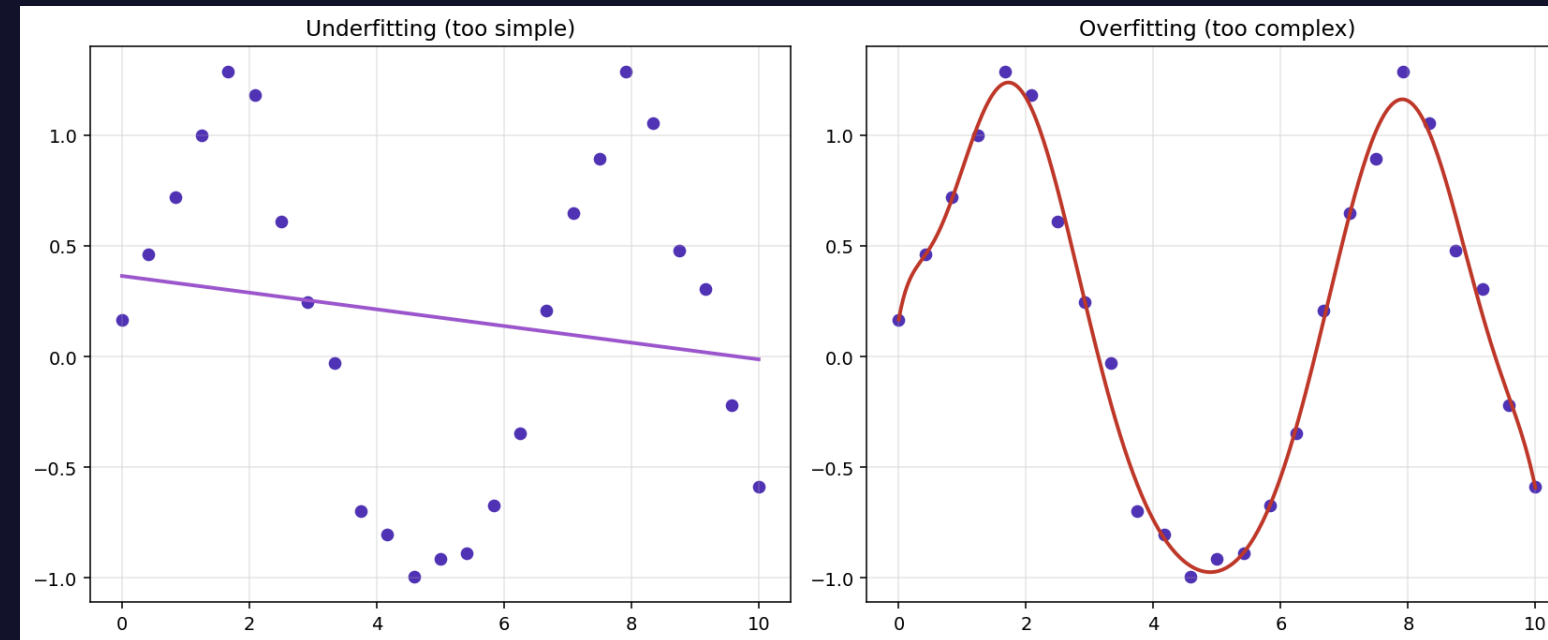
$${}_i\xi + \varepsilon \geq (b + w^T \phi(x_i)) - {}_iy$$

Constraints (continued)

$${}_i^*\xi + \varepsilon \geq {}_iy - (b + w^T \phi(x_i))$$

$$0 \leq {}_i^*\xi, {}_i\xi$$

Hyperparameters	
ROLE	HYPERPARAMETER
Width of the no-penalty tube around prediction	epsilon
Penalty strength for points outside the tube	C
Controls curve shape (linear, RBF, polynomial)	Kernel





.Use feature scaling before SVR

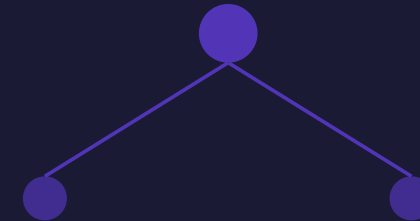
.Start with RBF kernel, then tune C, epsilon, and kernel parameters

SVR	TRADITIONAL REGRESSION	ASPECT
Ignores errors inside epsilon-tube	Minimizes all residuals	Error handling
More robust when tube and C are tuned	Can be sensitive	Outlier sensitivity
Uses kernels directly	Needs explicit feature engineering	Non-linear handling
Can become expensive for very large sample sizes	Usually faster for very large data	Large datasets

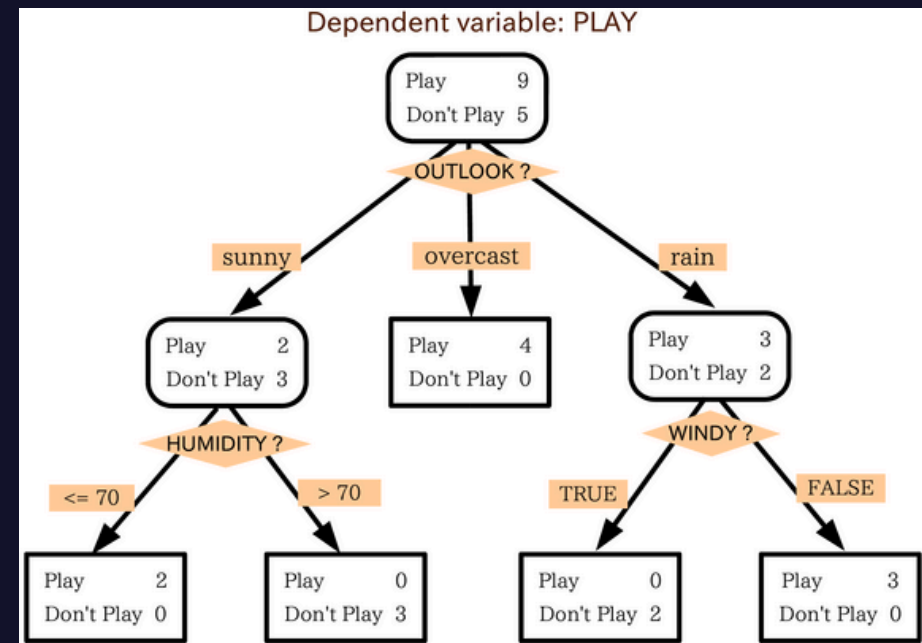


?What is Decision Tree Regressor

.Decision Tree Regressor predicts continuous values by splitting feature space into rule-based regions and assigning a constant value in each region



MSE / Gini split



.Captures non-linear relationships

.Easy to interpret if tree depth is controlled

.Handles complex feature interactions

.Internal node: decision rule (e.g., $x \leq 4.5$)

.Branch: outcome of the rule

.Leaf node: predicted value (usually region mean)

CART Perspective

TYPICAL CRITERION	TARGET TYPE	TREE TYPE
Gini / Entropy	Categorical target	Classification Tree
MSE (or MAE)	Continuous target	Regression Tree

.This section focuses on the Regression Tree part of CART

.CART picks the split that maximally reduces MSE (regression) or Gini/entropy (classification)

.Greedy splits are fast but not globally optimal

.Depth and min-samples-leaf control overfitting on small datasets

Node MSE

$$\frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y})^2 = \text{MSE}(S)$$

Split Score

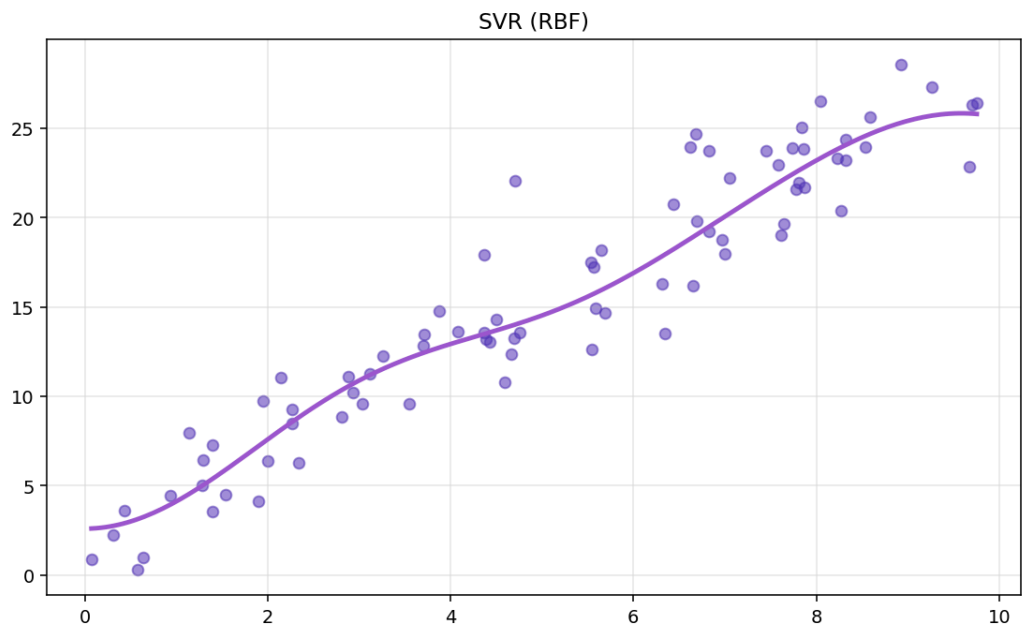
$$\text{MSE}(S_R) \frac{|_RS|}{|S|} + \text{MSE}(S_L) \frac{|_LS|}{|S|} = \text{splitMSE}$$

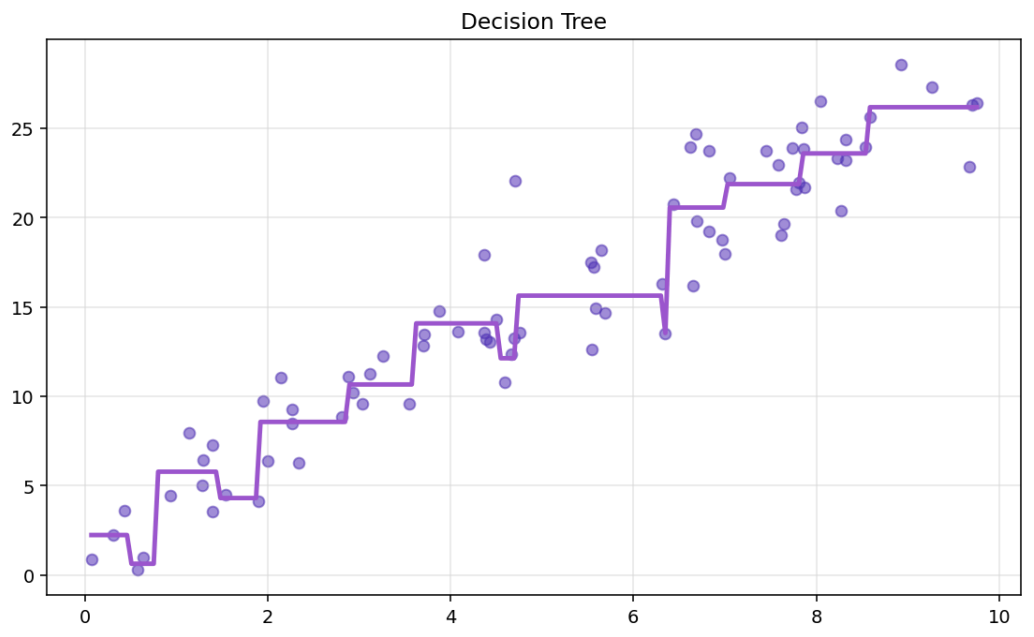
Prediction at Leaf

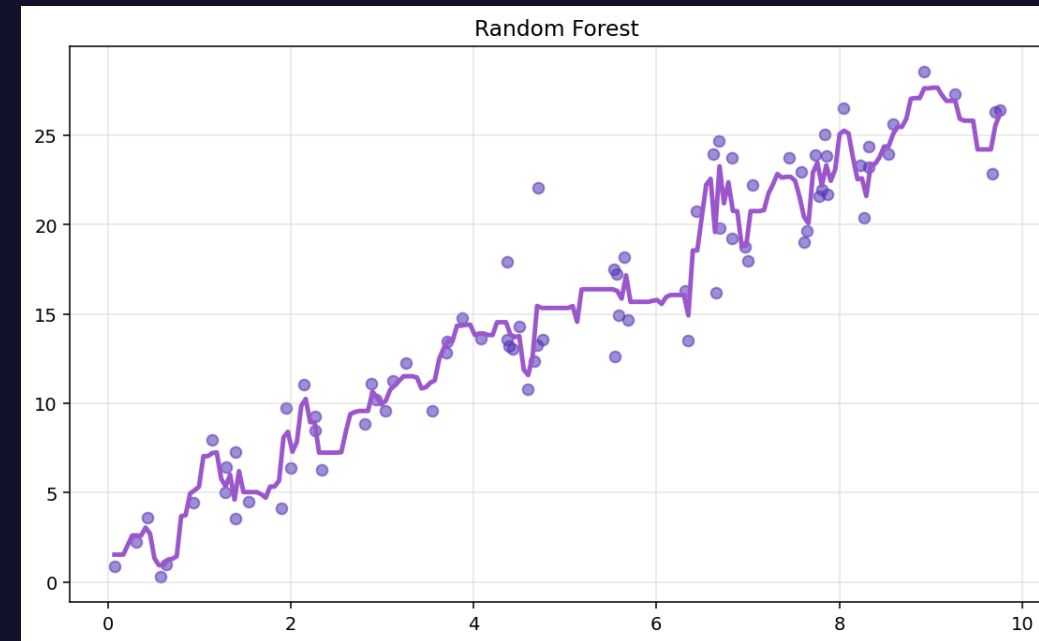
$$y_i \sum_{leaf \in S} \frac{1}{|leafS|} = \hat{y}_{leaf}$$

.Choose the split with the minimum weighted MSE









.Start with all data at the root node

.Evaluate candidate splits across features

.Compute weighted MSE for each split

- .Select the split with minimum error
- .Repeat recursively on child nodes
- .Stop based on rules (max depth, min samples)
- .Predict using leaf mean value

Important Hyperparameters	
EFFECT	HYPERPARAMETER
Controls model complexity	max_depth
Minimum samples required to split a node	min_samples_split
Minimum samples in each leaf	min_samples_leaf
Limits number of terminal regions	max_leaf_nodes

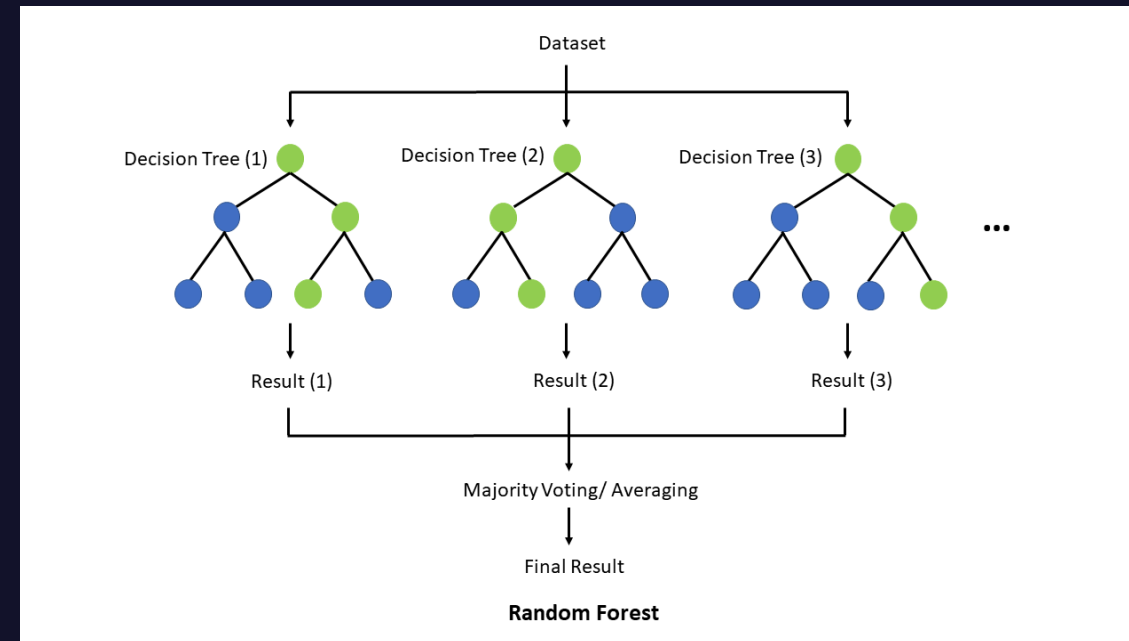
.These settings reduce overfitting and improve generalization



?What is Random Forest Regressor

.Random Forest Regression is an ensemble method that combines many decision trees and averages their predictions

$$(x)_t \hat{y} \sum_{t=1}^{trees} \frac{1}{trees} = (x)_{RF} \hat{y}$$

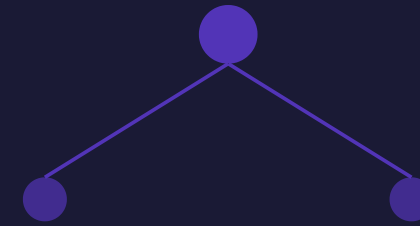


.Sample training data with bootstrap

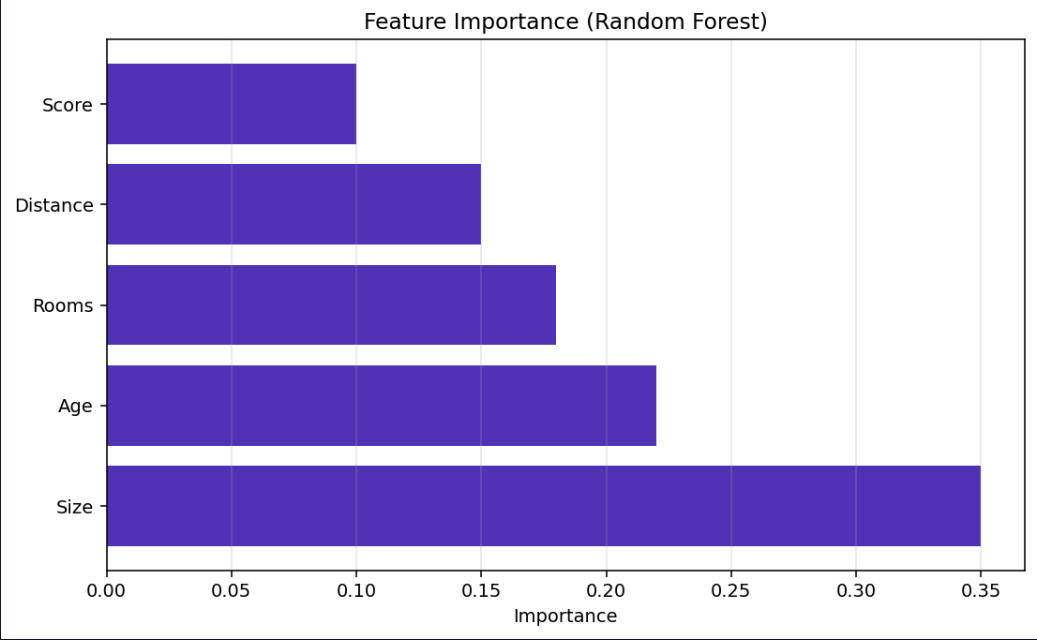
.Build many trees on different samples

.Predict with all trees and average outputs

.Averaging reduces variance and usually improves generalization



MSE / Gini split



RANDOM FOREST REGRESSOR	DECISION TREE REGRESSOR	ASPECT
Ensemble of many trees	Single tree	Model type
Trees split by MSE, then averaged	MSE per split	Split criterion
Lower (variance reduction)	Higher	Overfitting risk
More stable	Sensitive to data changes	Stability
Mean of all tree outputs	Output of one tree	Final prediction
Medium	High	Interpretability

.Use a single tree for interpretability; use random forest for stronger performance

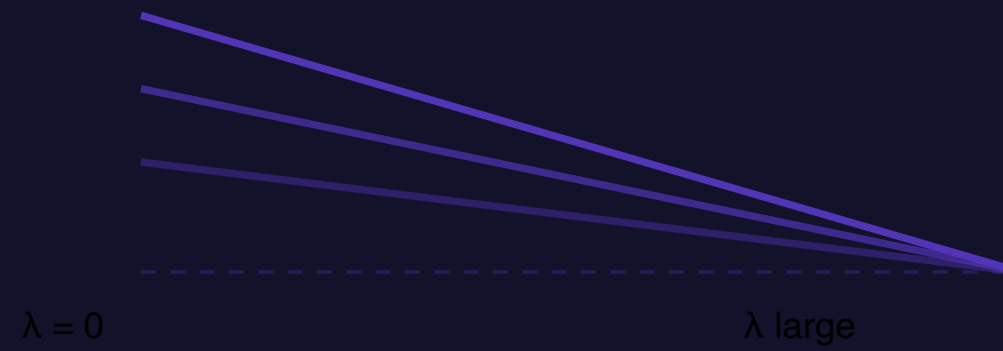
Key Performance Metrics		
WHAT IT TELLS US	FORMULA	METRIC
Average prediction error magnitude	$ \hat{y} - y \sum \frac{1}{n}$	MAE
Penalizes large errors more	$^2(\hat{y} - y) \sum \frac{1}{n}$	MSE
Error in original target scale	$\sqrt{^2(\hat{y} - y) \sum \frac{1}{n}}$	RMSE
Fraction of variance explained	$\frac{resSS}{totSS} - 1$	2R

.Evaluate using more than one metric (e.g., RMSE with 2R) for balanced judgment



Overfitting Problem

.When a model is too flexible, it may fit noise instead of true patterns



Coefficients shrink with regularization strength

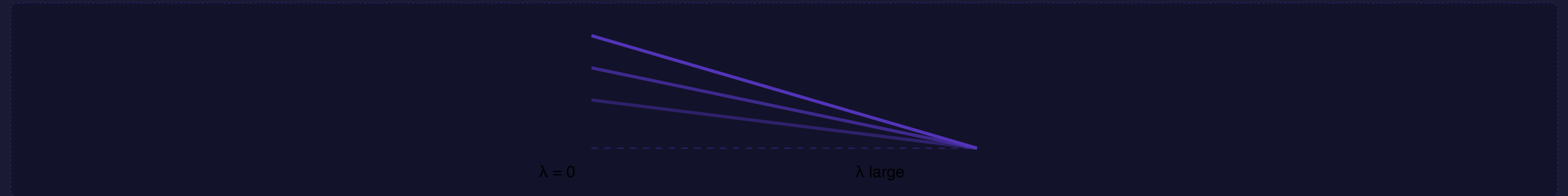
$$\underbrace{\Omega(\beta) \cdot \lambda}_{\text{penalty}} + \underbrace{\sum_{i=1}^n \frac{1}{2} (\hat{y}_i - y_i)^2}_{\text{data loss}} \min_{\beta}$$

Very low training error

Weak test performance

High variance and unstable predictions

.Regularization controls complexity by penalizing large coefficients



Coefficients shrink with regularization strength

.Ridge (L2): shrinks all coefficients — good when many features correlate

.Lasso (L1): can zero out coefficients — embedded feature selection

Without Regularization

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

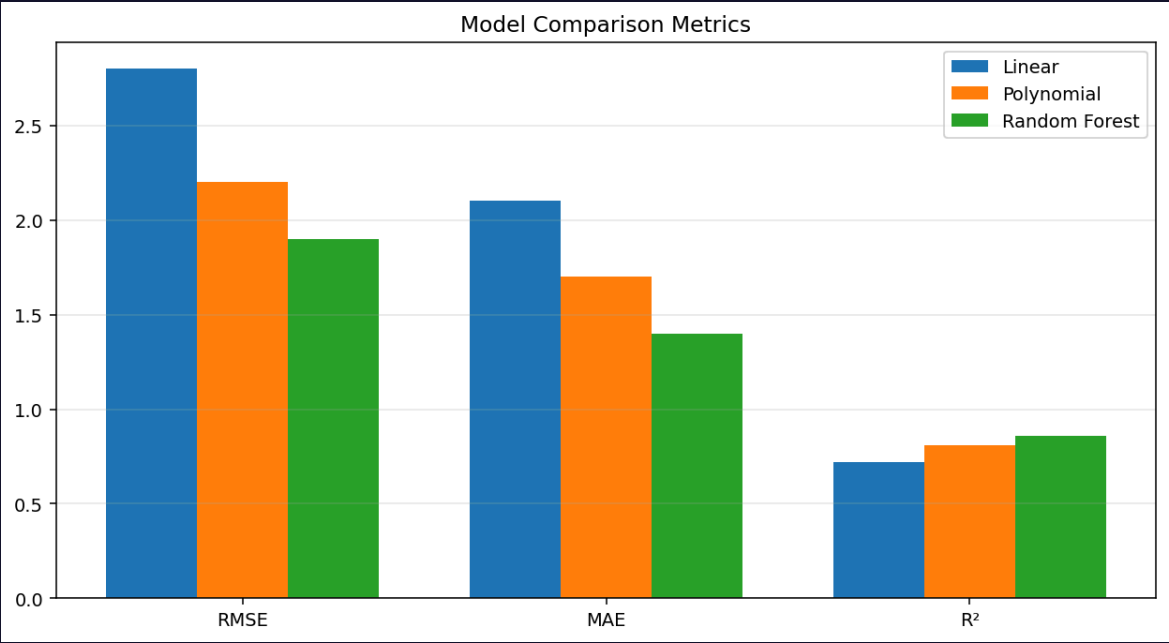
Ridge Regression (L2)

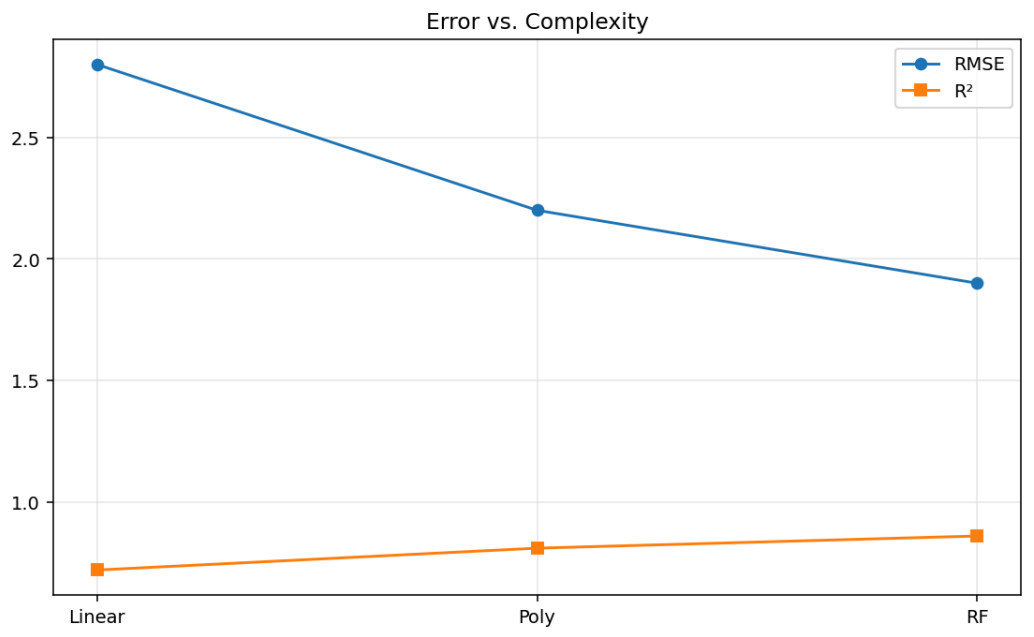
$$\min_{\beta} \lambda \sum_{j=1}^m \beta_j^2 + \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

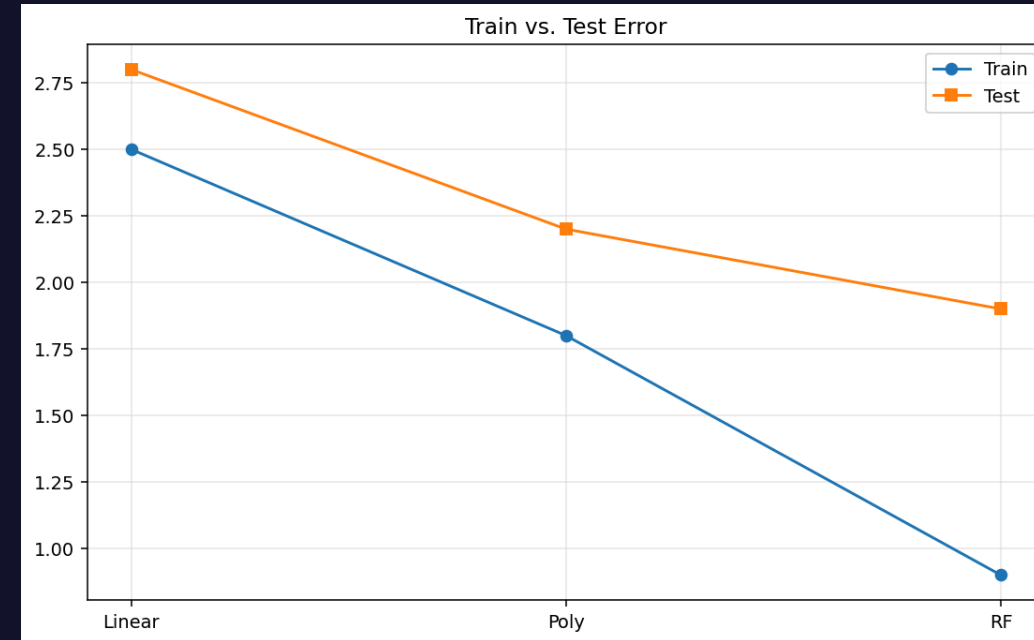
Lasso Regression (L1)

$$\min_{\beta} \lambda \sum_{j=1}^m |\beta_j| + \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\frac{1}{2}\lambda_2 \sum_{j=1}^m \beta_j^2 + \lambda_1 \sum_{j=1}^m |\beta_j| + \frac{1}{2}(\hat{\mathbf{y}} - \mathbf{y})^T \sum_{i=1}^n \min_{\beta} \left(\frac{1}{\lambda_1} \left(\frac{\lambda_2}{2} \beta^2 + \lambda_1 |\beta| \right) + \mathbf{X}_{i\cdot}^T (\hat{\mathbf{y}} - \mathbf{y}) \right)$$







.Ridge: many small/medium useful features

.Lasso: only few important features expected

.Elastic Net: correlated features and need both stability and sparsity

FEATURE SELECTION	MAIN BEHAVIOR	PENALTY TYPE	METHOD
No	Shrinks coefficients smoothly	L2 (sum beta_j^2)	Ridge
Yes	Shrinks and can set some coefficients to zero	L1 (sum beta_j)	Lasso
Yes	Combines shrinkage + selection	L1 + L2	Elastic Net

Regression Models - Advantages and Disadvantages

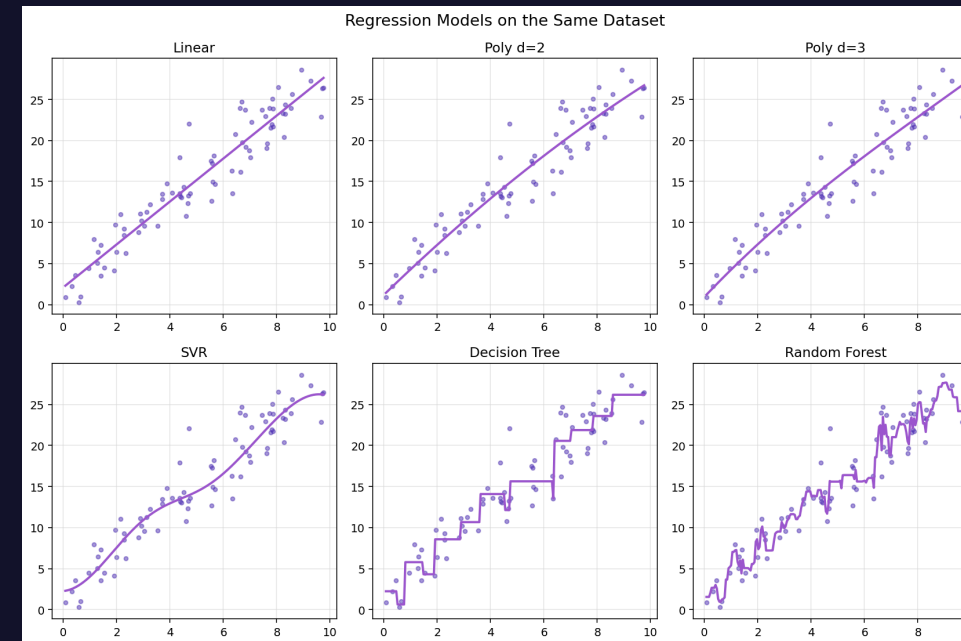
DISADVANTAGES	ADVANTAGES	MODEL
.Assumes linear relationship and is sensitive to outliers	.Simple, fast, easy to interpret, works well for linear patterns	Linear Regression
.Choosing degree is sensitive; high degrees may overfit	.Captures curved/non-linear relationships better than linear regression	Polynomial Regression
.Requires feature scaling and careful hyperparameter tuning; slower on large datasets	.Handles non-linear patterns with kernels and can be robust to noise with proper tuning	SVR
.Can overfit easily and may be unstable with small data changes	.Interpretable rules, captures non-linear interactions, no feature scaling required	Decision Tree Regression
.Less interpretable and can be computationally heavier	.Higher accuracy and stability, reduces overfitting by averaging many trees	Random Forest Regression

Model Summary (Name + Equation)

CORE EQUATION	MODEL
$b_1x + b_0 = \hat{y}$	Linear Regression
$b_d x^d + \dots + b_2 x^2 + b_1 x + b_0 = \hat{y}$	Polynomial Regression
$b + w^T \phi(x) = f(x)$	SVR
$\hat{y}_{leaf} = \sum_{i \in S} \frac{1}{ S } y_i$	Decision Tree Regression
$\hat{y}_{RF}(x) = \frac{1}{N} \sum_{t=1}^N \hat{y}_t(x)$	Random Forest Regression

.If interpretability is priority: Linear Regression or Decision Tree. If predictive performance is priority: Random Forest

.This slide compares model behavior on the same dataset: Linear Regression, Polynomial Regression, SVR, Decision Tree Regression, and Random Forest Regression



Model families at a glance

WATCH OUT	STRENGTH	MODEL
Nonlinear patterns	Interpretable, fast	Linear / Ridge
Slow on huge data	Margin-based, kernels	SVR
Extrapolation	Nonlinear, little scaling	Tree / RF

.Visual output from Python code is shown below

Section Takeaways

.R² measures explained variance — not accuracy or causation .1

.OLS minimizes squared errors; outliers can dominate the fit .2

.Regularization (Ridge/Lasso) trades bias for stable coefficients .3

?**Think:** When would a linear model be misleading even with high R²

Classification Basics

Threshold tuning

Decision boundaries

K-NN

Logistic regression

slides · ~21 min 13 ▶

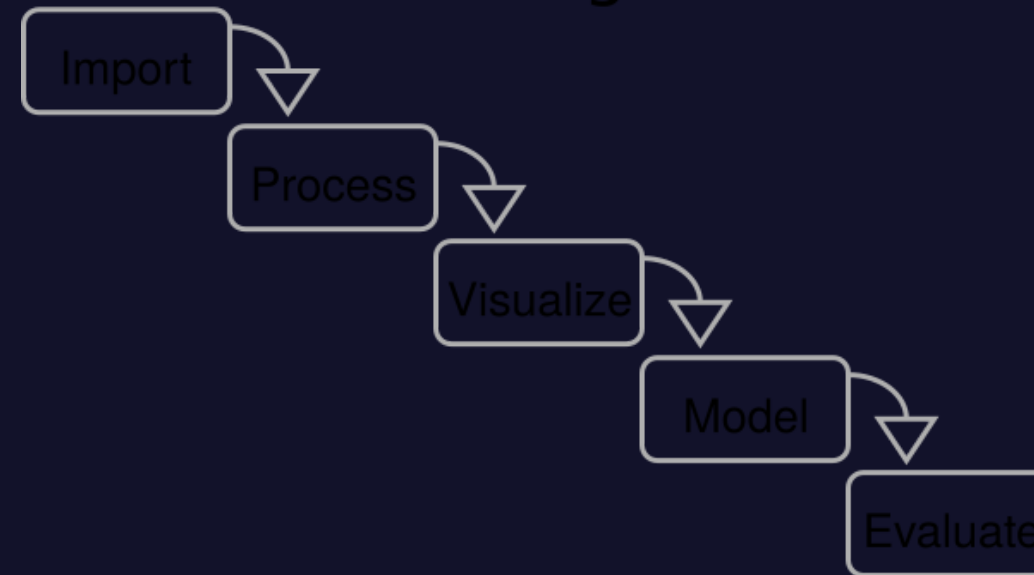
?Where Are We in the Bootcamp

.Move from continuous targets to discrete decisions with calibrated probabilities

- S1 Foundations & Preprocessing •
- S2 Regression Models •
- S3 Classification Basics •
- S4 Naive Bayes & Trees •
- S5 SVM & Kernels •
- S6 Clustering & PCA •
- S7 Deep Learning •
- S8 NLP Fundamentals •
- S9 Tokenization •
- S10 Text Analysis & NER •
- S11 Language Modeling •
- S12 Embeddings & RNNs •
- S13 Seq2Seq & NMT •
- S14 Generative AI •
- S15 RAG Systems •
- S16 MLOps •

.Classification is a supervised learning task used to assign a new observation to a predefined category based on learned patterns from labeled data

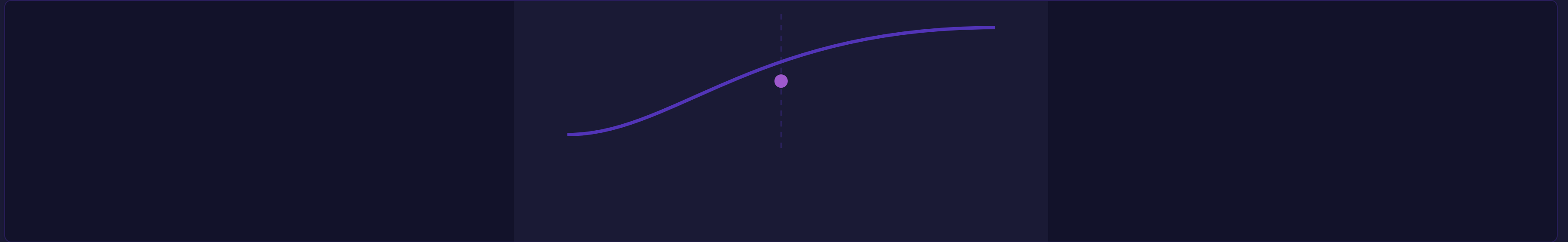
Machine learning workflow



.Output is a class label (e.g., 0/1, Yes/No)

.Used in spam detection, medical diagnosis, fraud detection, and customer churn prediction

.Logistic Regression is a supervised algorithm for binary classification that estimates the probability of class membership



.Produces probabilities in the range $[0, 1]$

.Final class is determined using a threshold (commonly 0.5)

.Models the relationship between features and log-odds of the target

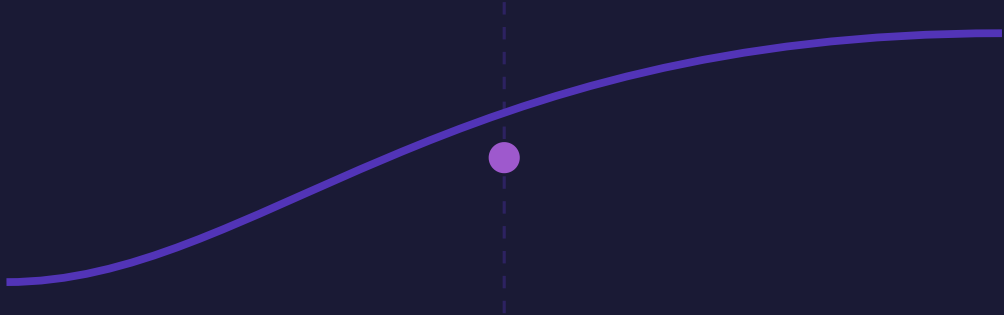
?Why Not Linear Regression for Classification

.Linear regression predicts continuous values rather than discrete classes

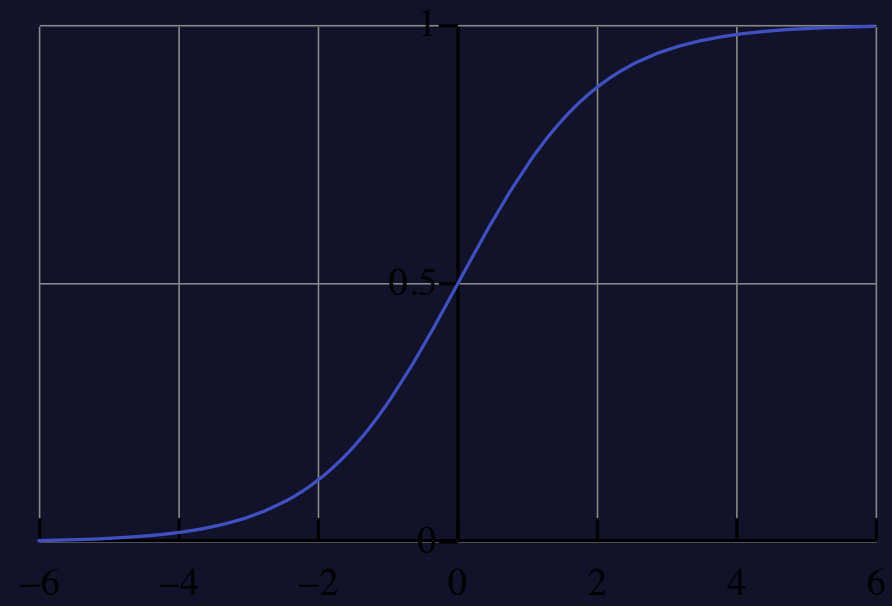
.Predicted values can be less than 0 or greater than 1, which is invalid for probabilities

.It does not naturally provide a robust classification boundary

.Logistic Regression solves this by mapping the linear score to probability through a sigmoid function



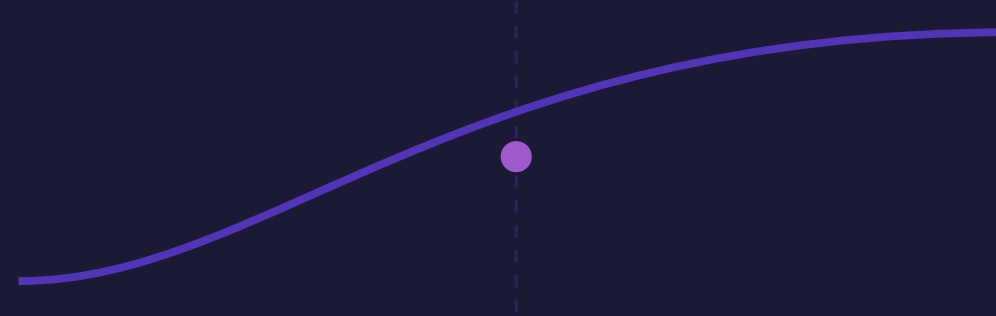
$$\beta^T x + \beta_0 = z, \frac{1}{1 + e^{-z}} = \sigma(z) = P(y = 1 | x)$$



.Sigmoid converts any real-valued score into a valid probability

.If probability ≥ 0.5 , predict class 1; otherwise class 0

.Threshold can be adjusted based on business/clinical needs



$$\left(\frac{p}{p-1} \right) \log = \text{Logit}(p) \quad , \frac{p}{p-1} = \text{Odds}$$

.Observations should be independent

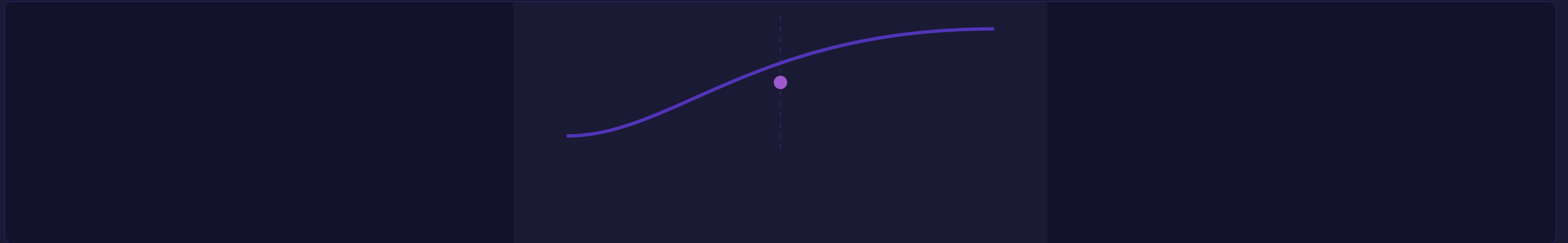
.Predictors should not have severe multicollinearity

.Relationship should be approximately linear in the log-odds space

.Logistic Regression parameters are estimated by maximizing the likelihood of observing the true class labels

.Different curves correspond to different parameter values

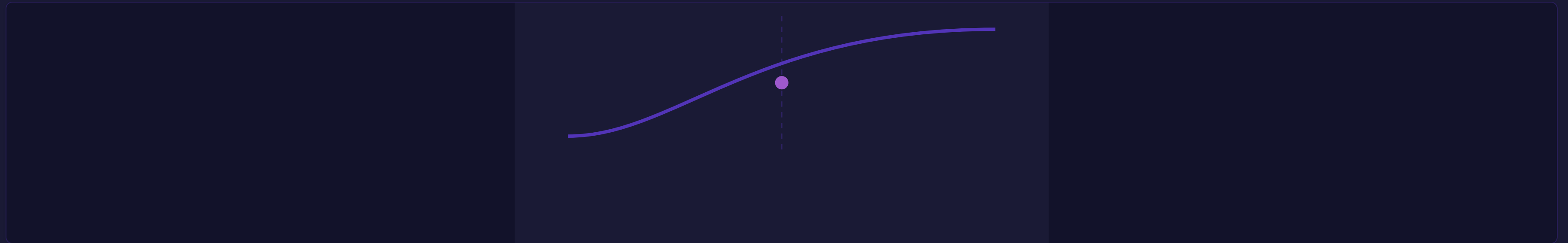
.The optimal model is the one with the highest likelihood (or lowest log-loss)

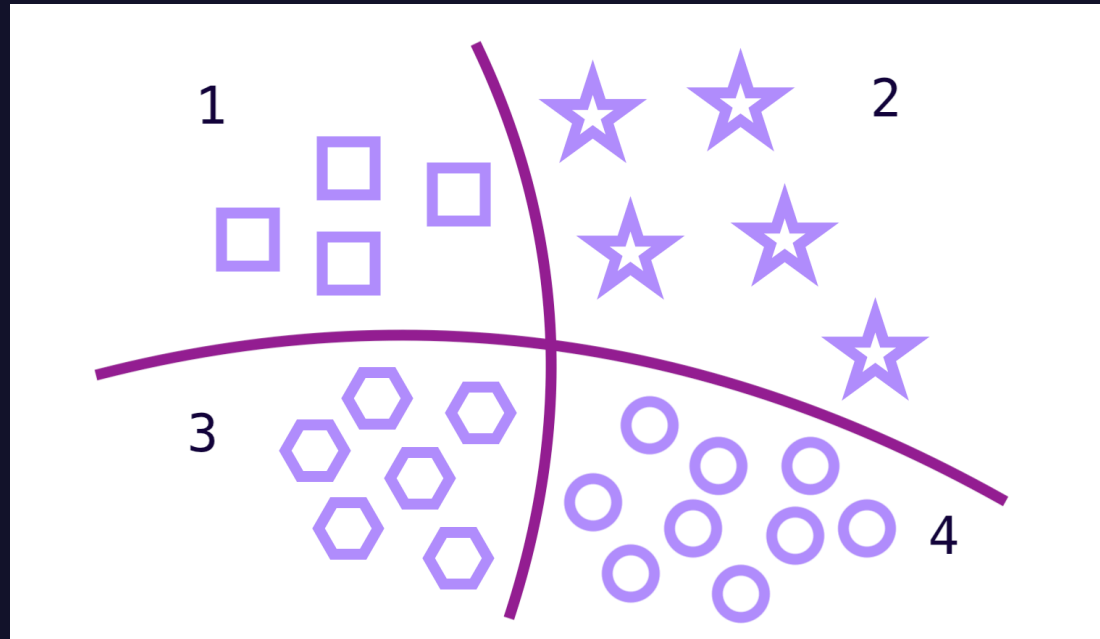


.Strength: calibrated probabilities and clear feature coefficients

.Limit: linear decision boundary in feature space (use polynomials or other models for complex boundaries)

Strengths	
	.High interpretability of coefficients and odds ratios
	.Fast training and strong baseline for binary tasks
Limitations	
	.May underperform on complex non-linear boundaries
	.Sensitive when classes strongly overlap



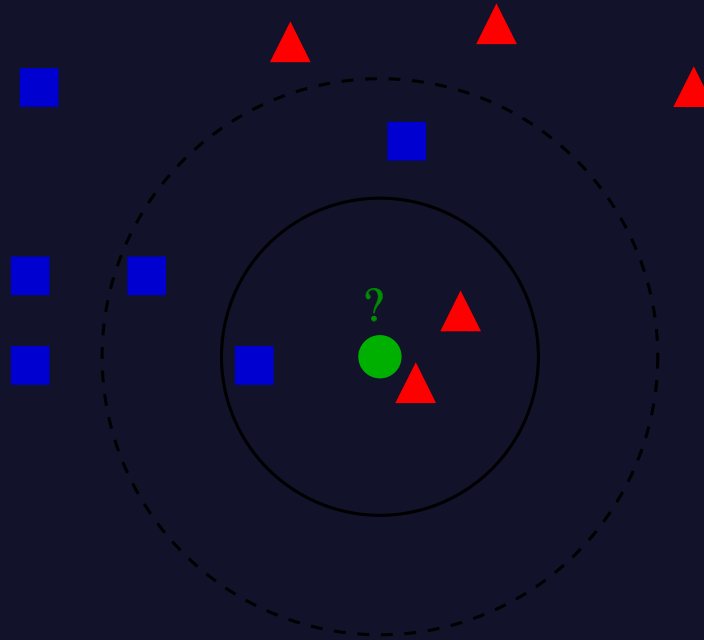


.Multinomial (Softmax) Logistic Regression directly handles multiple classes

.One-vs-All (OvA) trains one binary classifier per class

.Final class is selected by highest predicted probability

.K-NN is an instance-based, non-parametric algorithm that classifies a sample using the majority class among its nearest neighbors



.Choose K

.Compute distance to all training points

.Select K nearest points and apply majority voting

.Small K: lower bias, higher variance (sensitive to noise)

.Large K: smoother boundary, may miss local structure

.Common metrics: Euclidean, Manhattan, Hamming

.Use cross-validation to choose K and scale numeric features before distance-based modeling

.Strength: no training phase — lazy learning, simple baseline

.Limit: slow inference, curse of dimensionality, needs feature scaling

Strengths

.Simple and intuitive

.Effective on smaller, well-separated datasets

.Naturally supports multiclass classification

Limitations

.Prediction can be slow on large datasets

.Sensitive to irrelevant features and class imbalance

.Performance drops in high-dimensional spaces

Section Takeaways

.Logistic regression outputs probabilities — pick thresholds for the business cost .1

.K-NN is lazy learning: performance depends on scale and dimensionality .2

.Confusion matrix beats accuracy on imbalanced data .3

?**Think:** What threshold minimizes cost for your use case

,Naive Bayes, Trees Evaluation

Precision / recall / F1

Random Forest

Decision trees

Naive Bayes

slides · ~22 min 14 ▶

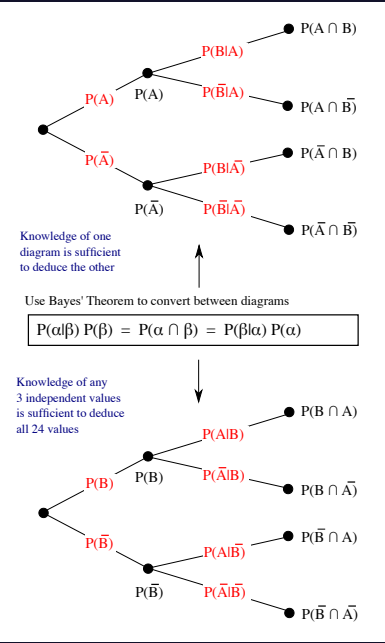
?Where Are We in the Bootcamp

.Combine probabilistic models with interpretable tree ensembles

- S1 Foundations & Preprocessing •
- S2 Regression Models •
- S3 Classification Basics •
- S4 Naive Bayes & Trees •
- S5 SVM & Kernels •
- S6 Clustering & PCA •
- S7 Deep Learning •
- S8 NLP Fundamentals •
- S9 Tokenization •
- S10 Text Analysis & NER •
- S11 Language Modeling •
- S12 Embeddings & RNNs •
- S13 Seq2Seq & NMT •
- S14 Generative AI •
- S15 RAG Systems •
- S16 MLOps •



$$\frac{P(B | A)P(A)}{P(B)} = P(A | B)$$



.P(A|B): posterior probability after observing evidence B

.P(A): prior belief before seeing evidence

.P(B|A): likelihood of evidence if class A is true

.P(B): evidence probability (normalization term)

.This theorem updates beliefs when new evidence appears

$$c = \underset{c \in \mathcal{C}}{\operatorname{arg\,max}} P(c) \prod_{j=1}^m P(x_j = y_j | c)$$

1. Compute class posterior for each class: $P(\text{Class} | \mathbf{X})$

2. Estimate prior and feature likelihoods from training data

3. Compare posterior scores across classes

4. Assign the class with the highest posterior probability

5. For numerical stability, implementations often use log-probabilities



Conditional Independence Assumption

.Naive Bayes assumes features are conditionally independent given the class label

.Simplifies parameter estimation and speeds up training

.Works well in text classification and high-dimensional sparse data

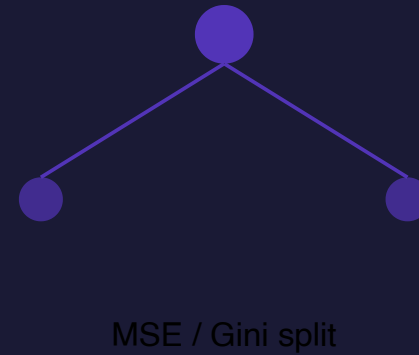
.Can still perform well even when independence is not fully true

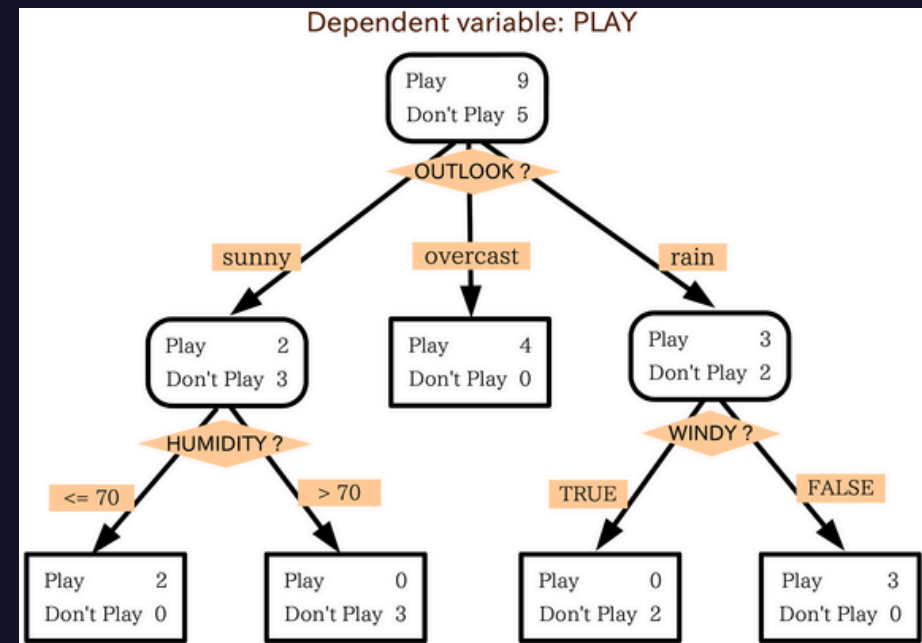
.When comparing classes for the same X , $P(X)$ is constant and can be ignored

- .Handle zero-frequency with smoothing (e.g., Laplace smoothing)
- .Scale and preprocessing depend on feature type and variant

KEY IDEA	TYPICAL DATA TYPE	VARIANT
.Assumes each feature follows a Gaussian distribution	Continuous	Gaussian NB
.Common for document word counts	Count-based	Multinomial NB
.Uses binary presence/absence features	Binary	Bernoulli NB

.Decision Tree Classification predicts class labels by recursively splitting feature space into purer class regions





.Internal nodes represent feature-based decisions

.Branches represent decision outcomes

.Leaf nodes output final class prediction

.Choose split that best separates classes at each node

.Stop splitting using rules like max depth or minimum samples

.Gini impurity and entropy measure node purity — lower is better

.Splits are chosen greedily to maximize information gain

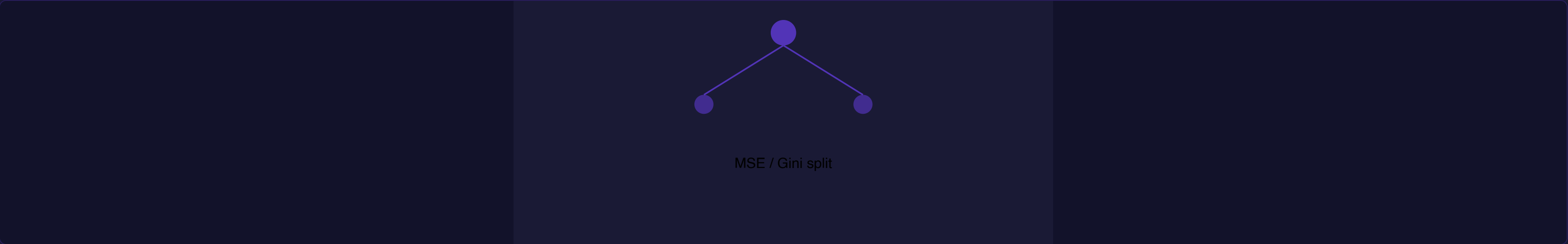
Entropy and Information Gain

$$-\sum_{k=1}^K p_k \log_2 p_k = \text{Entropy}(S)$$

Gini Impurity (CART default)

$$2 \sum_{k=1}^K p_k^2 - 1 = \text{Gini}(S)$$

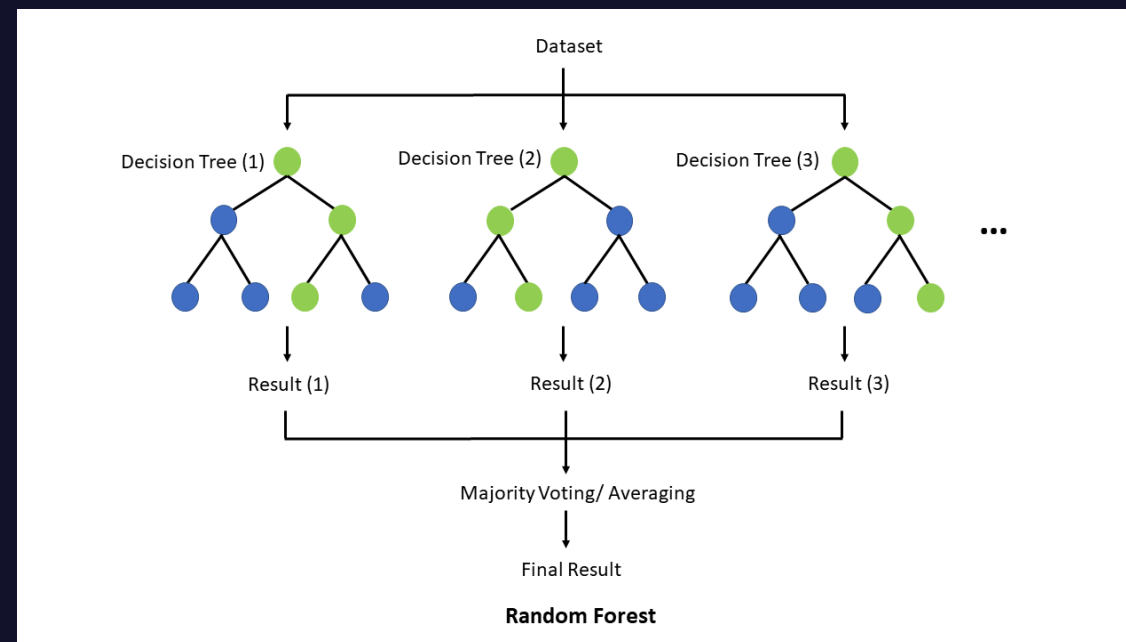
.Better splits reduce impurity and increase class purity in child nodes



REGRESSION TREE	CLASSIFICATION TREE	ASPECT
Continuous value	Categorical class	Target
MSE / MAE	Gini / Entropy	Split criterion
Numeric mean/median	Class label or class probability	Leaf output

.Both use recursive partitioning, but optimize different objectives

$$\text{mode}\{y_1(x), \hat{y}_2(x), \dots, \hat{y}_T(x)\} = (x)_{RF} \hat{y}$$

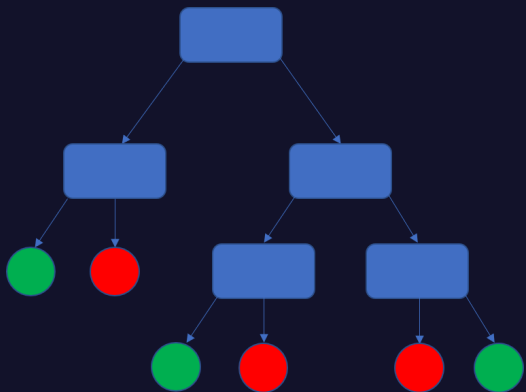


.Sample bootstrap data and train many trees

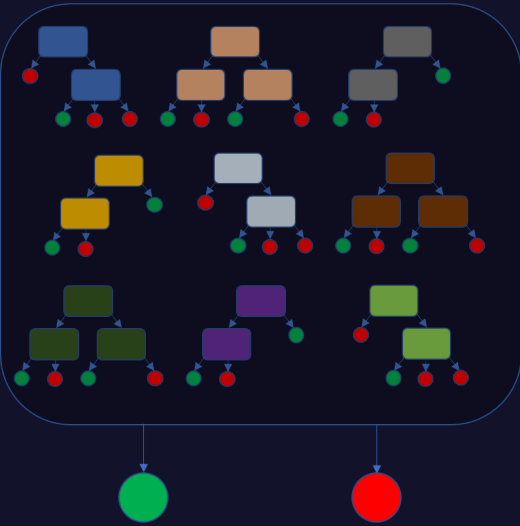
.Each tree predicts a class for the new sample

.Final class is selected by majority voting

Why Random Forest Often Outperforms a Single Tree



Decision Tree



Random Forest

RANDOM FOREST	SINGLE DECISION TREE	PROPERTY
Lower (averaging effect)	High	Variance
Lower	Higher	Overfitting risk
Moderate	High	Interpretability
More stable	Sensitive to data noise	Predictive robustness

.Random Forest improves generalization by combining diverse weak learners

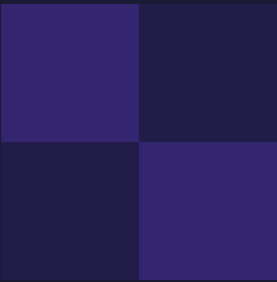
Classification Errors: FP and FN

.False Positive (Type I): model predicts positive, but actual is negative

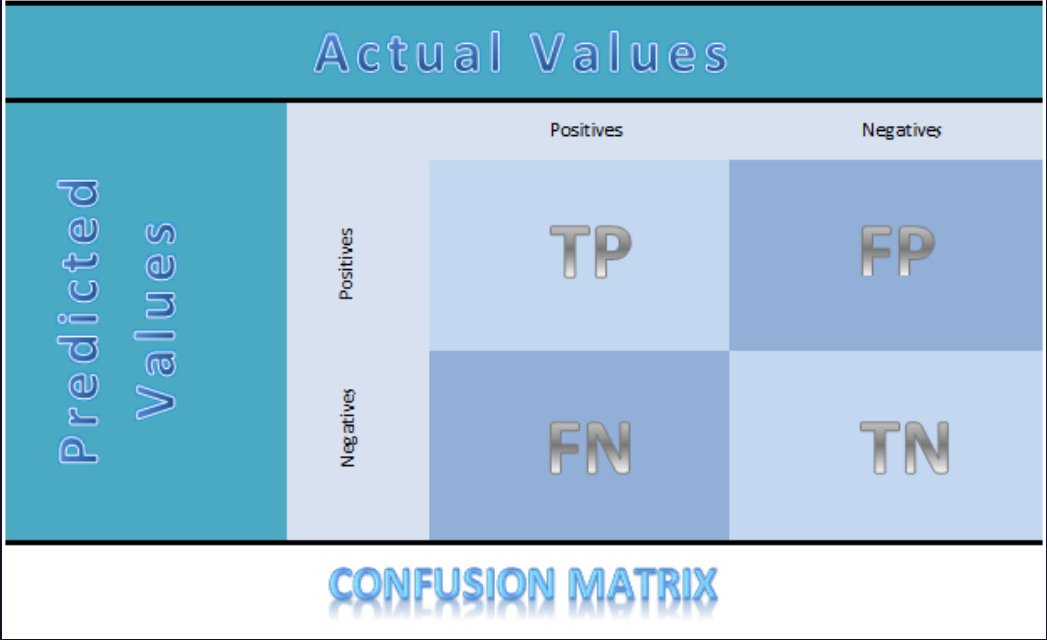
.False Negative (Type II): model predicts negative, but actual is positive

.Error impact depends on domain costs (e.g., healthcare vs spam filtering)

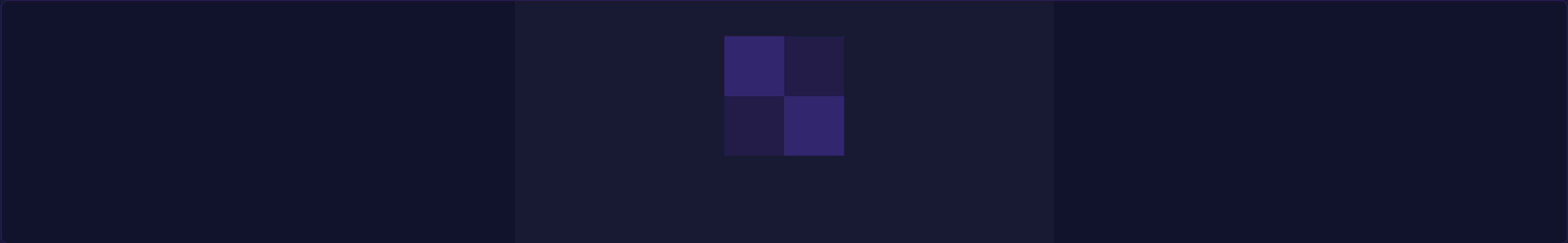
.Model evaluation should consider error trade-offs, not accuracy alone

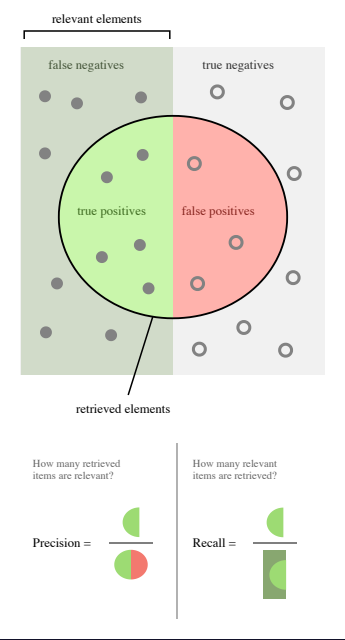


$$\frac{TP + TN}{TP + TN + FP + FN} = Accuracy$$



	MEANING	METRIC
	Correctly predicted positives	TP
	Correctly predicted negatives	TN
	Incorrectly predicted positives	FP
	Incorrectly predicted negatives	FN





.Precision: of predicted positives, how many are truly positive

.Recall: of actual positives, how many were detected

.F1-score: harmonic mean balancing precision and recall

Formulas

$$\frac{TP}{TP + FN} = Recall \quad , \quad \frac{TP}{TP + FP} = Precision$$

F1

$$\frac{Precision \cdot Recall}{Precision + Recall} \cdot 2 = F1$$

.Use F1 especially when class distribution is imbalanced

Section Takeaways

.Naive Bayes assumes feature independence — fast baseline for text .1

.Decision trees overfit; Random Forests reduce variance via bagging .2

.Precision/recall/F1 reflect different failure costs .3

?**Think:** When is interpretability worth more than a few points of AUC

SVM

Kernel Methods

Linear vs RBF

Support vectors

Kernel trick

Margin maximization

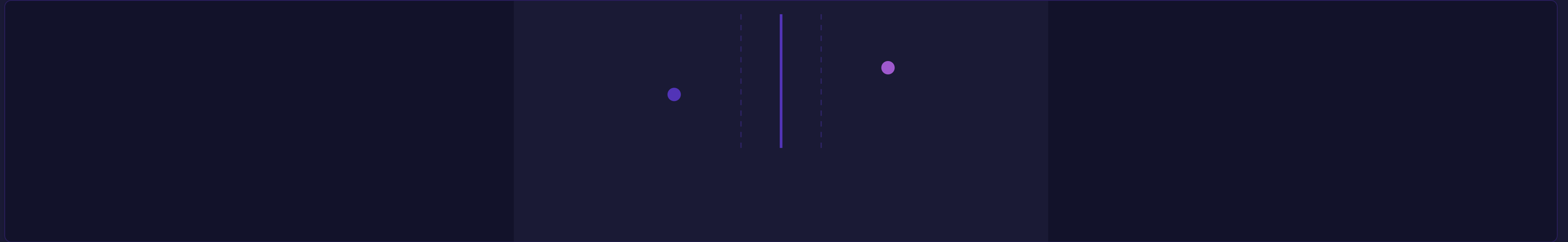
slides · ~18 min 11 ▶

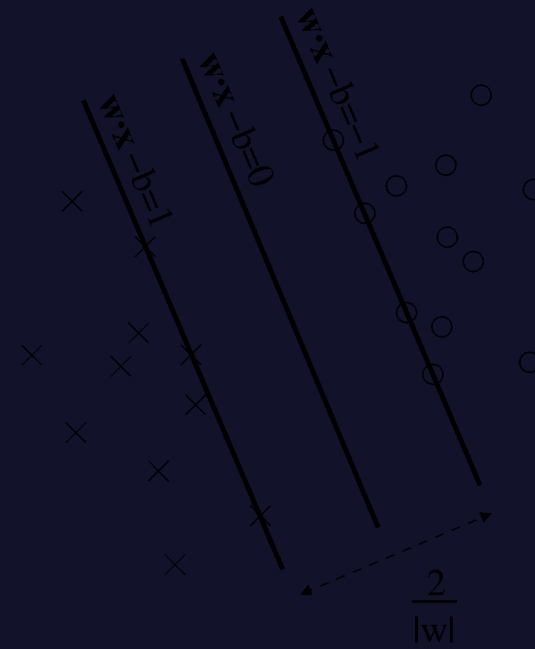
?Where Are We in the Bootcamp

.Find optimal margins and lift linear boundaries with kernels

- S1 Foundations & Preprocessing •
- S2 Regression Models •
- S3 Classification Basics •
- S4 Naive Bayes & Trees •
- S5 SVM & Kernels •
- S6 Clustering & PCA •
- S7 Deep Learning •
- S8 NLP Fundamentals •
- S9 Tokenization •
- S10 Text Analysis & NER •
- S11 Language Modeling •
- S12 Embeddings & RNNs •
- S13 Seq2Seq & NMT •
- S14 Generative AI •
- S15 RAG Systems •
- S16 MLOps •

.SVM is a supervised method that finds the best separating boundary (hyperplane) between classes

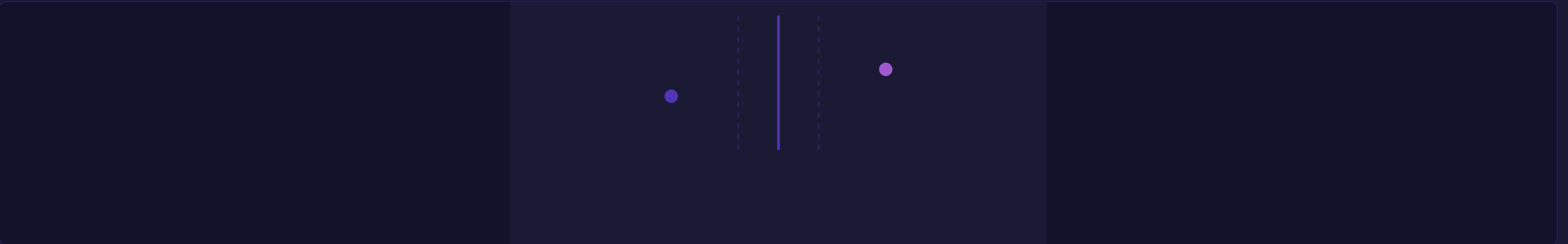




.It maximizes the margin between classes

.Only support vectors (closest points) determine the boundary

.A larger margin usually improves generalization



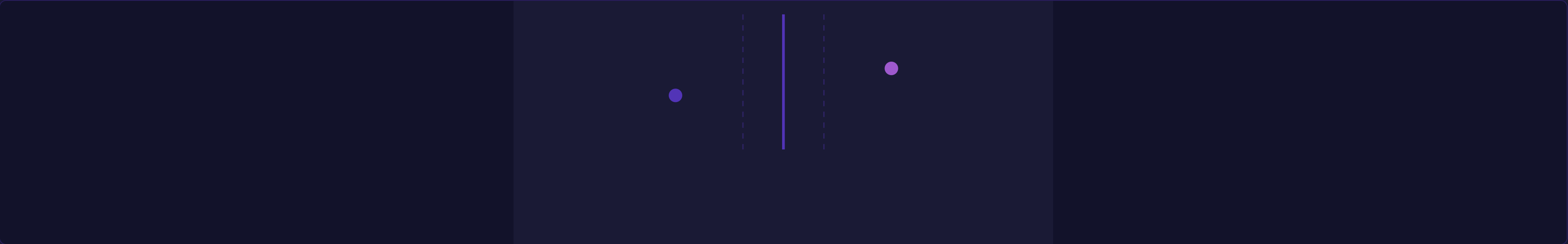
$$\text{sign}(f(x)) = \hat{y}, b + w^T x = f(x)$$

.New points are classified by the side of the hyperplane they fall on

.Hard margin: no misclassification allowed (strict separation)

.Soft margin: allows violations for better robustness with overlap/noise

.Soft-margin SVM adds penalty C to control margin-violation tradeoff

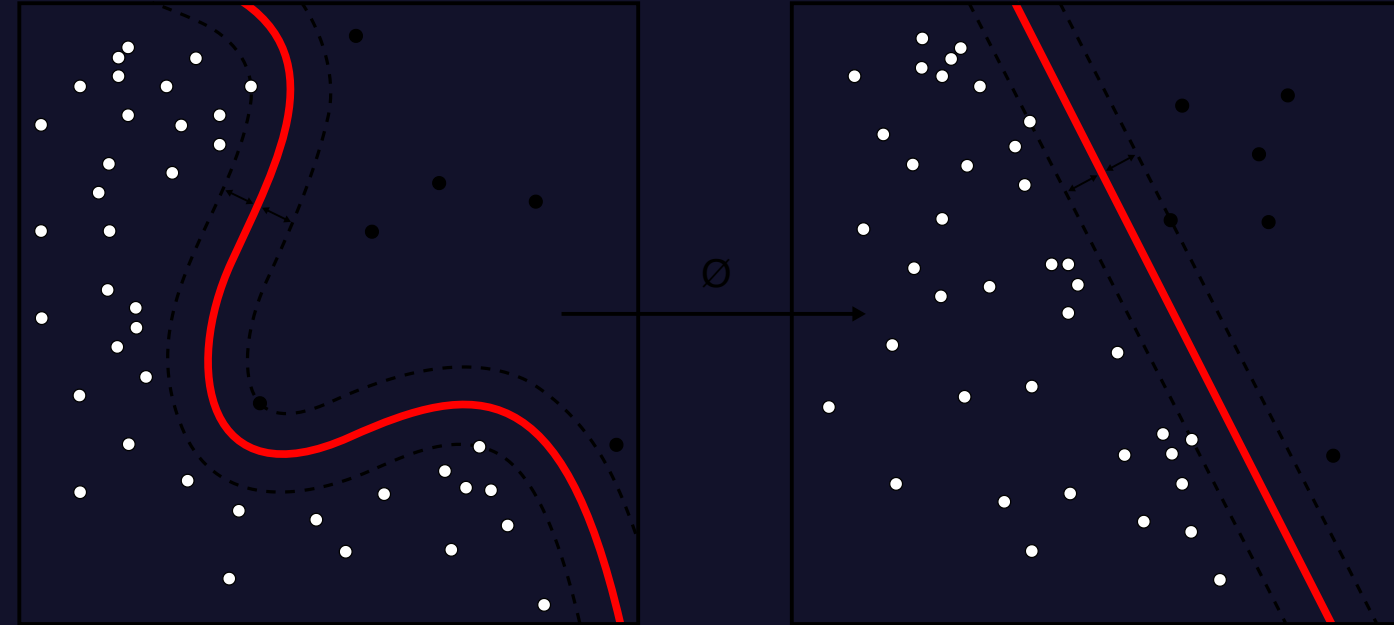


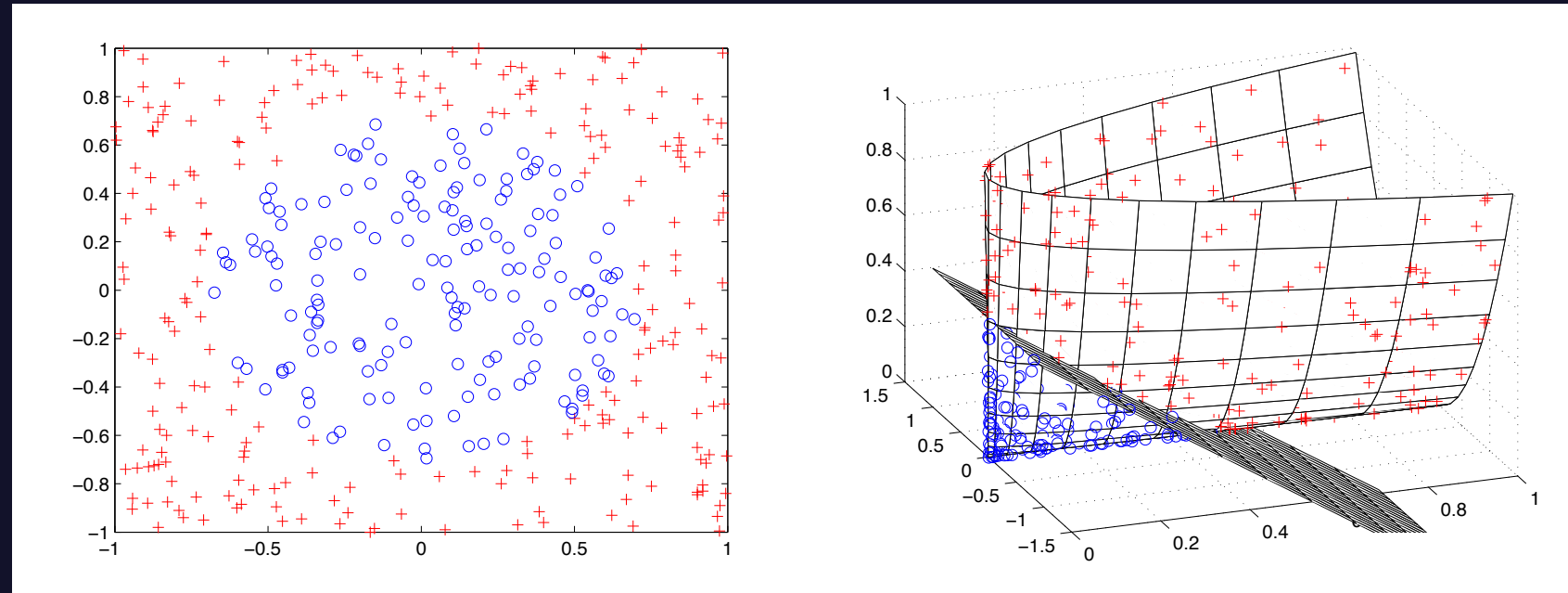
.Common in text classification and image-related features

.Often effective on small-to-medium datasets

WHY IT MATTERS	CAPABILITY
.Better robustness to small perturbations	Maximum-margin principle
.Handles non-linear class boundaries	Kernel support
.Works well in text and sparse feature spaces	High-dimensional performance

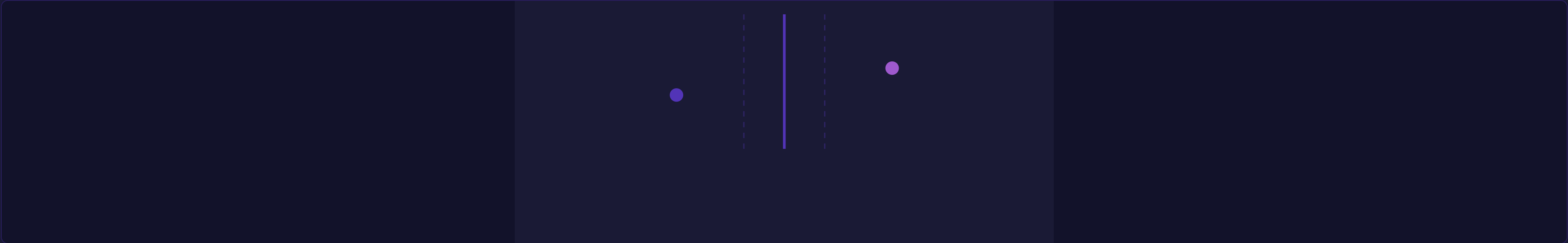
.When data is not linearly separable in original space, SVM uses kernel functions to separate it in a transformed feature space





.Avoids explicit high-dimensional mapping in many cases

. x_j, x_i (Computes similarity using kernel function K



$$\left(\frac{2\|x-l\|^2}{2\sigma}\right)\exp = K(x,l_i)$$

.If x is close to landmark l_i , similarity is near 1

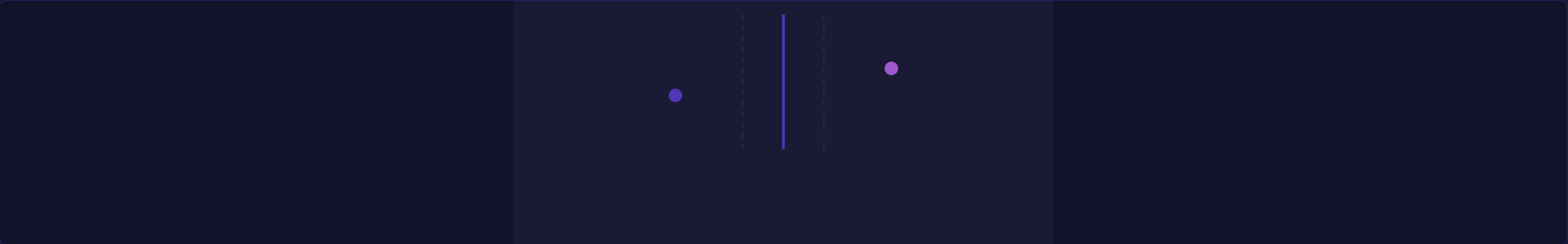
.If x is far, similarity approaches 0

.RBF is a common default for non-linear SVM

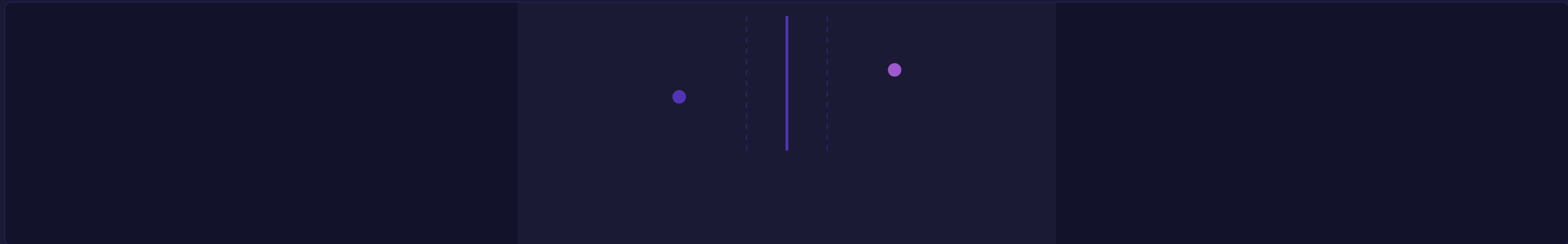
Effect of sigma

.Large sigma: smoother, wider decision boundary (higher bias)

.Small sigma: tighter, complex boundary (higher variance/overfitting risk)



NOTES	TYPICAL USE	KERNEL
.Fast and interpretable margin	High-dimensional sparse data	Linear
.Strong baseline in many tasks	General non-linear patterns	RBF (Gaussian)
.Degree controls complexity	Polynomial-like interactions	Polynomial
.Less common in practice	Neural-style boundary behavior	Sigmoid



.Use: medium-sized data, clear margin, high-dimensional text

.Avoid: massive datasets, heavy noise without kernel tuning

Use SVM When

.Data is high-dimensional or moderately sized

.Classes are reasonably separable

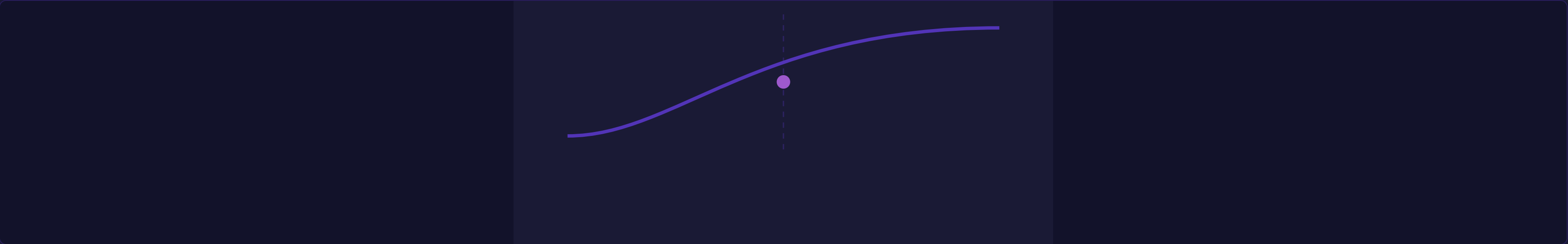
.You need non-linear boundaries via kernels

Avoid SVM When

.Dataset is very large (training can be expensive)

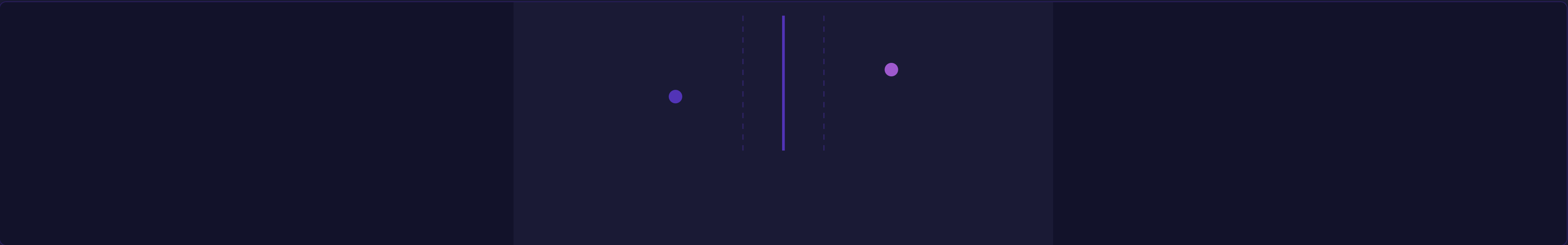
.Data is extremely noisy with weak class structure

.Feature count and sample size make kernel matrix too costly



K-NN	LOGISTIC REGRESSION	SVM	ASPECT
Classification	Classification	Classification (and SVR)	Primary task
Yes (distance-based)	Limited in linear form	Yes, with kernels	Non-linearity
Heavy at prediction	Usually efficient	Can be heavy on large data	Compute profile
Local neighborhood patterns	Fast interpretable baseline	Complex margins, high-dimensional spaces	Best for

.Model choice should be validated with cross-validation and error tradeoff metrics



TYPICAL TUNING DIRECTION	ROLE	PARAMETER
Higher C -> stricter fit, lower C -> smoother margin	Penalty for margin violations	C
Start with RBF, compare with linear/polynomial	Similarity function	Kernel
Higher gamma -> tighter boundary	Locality of influence	gamma (RBF)
Increase only when needed	Polynomial complexity	degree (poly)

Standardize features (e.g., using `StandardScaler`). .Scale features before SVM, especially for RBF/polynomial kernels

Section Takeaways

.SVM maximizes margin — support vectors define the boundary .1

.Kernel trick maps features implicitly to higher dimensions .2

.SVMs shine on medium data with clear margin structure .3

?**Think:** Why might an RBF kernel overfit on tiny datasets

Clustering

PCA

Silhouette & elbow

PCA

Hierarchical clustering

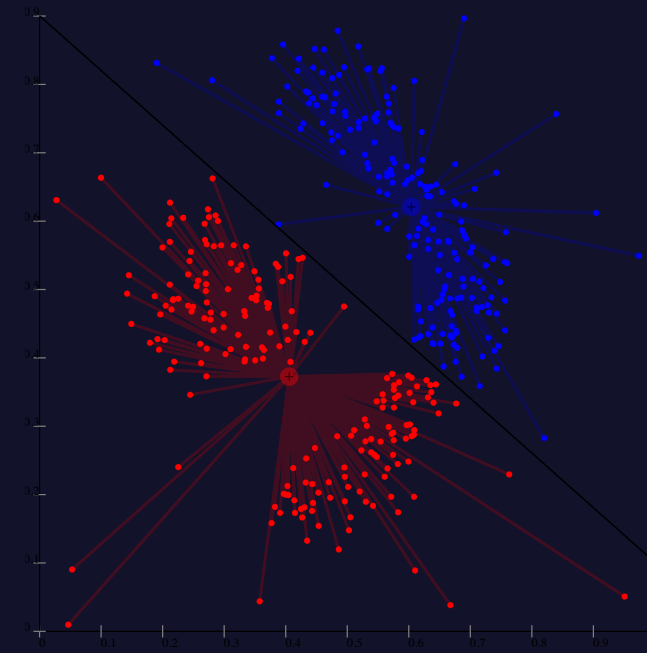
K-Means

slides · ~18 min 11 ▶

?Where Are We in the Bootcamp

.Discover structure without labels and compress high-dimensional views

- S1 Foundations & Preprocessing •
- S2 Regression Models •
- S3 Classification Basics •
- S4 Naive Bayes & Trees •
- S5 SVM & Kernels •
- S6 Clustering & PCA •
- S7 Deep Learning •
- S8 NLP Fundamentals •
- S9 Tokenization •
- S10 Text Analysis & NER •
- S11 Language Modeling •
- S12 Embeddings & RNNs •
- S13 Seq2Seq & NMT •
- S14 Generative AI •
- S15 RAG Systems •
- S16 MLOps •



.Clustering groups similar observations without target labels

.It is used for customer segmentation, anomaly discovery, and data exploration

.Cluster quality depends on feature scaling, distance metric, and algorithm assumptions

.In unsupervised settings, evaluation combines metrics with domain interpretation

Partition Data into K Compact Groups



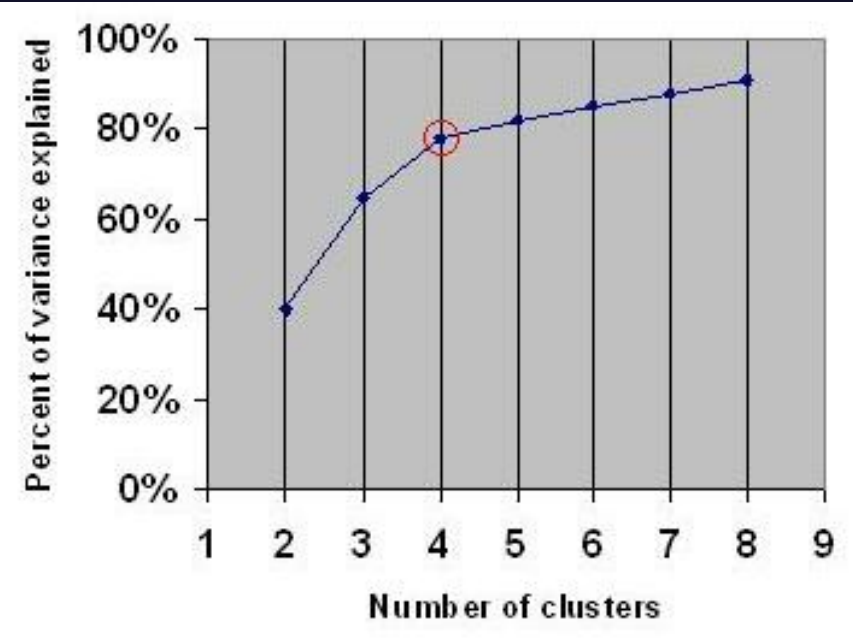
$$\sum_{x \in C} \|x - \mu_i\|^2 = J$$

.Initialize K centroids, assign each point to the nearest centroid, then update centroids

.Repeat assignment and update until cluster memberships stabilize

.K-Means favors spherical, similarly sized clusters in Euclidean space

.The objective minimizes within-cluster variance (WCSS/inertia)

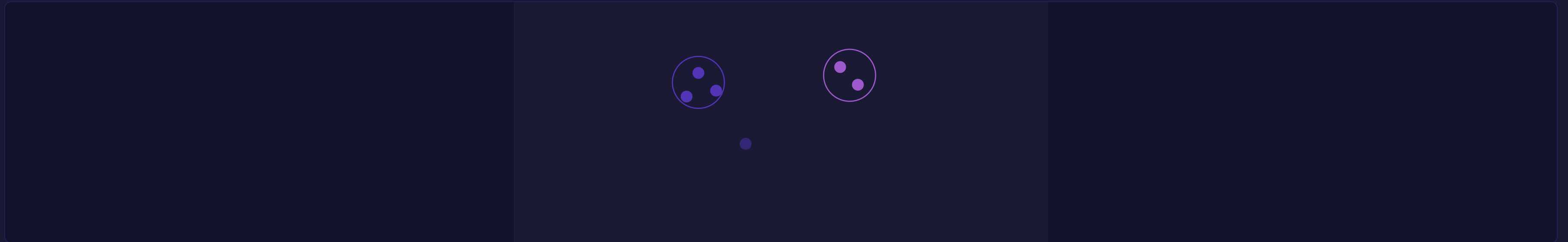


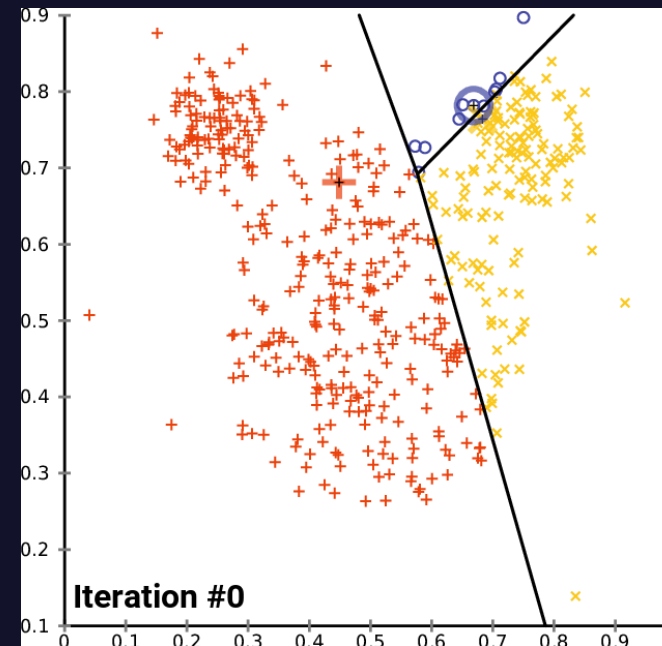
INTERPRETATION	WHAT TO INSPECT	METHOD
Choose the knee where improvement starts diminishing	WCSS vs number of clusters	Elbow
Higher values indicate better separation and compactness	Mean silhouette score in [-1, 1]	Silhouette

.Use both metrics and business context; the mathematically best K is not always operationally best



A Better Start than Pure Random Seeds



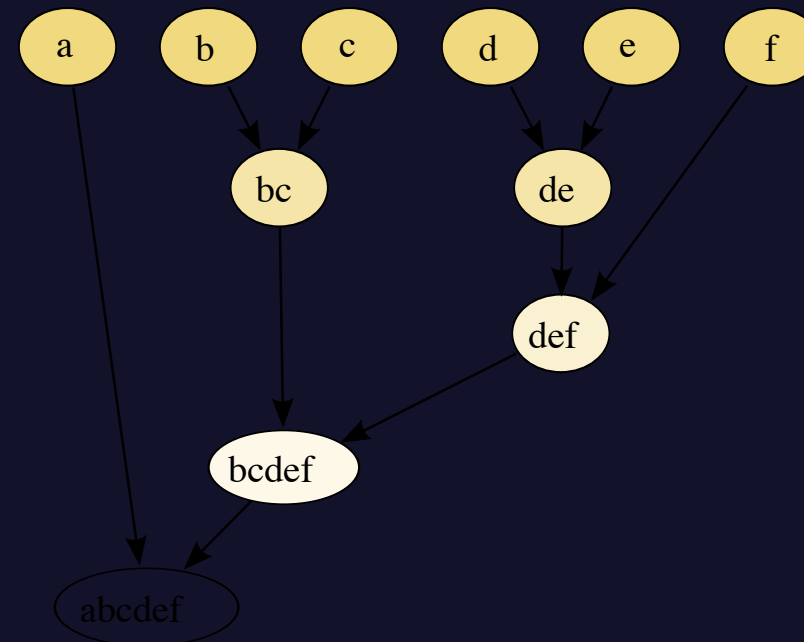


.Pick the first centroid randomly from data points

D^2 Choose subsequent centroids with probability proportional to squared distance

.Then run standard K-Means iterations

.K-Means++ reduces poor initialization risk and often converges to lower inertia

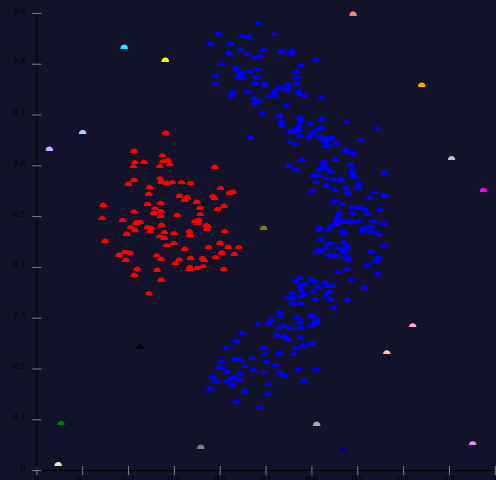


.Agglomerative clustering starts with one point per cluster and merges progressively

.Divisive clustering starts with one global cluster and splits recursively

.The dendrogram stores merge history and supports multi-resolution analysis

.Unlike K-Means, hierarchical methods can reveal nested structure in data



TYPICAL BEHAVIOR	DISTANCE BETWEEN CLUSTERS	LINKAGE
Can chain elongated clusters	Minimum pairwise distance	Single
Produces compact, tighter clusters	Maximum pairwise distance	Complete
Balanced tradeoff between single/complete	Mean pairwise distance	Average
Often forms homogeneous clusters	Increase in within-cluster variance	Ward

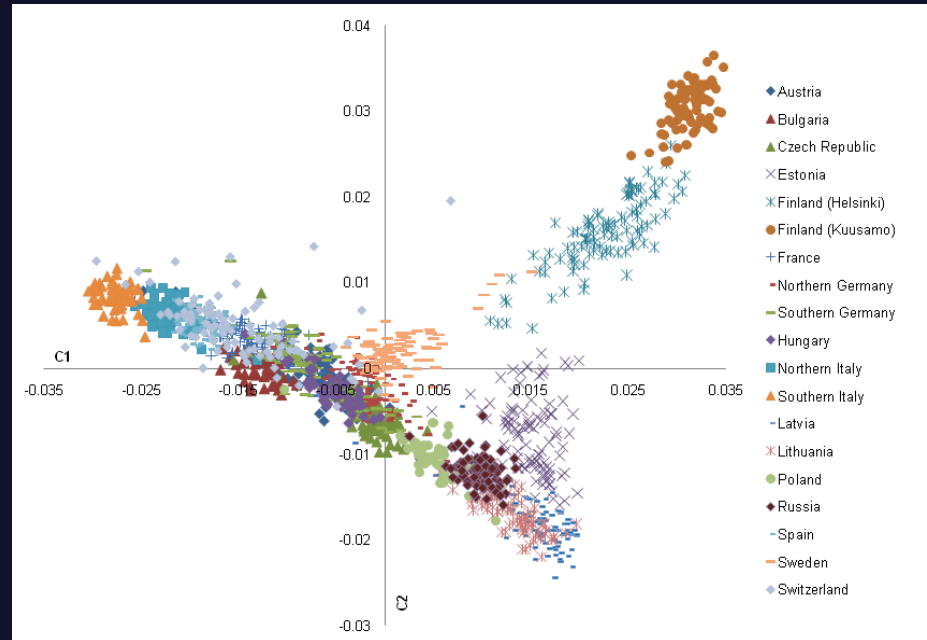
.Choose a dendrogram cut threshold where vertical gaps are most pronounced



Project Data onto High-Variance Directions



$${}_j\lambda{}_jv = {}_jX^\top X, \quad \Sigma v \frac{1}{n-1} = \Sigma$$



.PCA transforms correlated features into orthogonal principal components

.Components are ordered by explained variance (largest eigenvalues first)

.Keeping top components reduces noise and computational cost

.Standardize features before PCA when original scales differ substantially



$$\frac{\lambda_j}{\sum_{m=1}^p \lambda_m} = \text{Explained Variance Ratio}$$

OUTCOME	ACTION	STEP
Comparable feature scales	Standardize features	1
Ranked principal directions	Compute covariance matrix and eigenpairs	2
Compressed feature representation	Select top-k components	3
Lower-dimensional inputs	Project data onto selected components	4

.Select k using cumulative explained variance (e.g., 90-95%) and downstream model performance



.Scale features, run PCA, then cluster in reduced space

.Use 2D/3D PCA projections to visually inspect cluster separability

.Compare multiple algorithms (K-Means, Hierarchical, DBSCAN) before finalizing

.Validate with silhouette score, stability checks, and domain plausibility

.Combining PCA with clustering often improves interpretability and runtime on high-dimensional data

Section Takeaways

.Clustering needs domain sense — algorithms optimize geometry, not meaning .1

.PCA finds variance directions; scale features before applying it .2

.Elbow/silhouette guide k — always validate clusters qualitatively .3

?**Think:** How would you validate clusters with no ground truth

Deep Learning Neural Networks

RNNs & regularization

CNNs

Backpropagation

Perceptron & MLP

slides · ~48 min 30 ▶

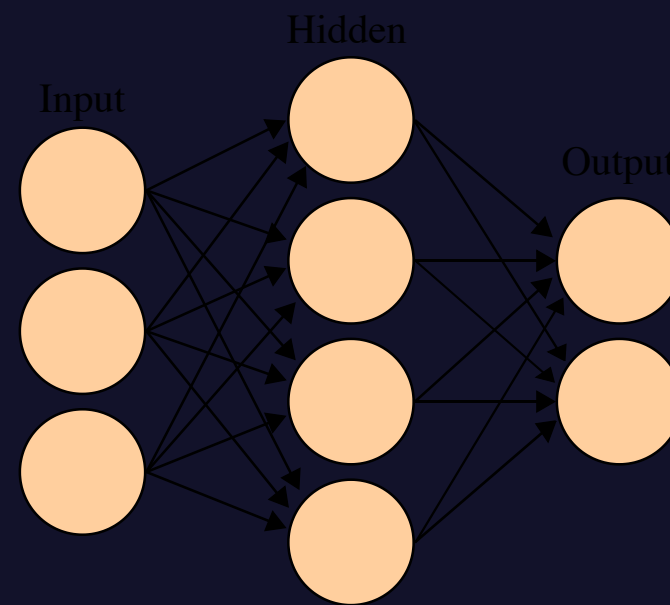
?Where Are We in the Bootcamp

.Stack layers to learn hierarchical representations from data

- S1 Foundations & Preprocessing •
- S2 Regression Models •
- S3 Classification Basics •
- S4 Naive Bayes & Trees •
- S5 SVM & Kernels •
- S6 Clustering & PCA •
- S7 Deep Learning •
- S8 NLP Fundamentals •
- S9 Tokenization •
- S10 Text Analysis & NER •
- S11 Language Modeling •
- S12 Embeddings & RNNs •
- S13 Seq2Seq & NMT •
- S14 Generative AI •
- S15 RAG Systems •
- S16 MLOps •



Each phase answers a different question: Phase 1 — why depth and what a neuron actually computes; Phase 2 — how training stays stable and generalizes; Phase 3 — which .architecture matches images vs sequences; Phase 4 — generation and reconstruction



.Four phases: foundations → optimization → architectures → autoencoders

.Each phase has a lab-style checkpoint — pause at phase boundaries

.Focus on deep learning basics — phase breaks are intentional

.Phase 1 focus: understand the “why” and the mathematical mechanics of a single neuron before scaling to layers and data

Phase 1: Foundations & Neural Core

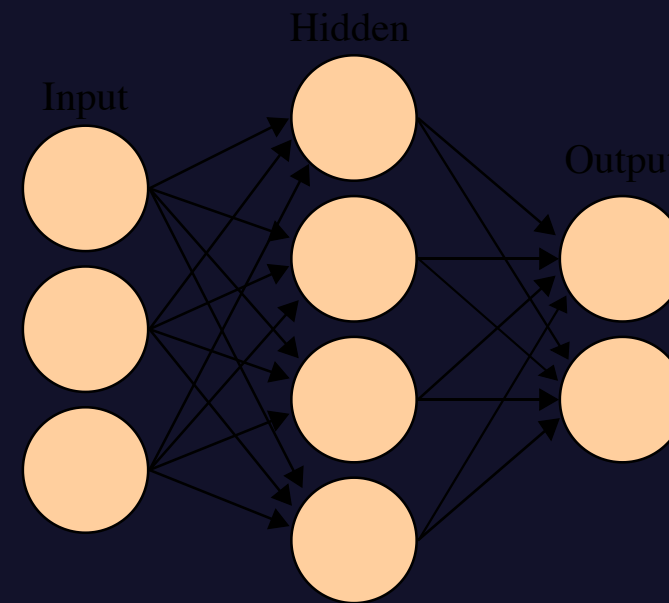


Why depth, neurons, and forward pass

- .Goal: understand what a neuron computes and why stacking layers enables hierarchical features
- .Topics: perceptron, activations, MLP, forward propagation, and loss functions
- .Exit check: explain $z = w \cdot x + b$ and why nonlinearity is required between layers

Phase 1 roadmap

QUESTION IT ANSWERS	BLOCK
?What does one layer compute	Perceptron & MLP
?How does inference flow through the graph	Forward pass
?What are we optimizing	Loss & gradients



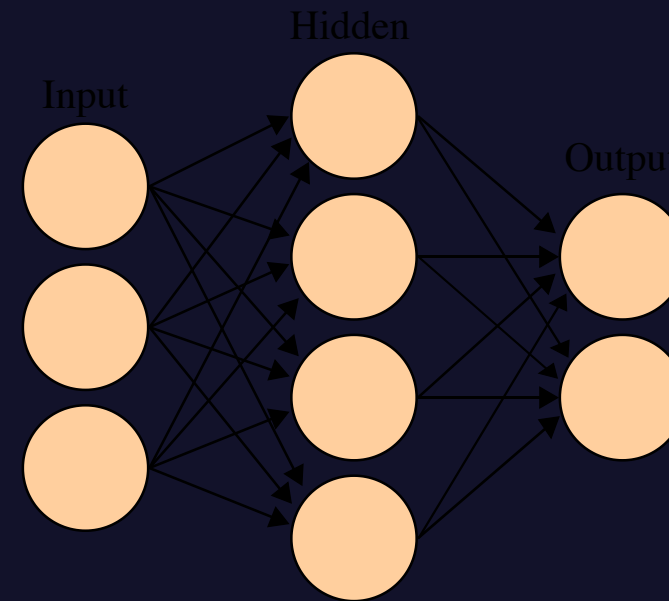
Deep learning learns a parameterized mapping from inputs to outputs by stacking nonlinear transformations (layers). “Deep” means multiple such transformations—hierarchical .features, not one giant linear rule

.Training = pick a loss that measures mistakes, then adjust weights with gradients so average loss drops on data (generalization to unseen data is the real goal)

.Unlike fixed pipelines of hand-crafted features, the network learns representations that are tuned to the task (edges → textures → parts → objects in vision, etc.)

Three ingredients you always have

WHAT IT MEANS	INGREDIENT
How signals flow (MLP, CNN, RNN, Transformer, ...)	Architecture
What “better” means for your task (MSE, cross-entropy, ...)	Loss / objective
How you search parameters and what examples you show	Optimizer + data



.Artificial intelligence (AI): the broad field—rules, search, optimization, learning, planning—anything that aims at intelligent behavior

- .Machine learning (ML): a subfield of AI where behavior improves from data via a learning algorithm (not only hand-written rules)
- .Deep learning (DL): a subfield of ML using deep neural networks—multiple learned nonlinear layers; representation learning is central

Containment (conceptual)	
CONTAINS	SET
... ,Rule systems, classical search, ML, knowledge bases	AI
Linear models, trees, kernel SVMs, shallow nets, deep nets	ML
... ,CNNs, RNNs/GRU/LSTM, Transformers, GANs, VAEs	DL

.Saying “we used AI” is vague; “we used supervised DL with a CNN” is precise

Use-Case Fit: When Deep Learning Shines — and When to Skip It

Match Method to Data, Risk, and Budget

.DL is strongest when: large data (or strong pre-training), high-dimensional raw inputs (images, text, speech), and compositionality you want learned end-to-end

.Think twice when: tiny tabular datasets, strict interpretability requirements, tight latency on CPU without optimization, or simple baselines already solve the problem

.Always compare against strong baselines (logistic regression, gradient boosting, k-NN on engineered features)—DL must earn its complexity on validation metrics you care about

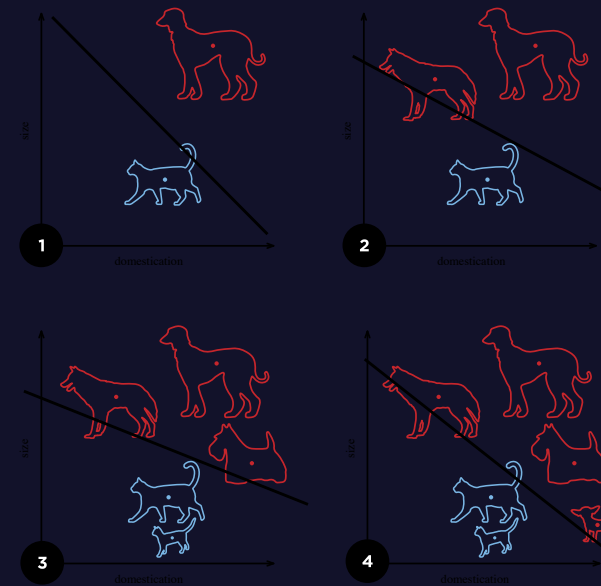
WHY	TYPICAL FIRST CHOICE	SITUATION
Raw inputs; hierarchical features	Pre-trained DL + fine-tune	Large labeled images / audio / text
Lower variance, fast iteration, interpretability	Trees / linear + regularization	Small tabular (< few k rows)
Auditability and stability	Simpler model or hybrid	Need causal or legal explanation
Capacity + regularization can scale	Larger nets + careful tuning	Plenty of GPUs + big data
Memory and power limits	Small distilled / quantized models	Edge device, millisecond latency

Single Neuron (Perceptron): Math, Weights, and Bias



One Affine Map + One Nonlinearity

$$a \text{ (or further processing)} = \hat{y}, \quad f(z) = b, \quad a + w^\top x = z$$



Affine part (see formula above): z is the weighted sum of inputs plus b —each x_i is multiplied by w_i . The bias b shifts the decision boundary (same orientation, normal w); it does not rotate the separating hyperplane

.Weights encode which patterns excite the neuron; training adjusts w and b so that on labeled data the loss decreases

.With no hidden layer and a monotonic activation, the decision boundary in input space is linear (a hyperplane)—powerful but limited

ROLE	SHAPE INTUITION	SYMBOL
One training example's inputs	Feature vector (length d)	x
Direction + strength of sensitivity to each feature	Same length as x	w
Offset / threshold in logit space	Scalar	b
Introduces nonlinearity (or identity)	Maps $\mathbb{R} \rightarrow \mathbb{R}$ (per neuron)	f

Weight Initialization: Why Zeros Break Training

Symmetry Must Be Broken

.If all weights in a layer start equal (especially zero), every hidden unit receives the same gradient and stays identical forever—symmetry never breaks

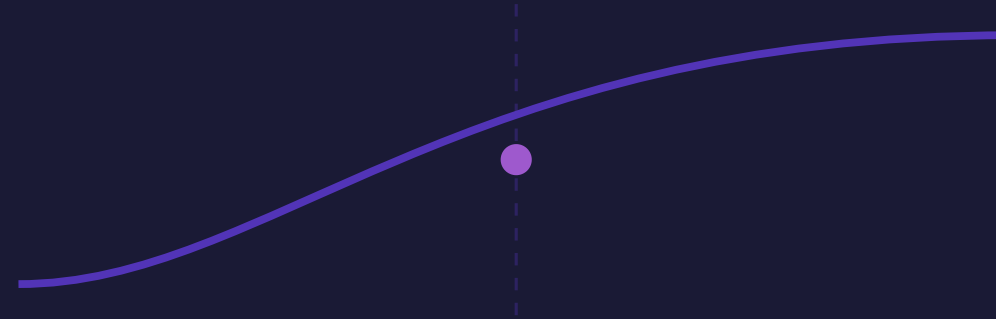
.Random initialization makes each unit walk a different path in parameter space so they can specialize as feature detectors

.Frameworks use formulas like He (Kaiming) for ReLU: $\text{variance} = 2/n_{in}$, and Xavier (Glorot) for Tanh: $\text{variance} = 2/(n_{in} + n_{out})$ to keep activation scale stable

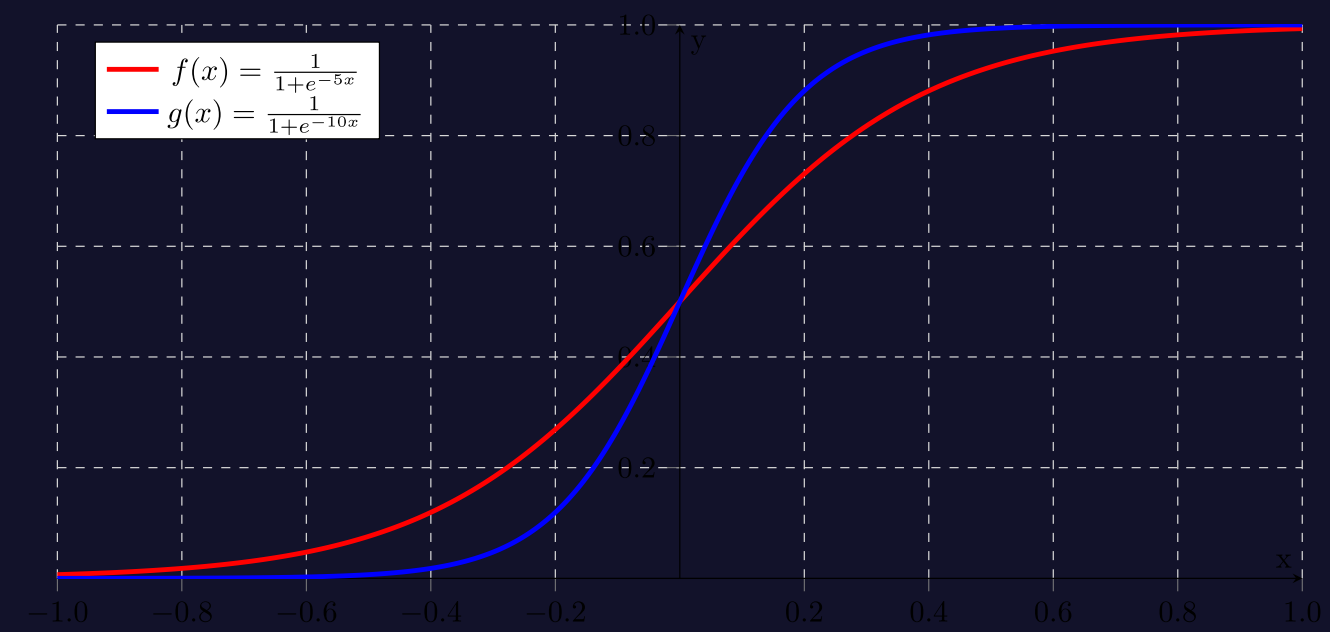
BEST SUITED FOR	PROBLEM IT AVOIDS	INIT SCHEME
None (always avoid for weights)	Hidden units stay identical; no representation diversity	All zeros
Sigmoid / Tanh activations	Vanishing or exploding variance across layers	Xavier (Glorot)
ReLU / Leaky ReLU activations	Variance drop due to ReLU zeroing out half the signals	He (Kaiming)

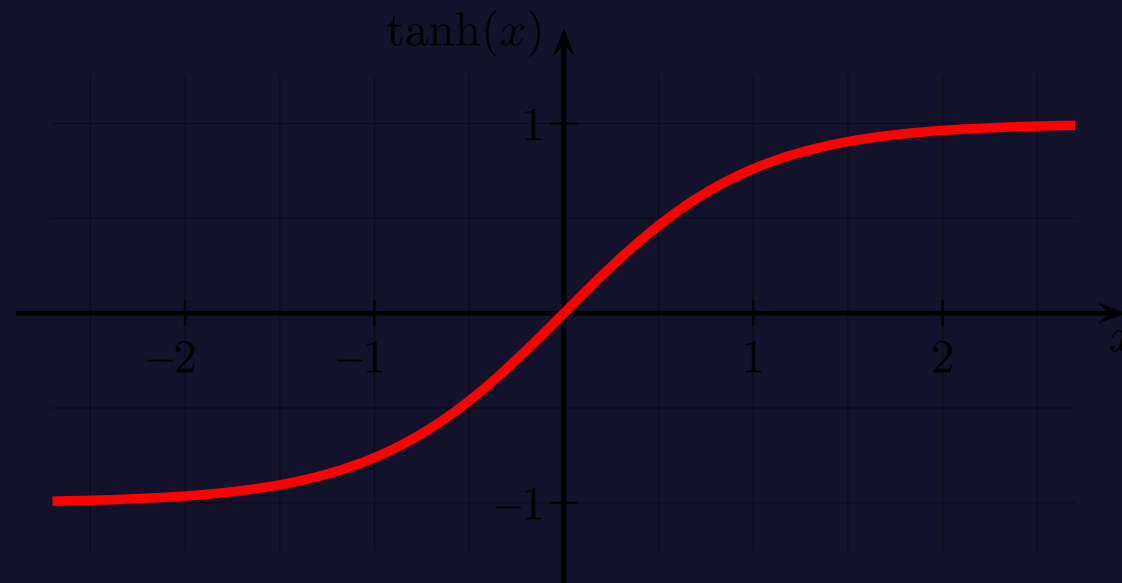


Nonlinearity is the Key to Network Depth



$$\max(0, z) = \text{ReLU}(z) \quad , \frac{e^z - 1}{e^z + 1} = \tanh(z) \quad , \frac{1}{e^z + 1} = \sigma(z)$$





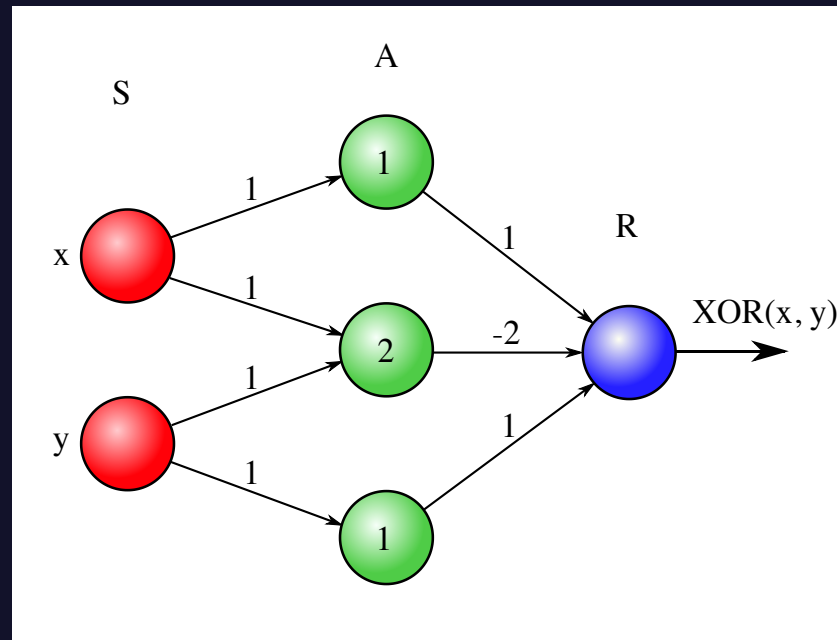
.Sigmoid: maps inputs to $(0, 1)$ —useful for probabilities, but suffers from vanishing gradients when z is very large or very small (saturation)

.Tanh: zero-centered, maps inputs to $(-1, 1)$ —often nicer than sigmoid for hidden layers but still saturates

.ReLU: modern default for deep stacks—cheap to compute, avoids vanishing gradient on positive domain; leaky ReLU / ELU are variants

PRIMARY USE-CASE	RANGE	FUNCTION
Binary classification output layer, gating networks	$]1, 0($	Sigmoid
Hidden layers (recurrent networks)	$]1, 1-($	Tanh
Default choice for hidden layers in MLPs and CNNs	$]^\infty, 0[$	ReLU

The XOR Problem: Why Hidden Layers Are Logically Necessary



.XOR is not linearly separable in the 2D input plane: no single line separates class 1 from class 0 on the four corners

.A single perceptron computes a half-space; XOR requires nonlinear mixing of inputs—exactly what a hidden layer provides

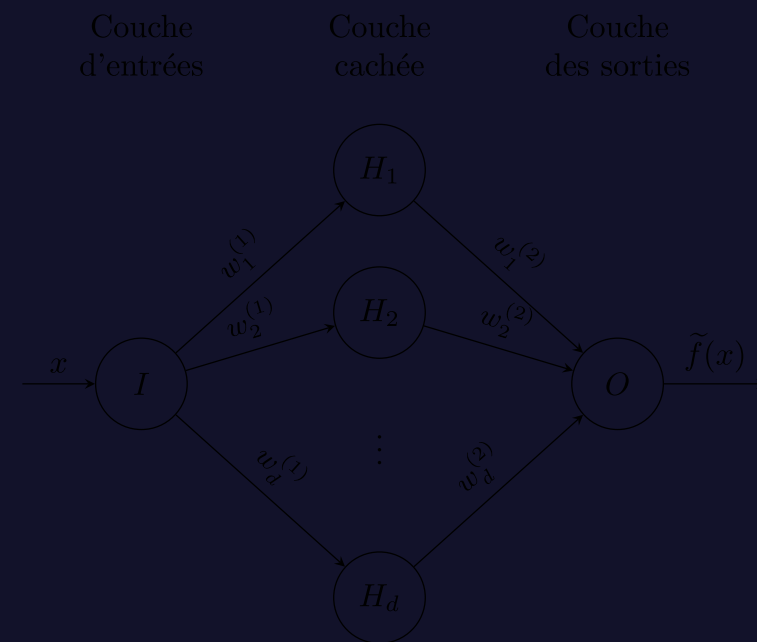
.Hidden units reparameterize inputs into a space where the task becomes linearly separable—this is the core intuition for representation learning

XOR	x_2	x_1
0	0	0
1	1	0
1	0	1
0	1	1



Depth = Composed Nonlinear Features

$$({}_2b + \tau(W_2h = \hat{y}), {}_1b + \phi(W_1x = h$$

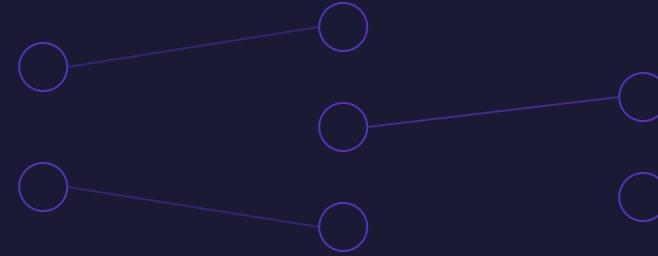


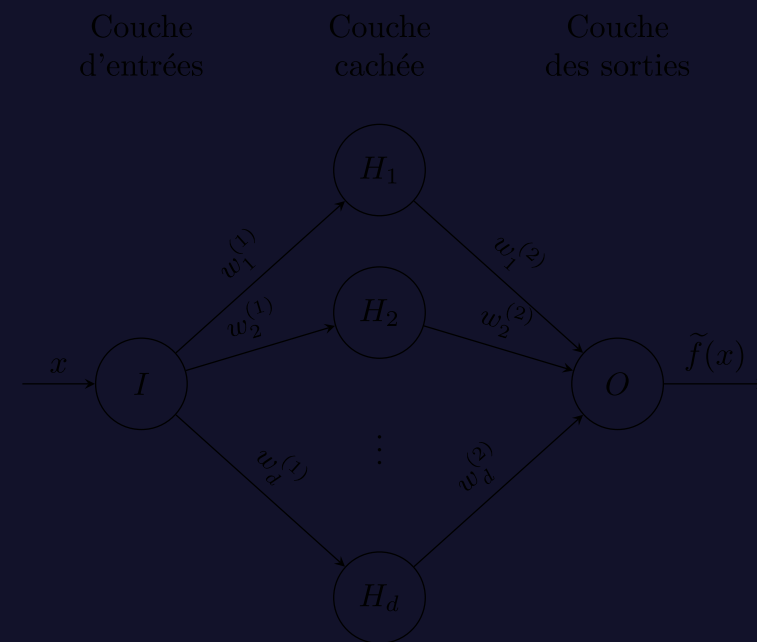
.Each hidden layer applies an affine map followed by a nonlinearity, producing new features h as functions of the original inputs

.Width (units per layer) and depth (number of layers) trade off capacity, data needs, compute, and optimization difficulty

Universal approximation (broad strokes): one sufficiently wide hidden layer can approximate many continuous functions on compact domains—in practice depth + inductive bias .wins

WHAT IT CONTROLS	HYPERPARAMETER
How many independent nonlinear features per layer	Hidden width
How many levels of composition (edges→textures→...)	Depth
Ease of optimization in very deep nets	Skip / residual





.Forward pass: feed input x through each layer in order—compute logits or scores at the end (then softmax for class probabilities if needed)

.Implementation-wise this is a computational graph: each op (matmul, ReLU, ...) knows how to propagate values forward

.Training repeats forward passes on mini-batches for efficiency and gradient noise that can help generalization

...OUTPUT CARRIES	OPERATION	STEP
Weighted sum at each unit	$b + Wx = z$:Linear	1
Nonlinear feature for next layer	$f(z) = a$:Activation	2
Class logits or regression value	Repeat / head	3



Match the Loss to the Output and Task

Regression — Mean Squared Error (MSE)

$$\mathcal{L} = \frac{1}{N} \sum_i \|y - \hat{y}\|^2$$

.Penalizes large errors heavily (quadratic); differentiable everywhere—classic for continuous targets

.Assumes Gaussian-like noise in a probabilistic story; outliers can dominate—Huber loss is a robust alternative

Classification — Cross-entropy

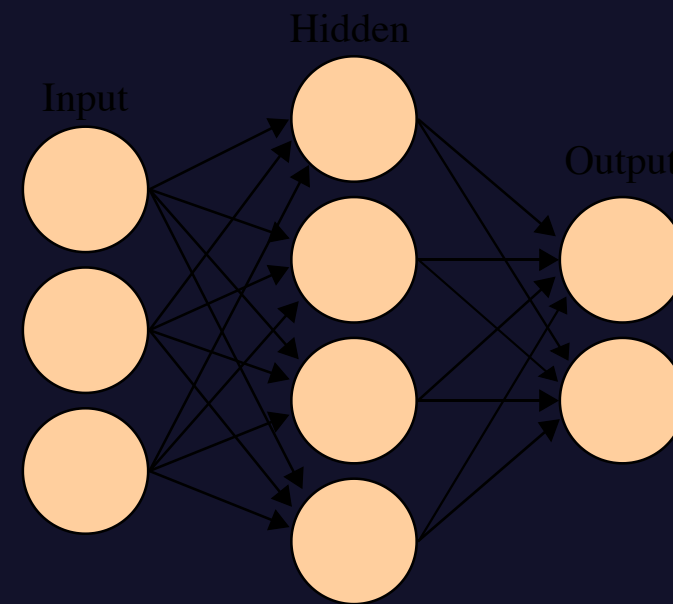
$$y_{ic} \log \hat{p}_{ic} \quad (\text{often with logits + log-softmax for stability}) \quad \sum_c \sum_i \frac{1}{N} = \mathcal{L}$$

.Compares predicted probabilities $\hat{p}$ to true distribution (one-hot or soft labels)

.Pairs naturally with softmax at the output; gradients are well-behaved when implemented in log-space

COMMON HEAD + LOSS	TASK
Linear output + MSE / Huber	Regression
Sigmoid + binary cross-entropy	Binary classification
Softmax + cross-entropy	Multi-class

(chain along paths) $\frac{z\partial}{w\partial} \frac{\mathcal{L}\partial}{z\partial} = \frac{\mathcal{L}\partial}{w\partial}$



.Backprop applies the chain rule systematically: compute adjoints $\partial \mathcal{L} / \partial$ each intermediate tensor from outputs backward

.Reverse-mode autodiff (backprop) is efficient when many parameters feed one scalar loss—exactly the neural net case

.Modern frameworks (PyTorch, JAX, TensorFlow) build the graph (explicit or traced) and implement this reliably—you still debug shapes, numerical stability, and vanishing gradients

MEANING	IDEA
Same forward values reused; backward visits each edge once	Shared subexpressions
Visit nodes after their successors—dynamic programming on the graph	Topological order

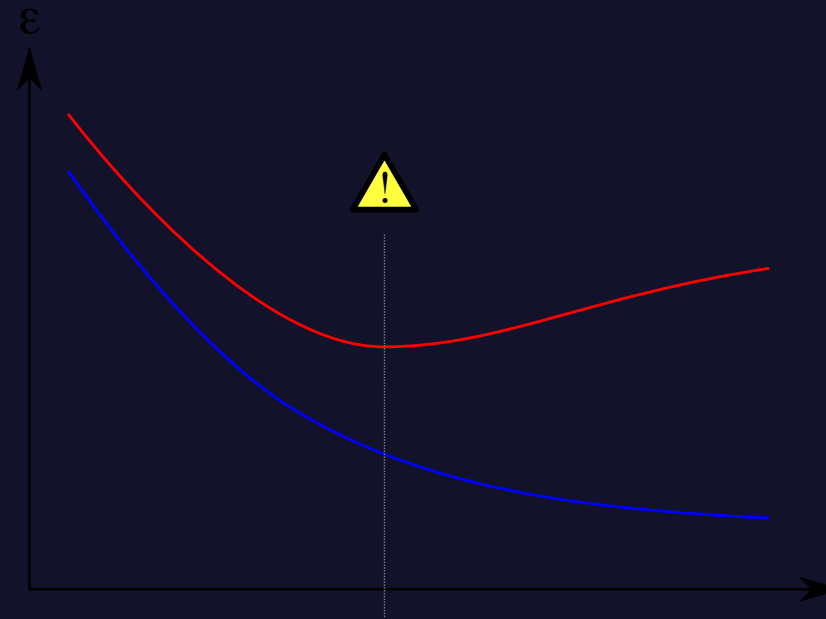
.Understanding backprop = understanding which paths carry gradient and which activations saturate

.Phase 2 is about the art of training: diagnosing fit, controlling gradients, picking optimizers, normalizing, and regularizing

.Good models are not only “big”—they match capacity to data, monitor validation faithfully, and fail visibly when something is wrong

What you will control in practice

EXAMPLES	LAYER
SGD, Adam, gradient clipping	Optimization
Batch Norm, residual paths, initialization	Stabilization
Dropout, weight decay, early stopping	Generalization



.Underfitting: both train and validation error stay high—the model is too simple, features are insufficient, or optimization is stuck (LR too low, wrong loss)

.Good fit: train and validation errors are both low and track each other; small gap is normal if train is slightly better

.Overfitting: train error keeps dropping while validation worsens or plateaus badly—the model memorizes idiosyncrasies (noise, augment leakage)

TYPICAL RESPONSE	VAL LOSS / METRIC	TRAIN LOSS	REGIME
More capacity, better features, longer train, higher LR (carefully)	High	High	Underfitting
Keep regularization; ship or iterate on data	Low, close to train	Low	Good fit
More data, dropout, wd, simpler model, early stop	Worse / diverging gap	Very low	Overfitting

Depth Multiplies Jacobians Layer by Layer

.Vanishing: backprop multiplies many factors >1 (saturated sigmoid/tanh, poor init)—early layers get tiny updates and learn slowly

.Exploding: repeated factors <1 or large weights produce huge parameter updates—loss spikes, NaNs

.Fixes are layered: architecture (residual paths), activations (ReLU), initialization (He/Xavier), normalization (BN/LN), and optimization (clipping)

MITIGATION	LIKELY CAUSE	SYMPTOM
ReLU, He init, residuals, norm	Vanishing chain through saturated units	Early layers barely move
Clip grads, lower LR, check loss scaling	Exploding activations or LR too high	Loss $\rightarrow$ NaN quickly
Finite checks, gradient clipping	Rare but real—mixed precision, bad data batch	Stable then sudden blow-up

.SGD + momentum: accumulates updates in a velocity vector to dampen oscillations in ravines and accelerate along consistent directions

.Adam: tracks both first-moment (momentum) and second-moment (recent squared gradients) of gradients to adapt learning rates per parameter

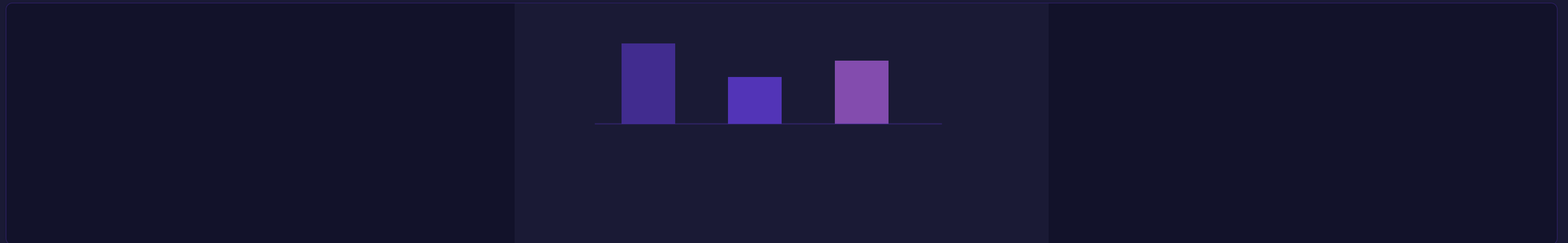
.SGD is often preferred in computer vision for generalization, while Adam is the default starting point for Transformers and general MLPs

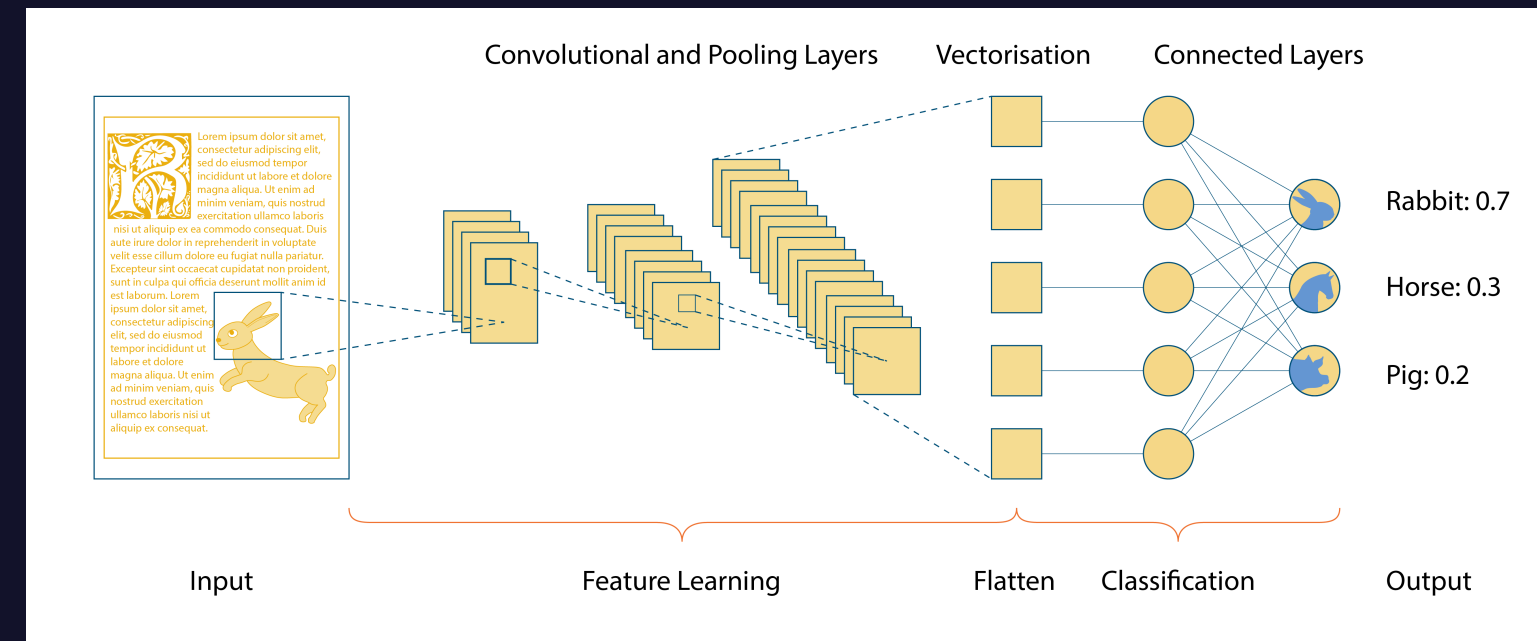
WHEN TO USE	CORE MECHANISM	OPTIMIZER
Rarely used alone today	Step in negative gradient direction	Vanilla SGD
Standard for CV models	Adds fraction of previous step direction	SGD + Momentum
Robust default for text/tabular/MLPs	Adapts learning rate per parameter based on gradients	Adam

.Learning rate (LR) is the single most important hyperparameter to tune (typically searched on a log grid, e.g., 1e-4, 1e-3, 1e-2)



Stabilize Activations and Speed Up Deep Training





.Batch Norm (BN): normalizes activations across the mini-batch for each channel, then learns scale γ and shift β

.It stabilizes internal activations, allowing higher learning rates and reducing dependency on precise initialization

.Alternative is Layer Norm (LN) which normalizes across features per example—standard in Transformers and sequence models

TYPICAL USE-CASE	NORMALIZATION DIMENSION	METHOD
CNN classifiers, ResNets	Across batch and spatial dimensions	Batch Norm
Transformers, RNNs	Across features for a single token/example	Layer Norm



Dropout

.Randomly zero out hidden units during training with probability p (e.g., 0.2-0.5) so neurons cannot co-adapt

.Acts as a powerful implicit ensemble method; disabled during inference (evaluation mode)

Weight Decay (L2 regularization)

.Adds a penalty to the loss proportional to the squared sum of weights to keep weights small and smooth

Early stopping

.Monitor validation loss during training and stop when it ceases to improve for a certain number of epochs (patience)

INFERENCE BEHAVIOR	PRIMARY ACTION	TECHNIQUE
Turned OFF (weights scaled)	Randomly drops connections	Dropout
Turned ON (weights are static)	Shrinks weights toward zero	L2 Weight Decay
N/A (model is checkpointed)	Halts training at lowest val loss	Early Stopping

Phase 3: Specialized Architectures (Vision & Sequences)



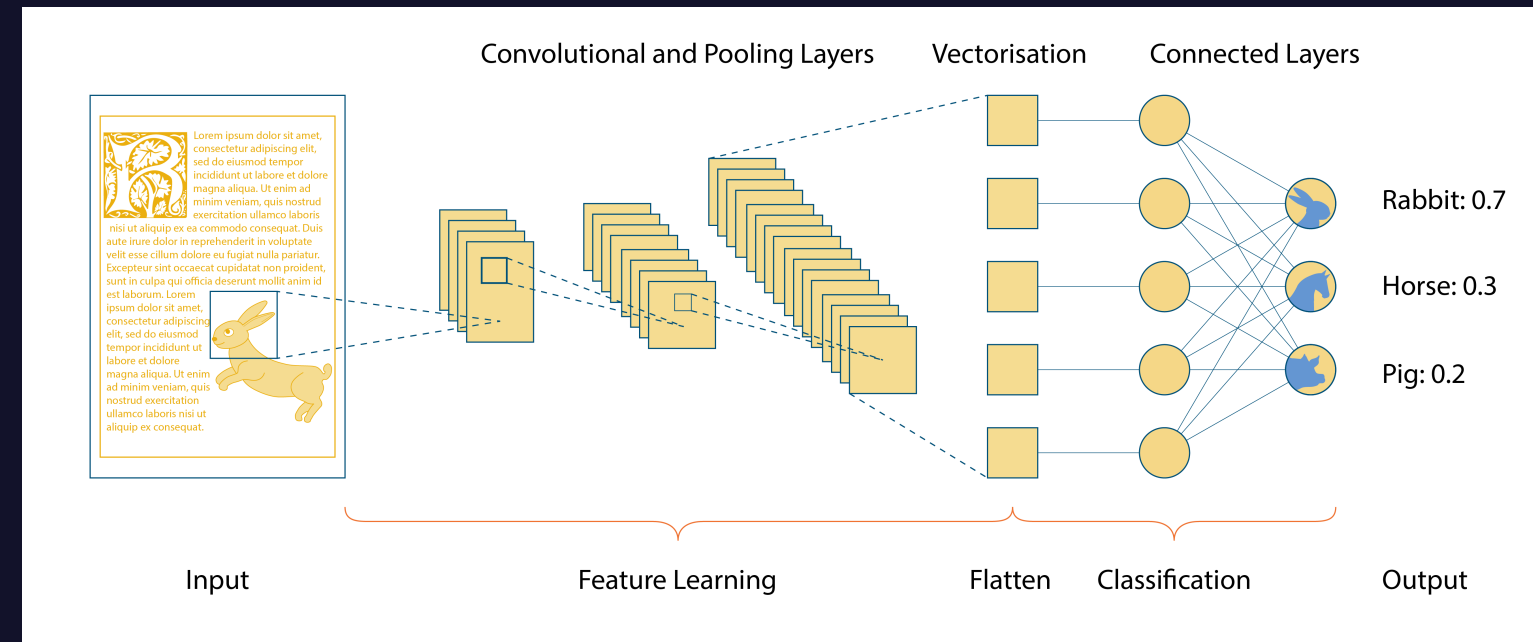
Images vs Ordered Sequences

.Images have 2D locality and translation symmetry—convolutions exploit shared weights across space

.Sequences have order—recurrence, causal convolutions, or attention carry context across time or position

.Choosing the right inductive bias beats blindly scaling the wrong architecture

REPRESENTATIVE LAYERS	CORE STRUCTURE	MODALITY
Conv2D, pooling, residual blocks	Spatial grids	Image / video frames
RNN/GRU/LSTM, causal conv, self-attention	Ordered tokens or time steps	Text / speech / sensors



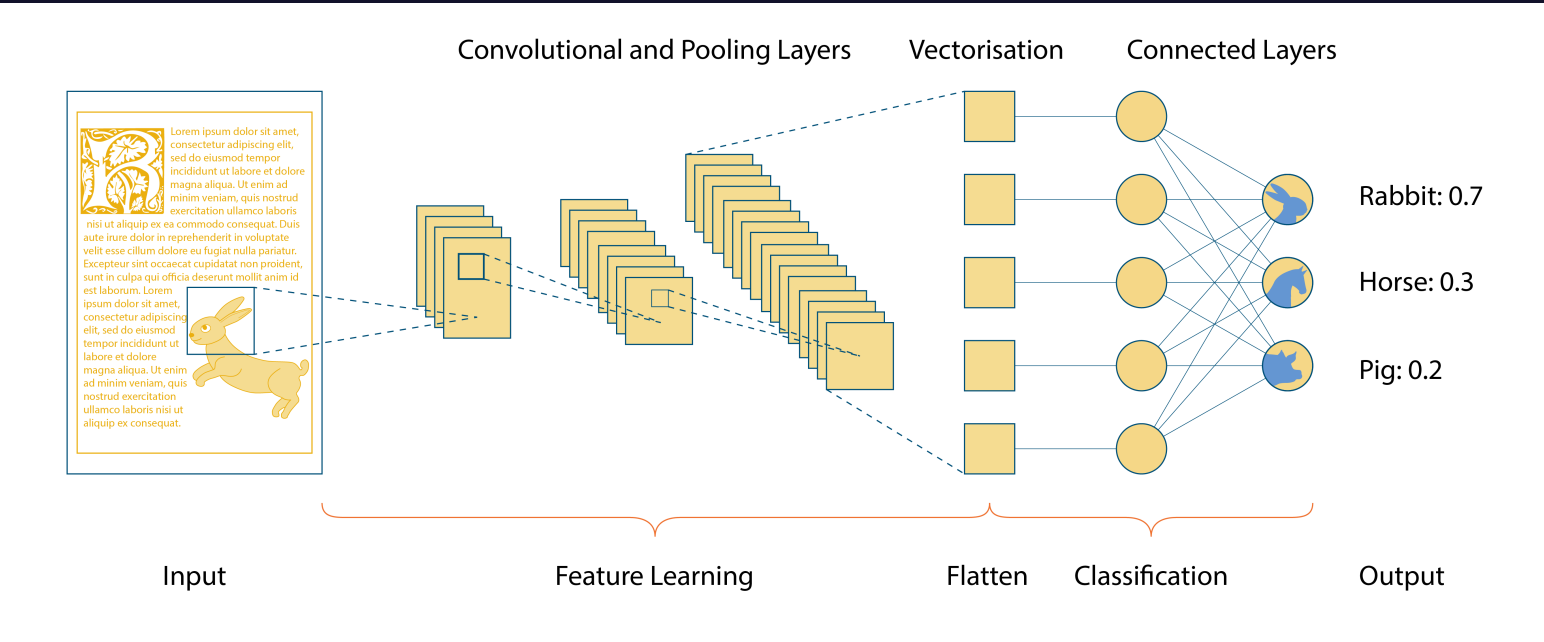
.A color image is usually a tensor of shape $H \times W \times 3$ (RGB); each entry is an intensity discretized into 8 bits (0–255) before normalization

- .Meaning lives in local neighborhoods: small patches reveal edges; larger contexts reveal objects—hierarchical composition
- .Augmentation (crop, flip, color jitter) teaches invariance; normalization (e.g., ImageNet mean/std) stabilizes optimization

WHAT HAPPENS	STAGE
Quantized intensities per channel per pixel	Raw sensor
Resize, crop, normalize, augment	Preprocess
Batch × C × H × W tensor fed to stem convolutions	Model input

Convolution: Kernels, Edge Filters, and Feature Maps

Sliding Linear Filters + Nonlinearity





.A kernel is a small learnable matrix (e.g., 3×3) that responds to local patterns; multiple kernels produce a stack of feature maps

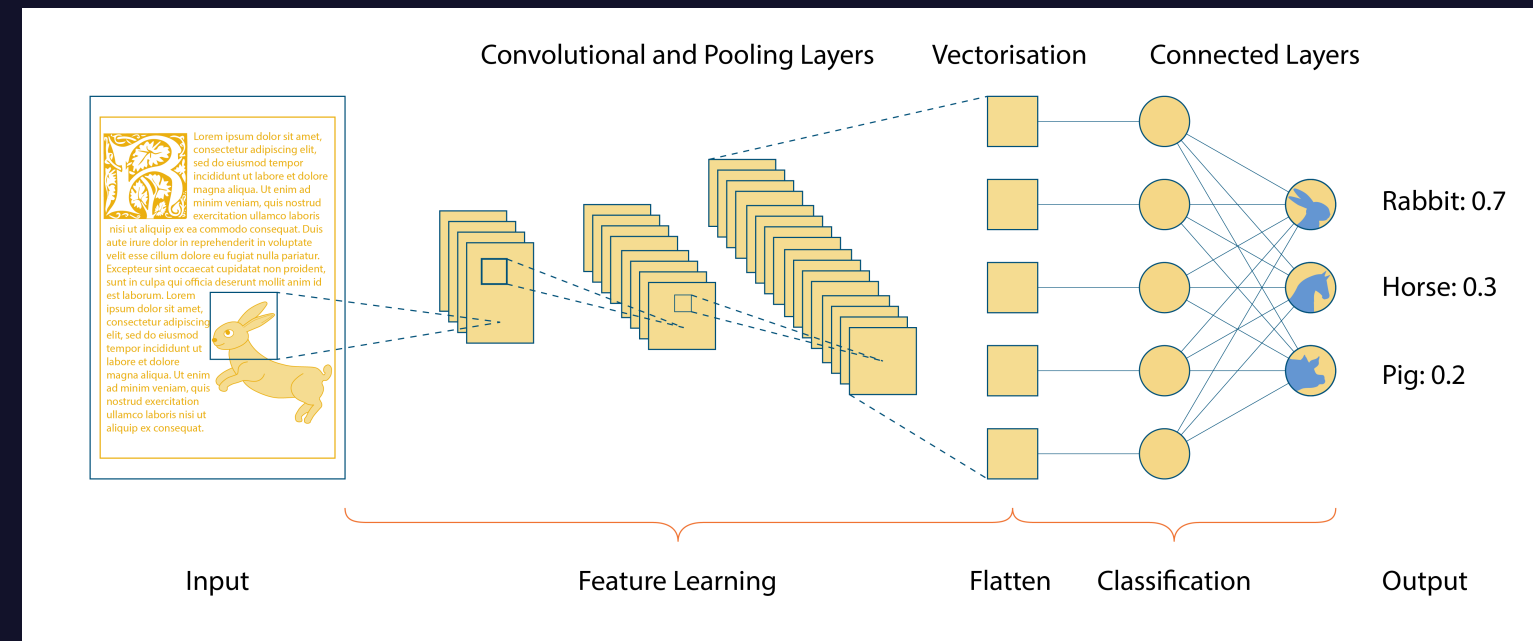
.Classical edge detectors (Sobel, shown) are fixed kernels; CNNs learn task-specific filters from data

.Deep stacks widen the receptive field: early layers $\approx$ edges/textures; late layers $\approx$ object parts and semantics

EFFECT	HYPERPARAMETER
Immediate neighborhood size (3×3 common)	Kernel size
Downsample spatially when >1	Stride
Keep spatial size (same padding) or control border	Padding



$$x \quad (\text{residual block learns } \mathcal{F}, \text{ identity carries gradient}) + \mathcal{F}(x) = y$$



.Standard CNN: Alternates Conv2D, Activation (ReLU), and Pooling (Max Pool) layers to extract spatial features and downsample

.AlexNet (2012): First large-scale CNN to win ImageNet; used ReLU, dropout, and multi-GPU training

.ResNet (2015): Introduced skip (shortcut) connections, allowing gradients to flow back easily through identity mappings. Enables training networks with 100+ layers

Landmark ideas (simplified)

IDEA YOU SHOULD REMEMBER	MODEL / BLOCK
Feature extraction + spatial downsampling	Conv + MaxPool
Scale + ReLU + data aug on GPUs	AlexNet
Residual mapping $F(x)+x$; skip paths help gradients	ResNet

.Pretraining on millions of labeled images (e.g. ImageNet) learns general low/mid-level filters (edges, textures) transferable to new domains

.Typical recipe: replace the final classifier head; freeze the pretrained backbone for small datasets, or fine-tune with a very small learning rate for larger datasets

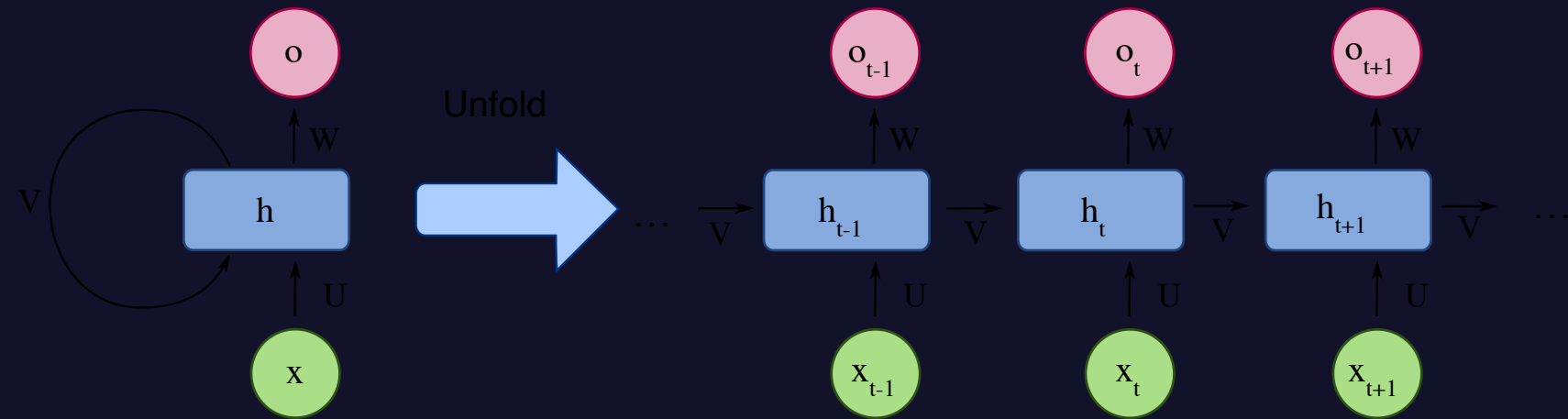
.Transfer learning dramatically reduces training time and required labeled samples for a new vision task

STRATEGY	TARGET DATASET SIZE
Freeze backbone, train head only	Small target dataset
Fine-tune top blocks with a smaller learning rate	Medium dataset
Full fine-tune of all layers with a small learning rate	Large target dataset



One Transition, Unfolded Through Time

$$\left(b + {}_tW_{xh}x + {}_{t-1}\phi\left(W_{hh}h = {}_th\right.\right.$$

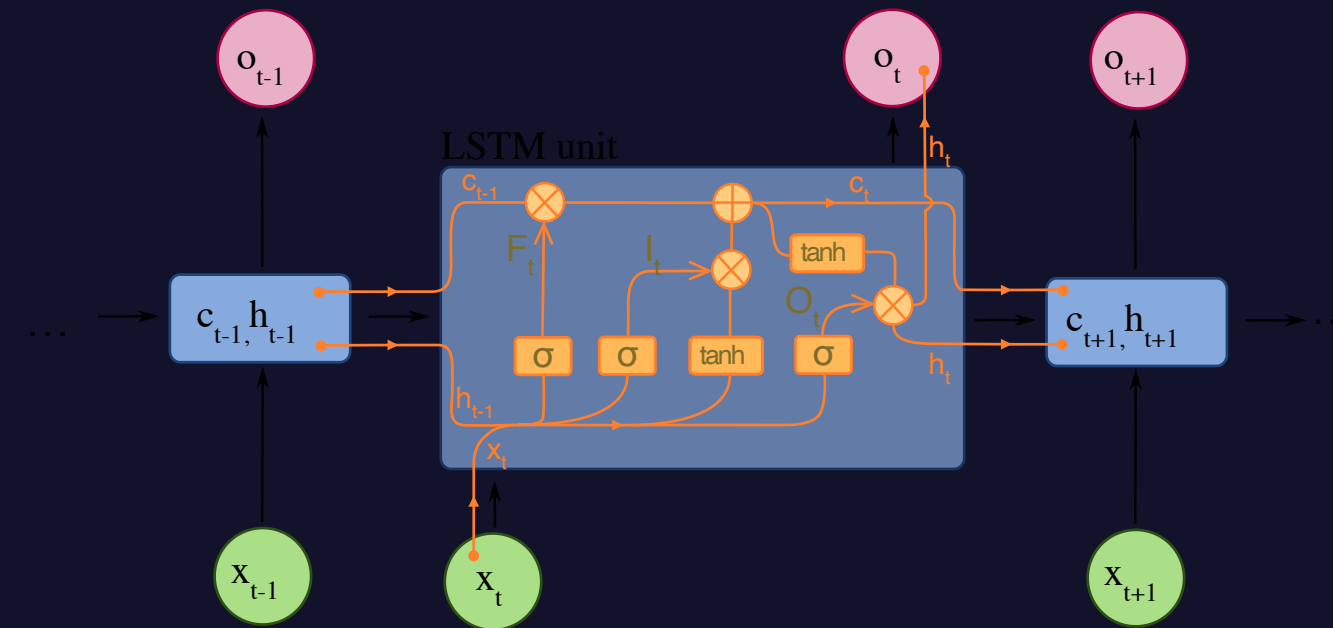


.Sequence modeling: text, audio, and sensor streams have order; shuffling them breaks their semantic meaning

.Recurrent Neural Networks (RNN): maintain a recurrent state vector $_th$ that acts as a memory summary of the past

.The same weights W_hh and W_xh are reused across all timesteps (parameter efficiency), but long-range dependencies suffer from vanishing/exploding gradients

LIMITATION		FEATURE	
Sequential execution (cannot parallelize steps)		Reuses parameters	
Cannot hold long-term memories in practice		$\mathcal{H}_t$ Tracks state	



.Cell state c_t can accumulate information with an additive path—mitigates vanishing signal compared with plain tanh RNNs

- .Forget gate $_t f$ decides how much past cell content to erase; input gate $_t i$ and candidate $\tilde{g}$ control new information written
- .Output gate $_t o$ filters what becomes hidden state $_t h$ exposed to the next layer or timestep

ROLE (INTUITION)	GATE
Erase irrelevant history from the cell	$_t f$
Allow new candidate information in	$_t i$
$_t h$ Expose part of the cell as	$_t o$

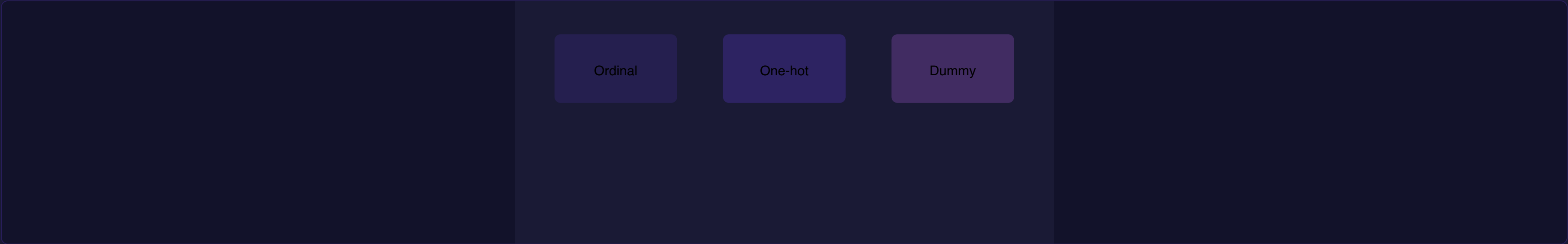
.Phase 4 explores unsupervised representation learning and basic generative frameworks

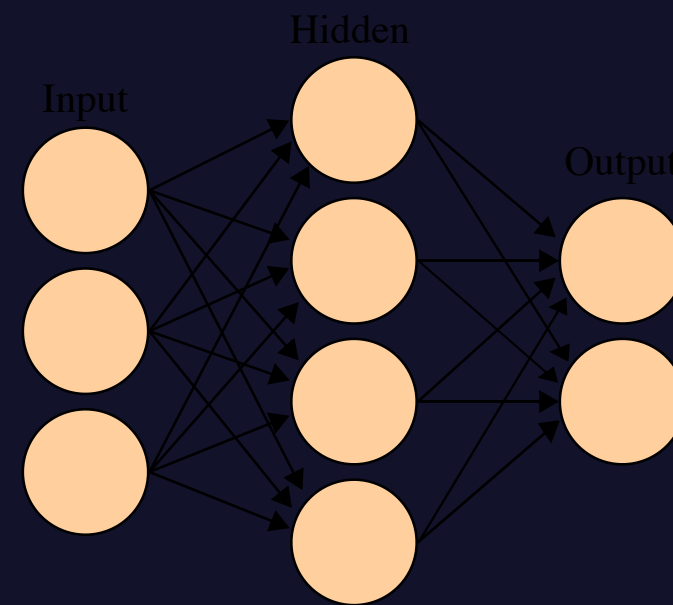
.Autoencoders are the foundation here: they learn to compress inputs to a low-dimensional bottleneck (latent space) and reconstruct them

GOAL	INPUT	CONCEPT
Compress and reconstruct $\hat{x} \approx x$	Unlabeled data x	Autoencoder



Encoder → Bottleneck → Decoder





.Encoder f maps input x to a low-dimensional code $z = f(x)$; decoder g maps z back to $\hat{x} \approx x$

.The bottleneck forces a compressed representation—useful for denoising (train on noisy→clean), anomaly detection (high recon error = abnormal)

.Linear AE with MSE relates to PCA when constraints align—nonlinear AE learns curved manifolds

IDEA	VARIANT
Corrupt input, reconstruct clean—robust features	Denoising AE
Penalty on activations—encourages informative sparse codes	Sparse AE
Penalize sensitivity of code to small input changes—smoother latent map	Contractive AE

Section Takeaways

.Neural nets learn features — depth helps but needs regularization .1

.Activations + initialization control gradient flow .2

.CNNs for grids, RNNs for sequences — match architecture to structure .3

?**Think:** What symptom tells you the network is underfitting vs overfitting

NLP Workshop: Foundations to Seq2Seq

SpaCy & NLTK

Bag-of-words

Text preprocessing

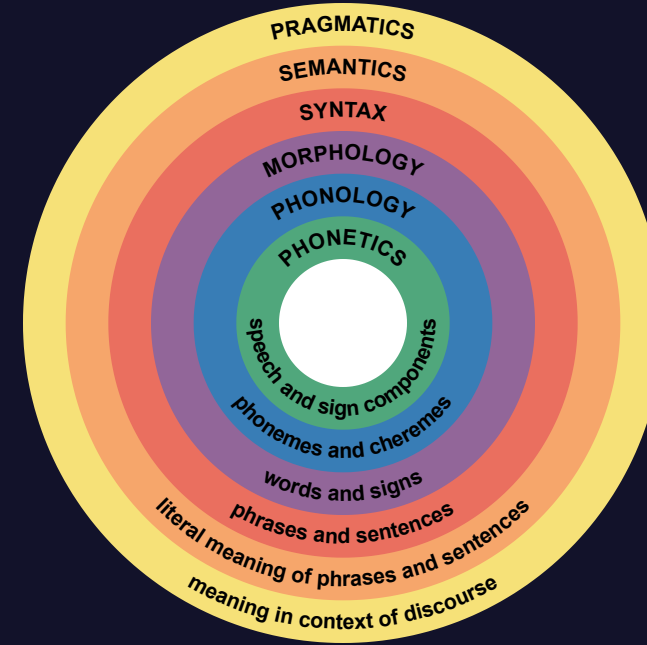
NLP pipeline

slides · ~51 min 32 ▶

?Where Are We in the Bootcamp

.Week 3 Session 1 — NLP goals, regex, normalization, and hands-on text cleaning

- S1** Foundations & Preprocessing •
- S2** Regression Models •
- S3** Classification Basics •
- S4** Naive Bayes & Trees •
- S5** SVM & Kernels •
- S6** Clustering & PCA •
- S7** Deep Learning •
- S8 NLP Fundamentals** •
- S9** Tokenization •
- S10** Text Analysis & NER •
- S11** Language Modeling •
- S12** Embeddings & RNNs •
- S13** Seq2Seq & NMT •
- S14** Generative AI •
- S15** RAG Systems •
- S16** MLOps •



.Natural language is ambiguous, context-dependent, and full of idioms

.Meaning often depends on pragmatics, domain, and cultural background

.The same sentence can map to multiple valid interpretations

.NLP systems must model form, meaning, and context simultaneously

POSSIBLE INTERPRETATION B	POSSIBLE INTERPRETATION A	EXPRESSION
River bank area is inaccessible	Financial institution is closed	The bank is closed
Emotionally distant behavior	Low temperature	It's cold
Animal is ready to eat food	Food is ready to be eaten	The chicken is ready to eat

.Robust NLP requires disambiguation using surrounding textual and situational context

Regular Expressions (Regex): Core Idea

- .Regex is a compact pattern language for matching and transforming text
- .Typical uses include validation, extraction, and rule-based cleanup
- .Useful for emails, phone numbers, logs, and normalization pipelines

EXAMPLE MATCH	MEANING	PATTERN
2026	one or more digits	+d\
nlp_model1	one or more word chars	+w\
Hello	starts with uppercase letter	]A-Z[^
token123	one or more non-space chars	+S\

Text Cleaning — Conceptual Steps & Python Code

Conceptual stages matched with Python regex implementations

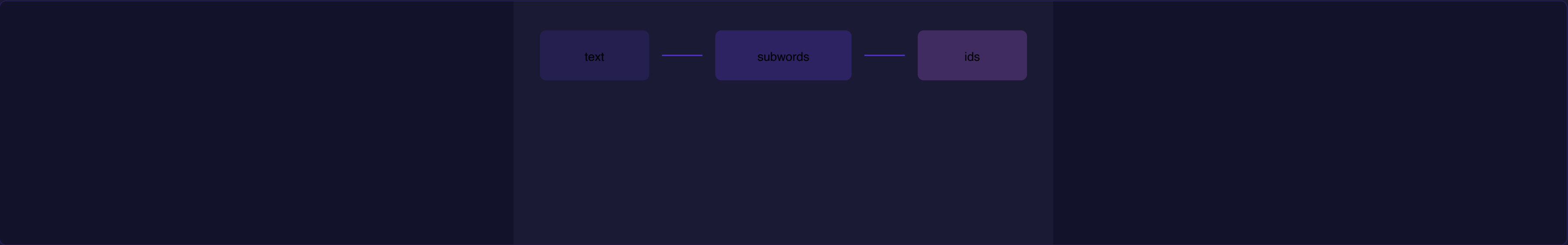
GOAL	PYTHON IMPLEMENTATION	STAGE
Keep readable text only	<code>html.unescape(text) & re.sub(r'<[^>]+>', '', text)</code>	Decode & Clean HTML .1
Drop URL metadata	<code>re.sub(r'https?://\S+ www\.\S+', '', text)</code>	Remove Contacts/URLs .2
Remove handles & hashtags	<code>re.sub(r'@\w+ #\w+', '', text)</code>	Remove Mentions/Socials .3
Drop emoji/special symbols	<code>text.encode('ascii', 'ignore').decode('ascii')</code>	Drop Non-ASCII & Emoji .4
Normalize extra spaces	<code>)(re.sub(r'^s+', ' ', text).strip)</code>	Normalize Whitespace .5

.A regex-based cleaning function is usually the first stage in an NLP preprocessing pipeline

Trace of raw input transitioning to clean text

TEXT SNAPSHOT	OPERATION PERFORMED	STAGE
John: "AI is amazing!!!! 🤖🔥 " Visit: https://ai.com @john #AI	Original raw string	Raw Input .0
John: "AI is amazing!!!! 🤖🔥 " Visit: https://ai.com @john #AI	Decode entities & remove tags	Clean HTML .1
John: "AI is amazing!!!! 🤖🔥 " Visit: @john #AI	Remove web addresses	Remove URLs .2
:John: "AI is amazing!!!! 🤖🔥 " Visit	Remove @mentions and #hashtags	Remove Socials .3
John AI is amazing!!!! Visit	ASCII-only encoding & cleanup	Drop Emoji/Symbols .4

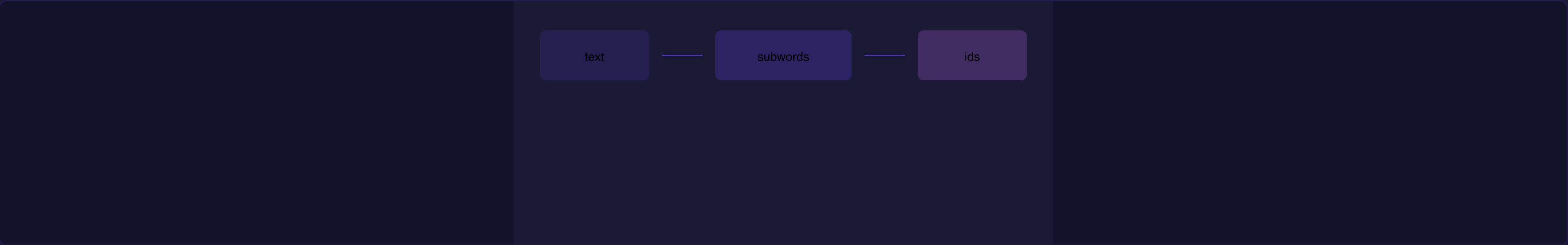
.Text cleaning produces standard, noise-free input for tokenizer components



EXAMPLE OUTPUT (FOR 'RUNNING')	TRADEOFF	STRENGTH	GRANULARITY
['running']	Weak on rare/compound words (OOV)	Human-readable tokens	Word / Space-based
['r', 'u', 'n', 'n', 'i', 'n', 'g']	Very long sequences, loss of meaning	No unknown words (no OOV)	Character-based
['run', '##ning']	Requires tokenizer training/tuning	Handles rare words, compact vocab	Subword-based

.Subword tokenization is the industry standard for modern LLMs and neural NLP models

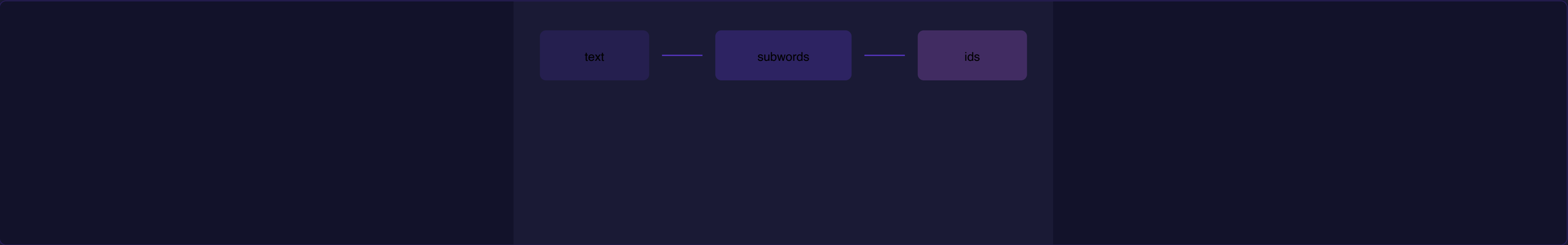
Same sentence, word / char / subword splits side by side



RESULT ON "I'M RUNNING TO THE STORE"	PYTHON IMPLEMENTATION	TOKENIZER TYPE
['!', 'I', "'m", 'running', 'to', 'the', 'store']	word_tokenize(text)	Word
[..., 'I', "'", 'm', ' ', 'r', 'u', 'n', ' ', 'i', 'n', 'g']	list(text)	Character
['!', 'I', "'m", 'run', '##ning', 'to', 'the', 'store']	model_tokenizer.tokenize(text)	Subword
["!I'm running to the store"]	sent_tokenize(text)	Sentence

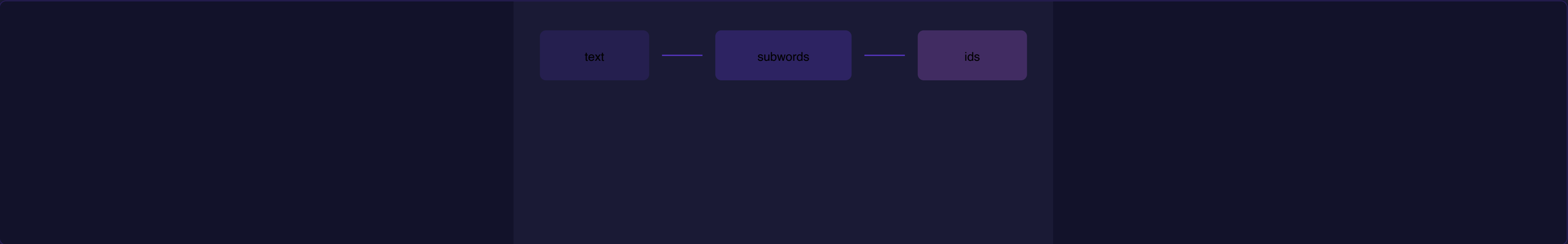
Subword Tokenization (LLMs) — Why It Wins

How subwords solve core vocabulary challenges



EXAMPLE WORKED OUT	SUBWORD SOLUTION	WORD TOKENIZER ISSUE
unbelievably → un + ##believe + ##ably	Reuse frequent morphemes/pieces	Huge vocabulary size
ChatGPT → ['Chat', 'G', 'PT']	Break unknown forms into known units	Out-of-Vocabulary (OOV) words
'running/runs/runner share 'run	Shared roots reduce feature sparsity	Morphological variants

How Byte Pair Encoding learns merges and splits unseen words



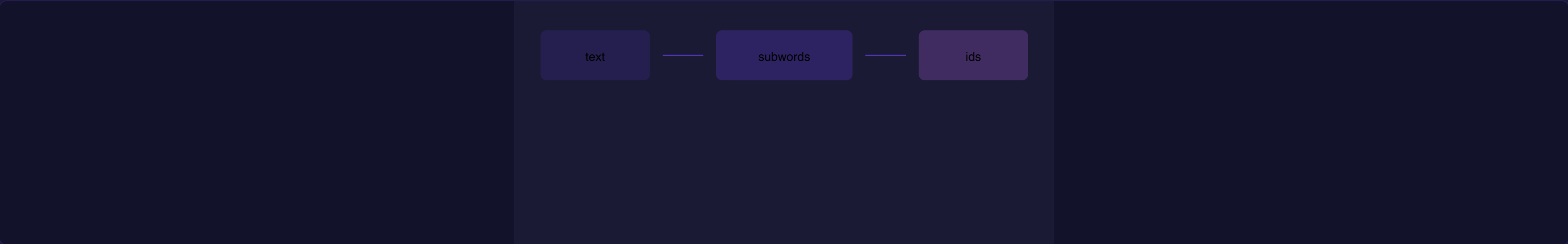
.Walkthrough: start with chars (l, o, w, e, s, t, r) -> merge l+o (lo) -> merge lo+w (low) -> compact vocabulary

.Inference on 'lowered the lowest curtain': splits rare words to ['low', 'er', 'ed'] and ['low', 'est'] while keeping 'the' and 'curtain' whole

EXPLANATION	SUBWORD OUTPUT	WORD
Decomposes into learned root and suffix subwords	]low', 'er', 'ed['	lowered
'Decomposes into base 'low' and suffix 'est	]low', 'est['	lowest

.BPE repeatedly merges frequent adjacent units until target vocabulary size is reached

Filtering low-information tokens from texts



.Function words (the, is, and) dominate word counts but carry minimal topic signal

.Removing stop words reduces dimensionality for TF-IDF & Bag-of-Words

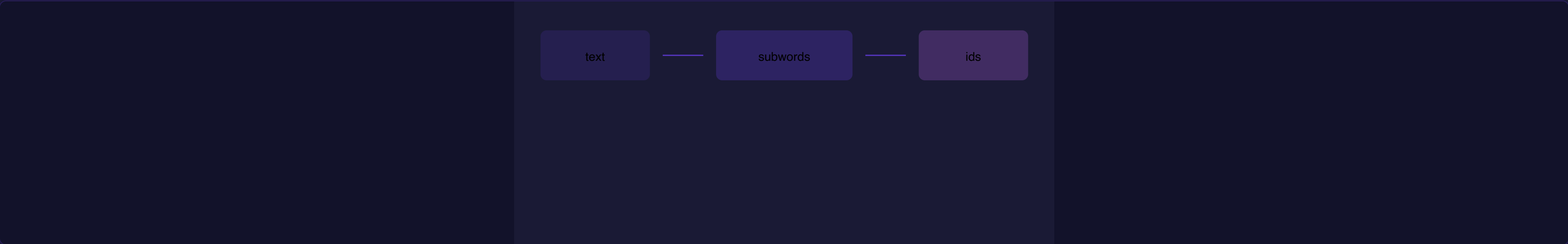
.Critical Rule: NEVER remove stop words for language models (GPT/BERT), translation, or sentiment analysis where negation ('not') and syntax carry the core meaning

REASONING	STOP WORD POLICY	TASK CATEGORY
Improves speed and focus on key words	Remove	Search & Topic Modeling
Syntactic and semantic flow is vital	Keep	Large Language Models
Negations like 'not' reverse polarity	Keep	Sentiment Analysis

Normalizing words to their base forms

LEMMATIZATION	STEMMING	ASPECT
Dictionary lookup + morphological analysis	Rule-based suffix chopping (heuristic)	Mechanism
Slower, requires resource lookup	Very fast, computationally cheap	Speed
Always dictionary words (e.g., study -> study)	Can be non-words (e.g., study -> studi)	Output
Yes (e.g., saw -> see or saw depending on POS)	No	POS Awareness

.Lemmatization preserves semantics better than aggressive stemming



- ... ,Universal categories include NOUN, VERB, ADJ, ADV, DET, ADP, PROPN, NUM
- .POS feeds lemmatization quality and downstream relation extraction
- ._SpaCy exposes coarse pos_ and fine-grained tag_ plus dependency dep

QUICK EXAMPLES	ROLE	POS
city, Tesla	Things and names	NOUN / PROPN
run, is	Actions and helpers	VERB / AUX
quick, quickly	Modifiers	ADJ / ADV
the, in	Structure words	DET / ADP

Typed Spans over Text

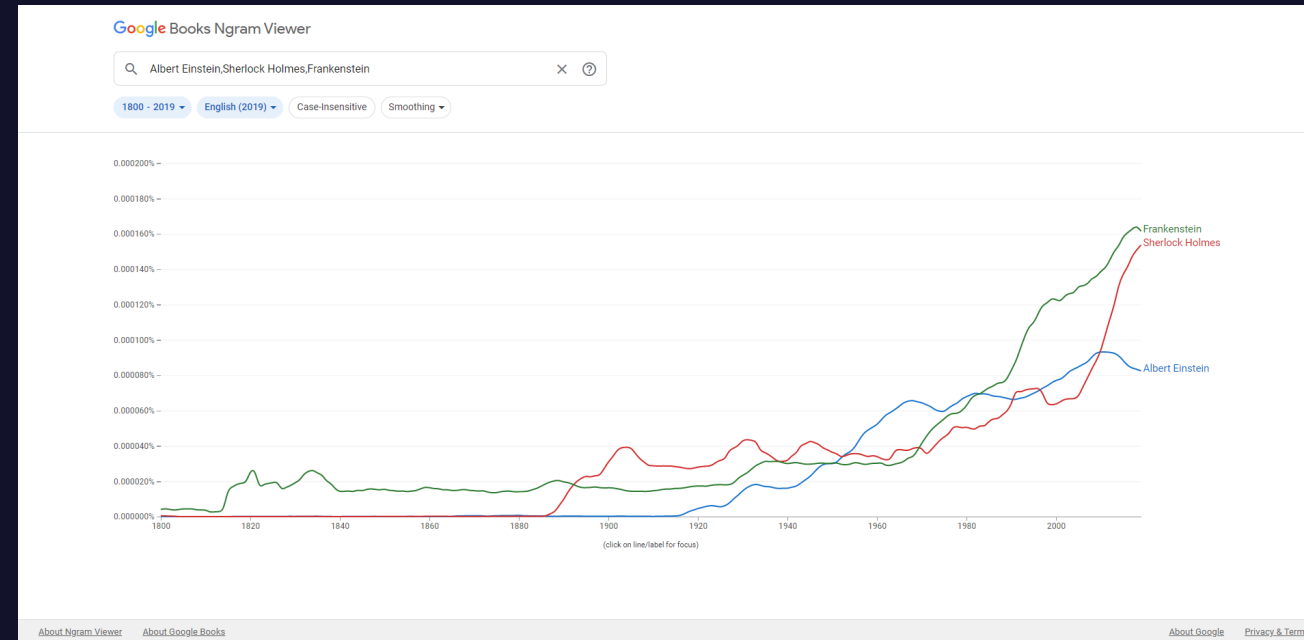
- ... ,Labels include PERSON, ORG, GPE, DATE, MONEY, PERCENT, PRODUCT, EVENT
- .Useful for indexing, compliance redaction, financial news graphs, and search facets
- .Small models err on edge cases — always spot-check domain text

EXAMPLES	LABEL
Tim Cook, Jensen Huang	PERSON
Apple, Texas	ORG / GPE
billion, 3.5% \$430	MONEY / PERCENT

.One doc object: tokens, lemmas, POS, entities, noun_chunks, sents

.Typical export: clean token list, lemma bag, entity list, POS histogram

.Use length-based heuristics (e.g. longest sentence) only as weak importance cues



.Language models estimate probabilities over word sequences

.N-gram models predict the next token from a limited context window

.This module connects N-gram intuition to modern LLM generation

$$(w_1, \dots, w_T \mid w_{t-1}) P(w_1, \dots, w_T) = P(w_1, \dots, w_T)$$

.A language model can score full sentences or predict the next word

.Exact estimation over full histories is intractable for real corpora

.Approximation strategies include N-grams, smoothing, and neural LMs

Given counts: $C(\text{"want to"}) = 608$, $C(\text{"want"}) = 927$

$$0.656 \approx \frac{608}{927} = P(\text{"to"} | \text{"want"})$$

. Interpretation: in this corpus, "to" follows "want" about 65.6% of the time

. Count quality depends strongly on corpus domain and size

- .Multiplying many small probabilities causes numerical underflow
- .Log transform converts multiplication into stable addition
- .Sequence scoring becomes computationally robust and efficient

LOG SPACE	ORIGINAL SPACE
$\log P = \log p_1 + \log p_2 + \dots + \log p_k$	$P = p_1 * p_2 * \dots * p_k$
Numerically stable sums	Very small numbers

$$\exp \left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t | h_t) \right) = \frac{1}{T} P(w_1, \dots, w_T) = \text{PP}(W)$$

.Perplexity is the average branching uncertainty of the model

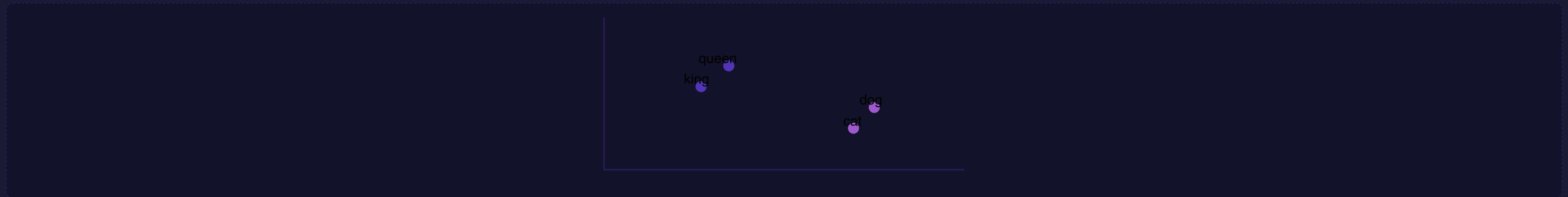
.Lower PP means better predictive confidence on unseen text

.Use identical test sets when comparing models

.Raw sentence probabilities are length-sensitive; perplexity normalizes by token count

CONTEXTUALIZED EMBEDDINGS (ELMO/BERT/GPT)	STATIC EMBEDDINGS (WORD2VEC/GLOVE)	PROPERTY
Different vectors per context	One fixed vector	Vector per word
Strong	Weak	Polysemy handling
Bidirectional or autoregressive sequence context	Usually local/global corpus only	Context direction
High with pretraining + fine-tuning	Moderate	Task transfer

.Contextualization is a core enabler for modern LLM quality



Words as vectors in embedding space

.Static embeddings: one vector per word type

.Contextual: vector depends on surrounding tokens (ELMo, BERT-style)

Architecture Layers

.Token/base embedding layer maps input ids to dense vectors

.Context encoder layers (often Transformer blocks) enrich each token with sequence context

.Task head uses contextual features for prediction

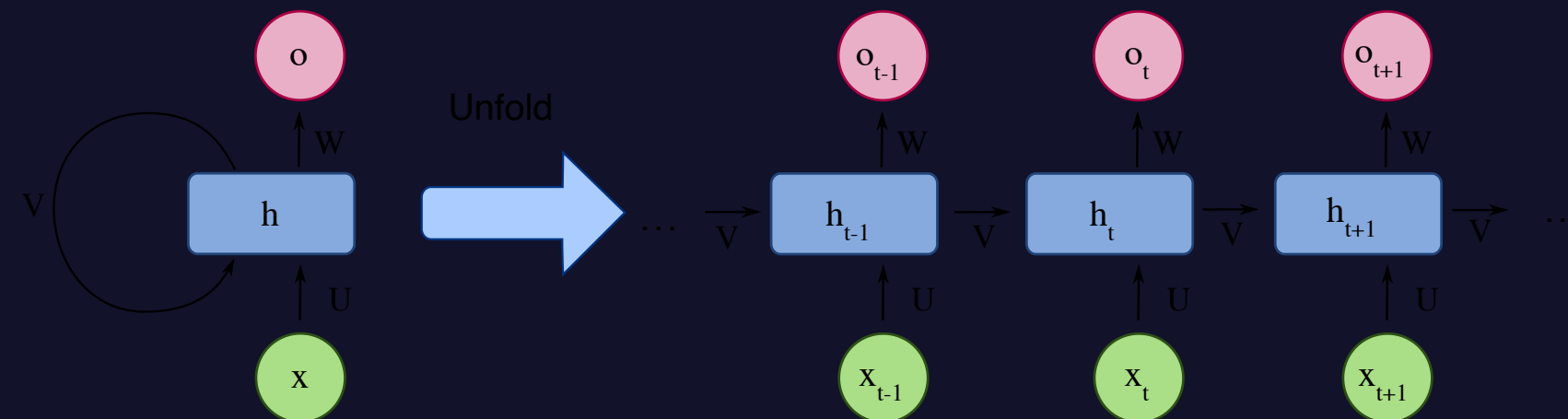
Layer Semantics

.Lower layers capture local syntax/patterns

.Middle layers improve sense disambiguation

.Upper layers capture richer semantics and task-specific signals

$$(b + {}_{t-1}W_h h + {}_t f(W_x x = {}_t h$$



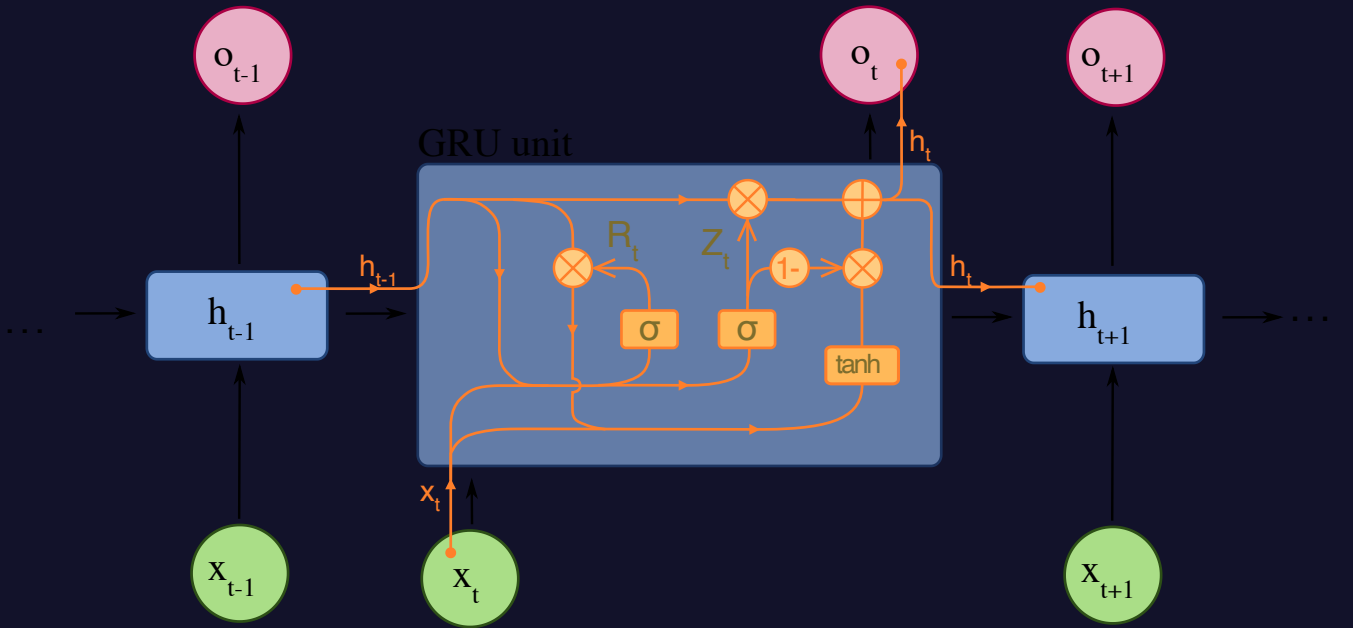
.At time t, the RNN consumes current input and previous hidden state

.The same cell and weights are reused at every time step

.This recurrence captures sequential information in variable-length text

EXAMPLE TASK	MAPPING	PATTERN
basic regression/classification	single input -> single output	One-to-One
image captioning	single input -> sequence output	One-to-Many
sentiment classification	sequence input -> single output	Many-to-One
machine translation	sequence input -> sequence output	Many-to-Many

.Sequence-aware patterns are where recurrent models provide clear value



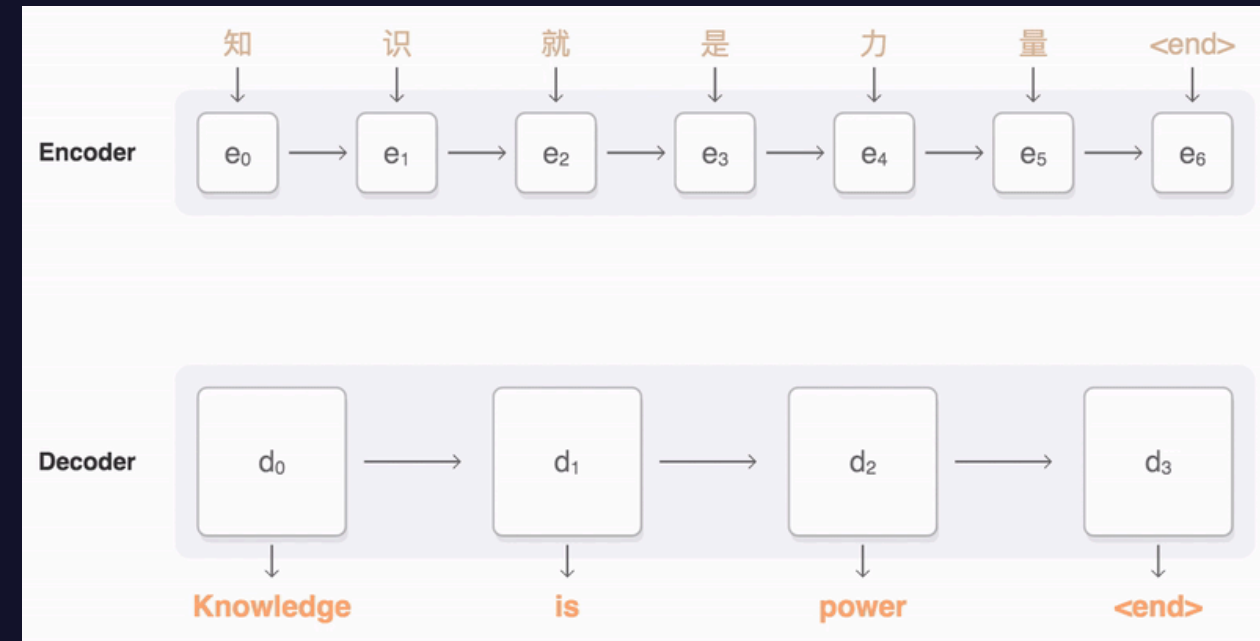
TYPICAL LIMITATION	KEY STRENGTH	VANISHING GRADIENT RISK	MODEL
Poor long-range memory	Simple architecture	High	Vanilla RNN
Can still degrade on very long contexts	Efficient gating with fewer parameters	Lower	GRU
Higher compute and memory	Uses past and future context	Medium	BiRNN
Heavier than GRU	Strong long-term dependency handling	Low (relative)	LSTM



Ordinal

One-hot

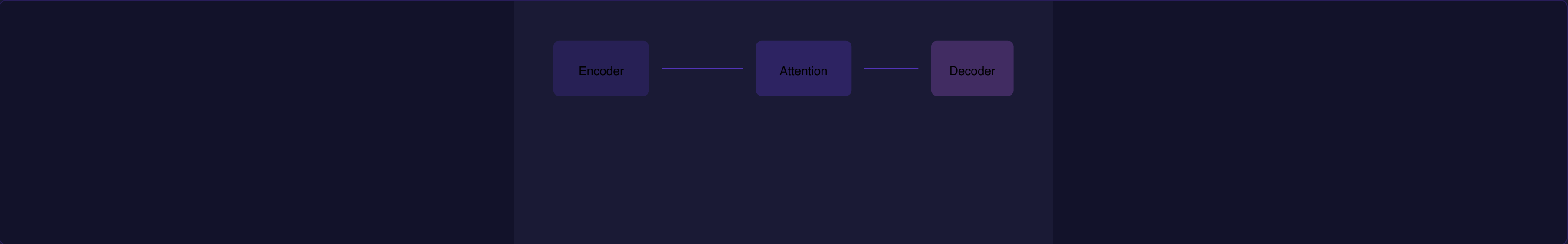
Dummy



.Seq2Seq maps variable-length input sequences to variable-length outputs

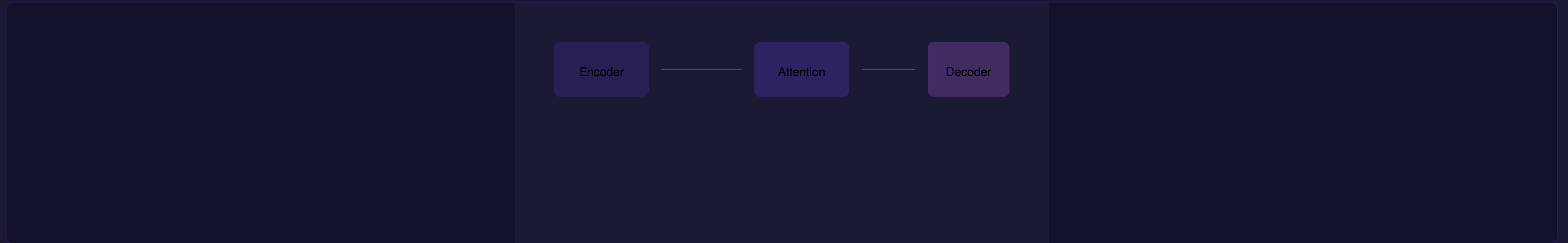
.Encoder-decoder models were foundational for neural machine translation

.This module covers training, decoding, bottlenecks, attention, and metrics



RISK	STRENGTH	DECISION RULE	METHOD
May miss globally better sequence	Fast and simple	Pick top token each step	Greedy
Higher compute; can become generic/bland	Better global sequence quality	Track top-k hypotheses each step	Beam Search

.Typical beam sizes are moderate (e.g., 4-10) to balance quality and cost

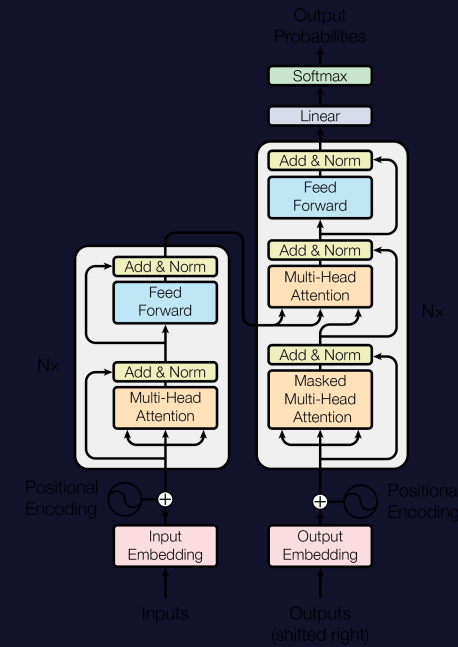


.Compressing a full source sentence into one fixed vector can lose detail

.Longer/complex inputs worsen the bottleneck effect

.Decoder needs different source details at different output steps

.This motivates attention over all encoder hidden states

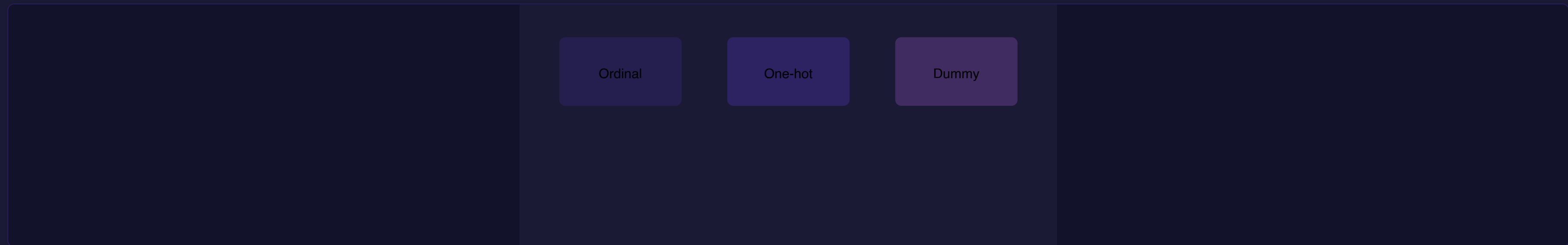


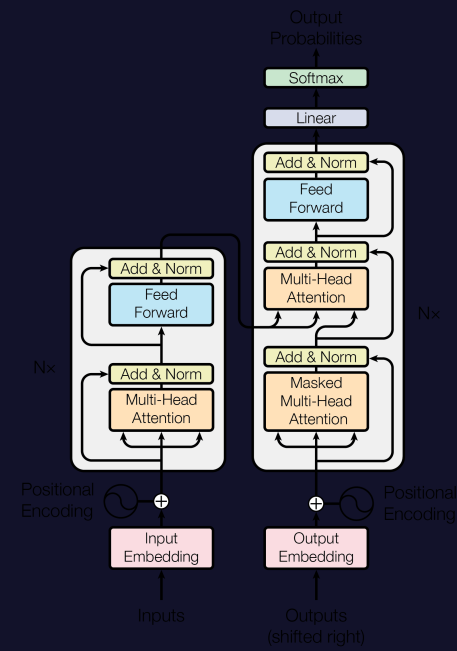
.Decoder attends to relevant encoder positions at each generation step

.Dynamic alignment improves translation adequacy and fluency

.Attention laid the foundation for transformer-dominant NMT systems

COMMON LIMITATION	WHAT IT MEASURES	ORIENTATION	METRIC
Weak semantic/syntactic sensitivity	n-gram overlap from candidate to reference	Precision-oriented	BLEU
Can reward lexical overlap over meaning	n-gram overlap from reference to candidate	Recall-oriented	ROUGE-N
Still limited on deep semantics	Harmonic tradeoff view	Balance precision/recall	F1 (with overlap metrics)





.Encoder builds contextual token representations from source text

.Decoder generates target tokens autoregressively with attention to encoder states

.Foundation architecture for translation, summarization, and many LLM pipelines

.Attention mechanisms reduce reliance on recurrence for sequence modeling

Section Takeaways

.Language is ambiguous — preprocessing choices persist through the pipeline .1

.Clean raw text before tokenization; regex handles structured patterns .2

.The full NLP pipeline: ingest → clean → tokenize → analyze → model .3

?**Think:** Which cleaning step would break downstream NER if skipped

GenAI: BERT, T5, GPT, Attention

LLM scaling

Fine-tuning

BERT & GPT

Transformers

slides · ~35 min 22 ▶

?Where Are We in the Bootcamp

.Pretrain at scale, fine-tune for tasks, generate with decoders

- S1 Foundations & Preprocessing •
- S2 Regression Models •
- S3 Classification Basics •
- S4 Naive Bayes & Trees •
- S5 SVM & Kernels •
- S6 Clustering & PCA •
- S7 Deep Learning •
- S8 NLP Fundamentals •
- S9 Tokenization •
- S10 Text Analysis & NER •
- S11 Language Modeling •
- S12 Embeddings & RNNs •
- S13 Seq2Seq & NMT •
- S14 Generative AI •
- S15 RAG Systems •
- S16 MLOps •



BERT, T5, GPT, LLMs, and Attention (Focused Edition)

.This section extracts the most important ideas into a compact teaching flow

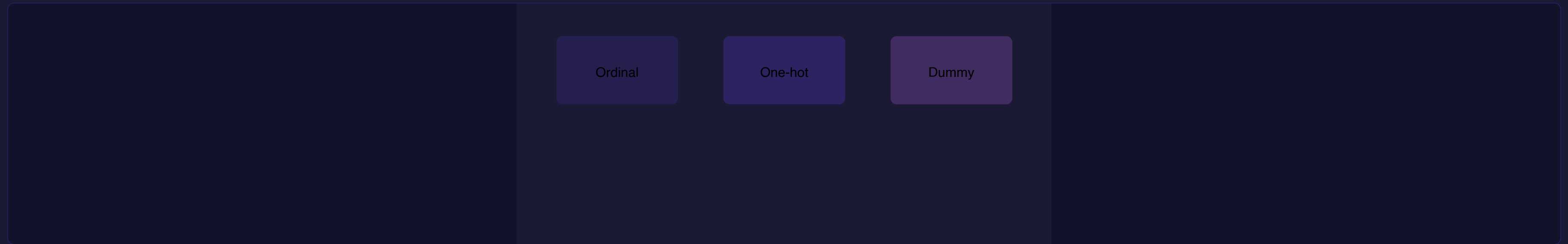
.Focus areas: transfer learning, model families, attention mechanics, scaling, and practical deployment

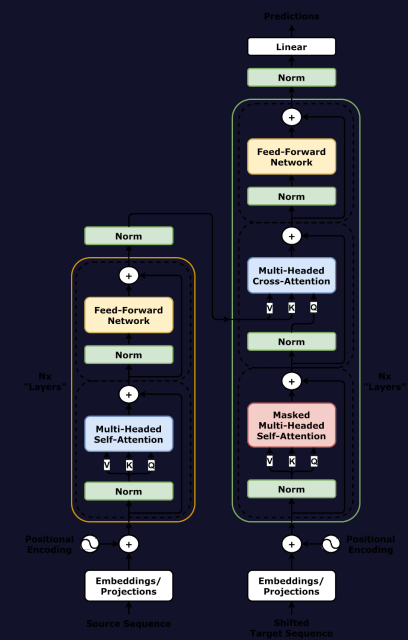
.Designed for bootcamp delivery within roughly 20 slides

GOAL	DATA TYPE	STAGE
Learn general language representations	Large mostly unlabeled corpora	Pretraining
Adapt model to specific downstream objective	Task-labeled dataset	Fine-tuning
Generate or classify outputs for real tasks	User prompt/context	Inference

.Transfer learning reduces labeled data needs and accelerates convergence

TRADEOFF	PROS	WHAT CHANGES	STRATEGY
Less task adaptation	Fast and lightweight	Freeze pretrained backbone; use embeddings as input features	Feature-based
More compute and tuning effort	Higher task performance	Update model weights (often with a task head)	Fine-tuning





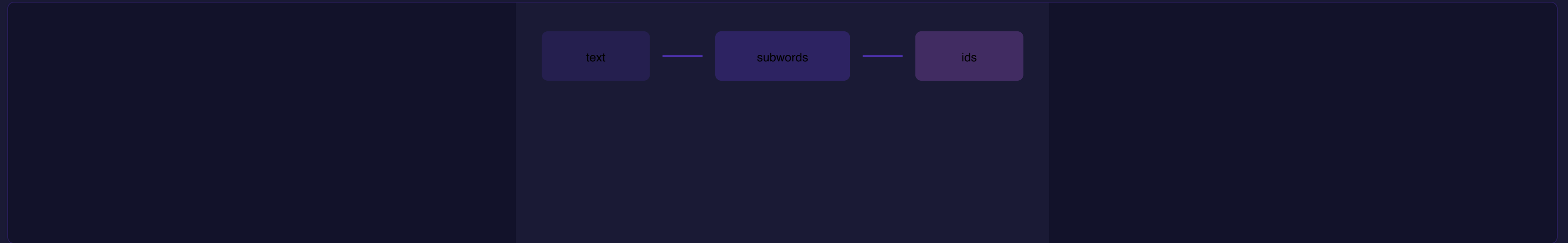
.BERT learns contextual token representations using both left and right context

.Strong for understanding tasks: classification, NER, QA, and retrieval-oriented use cases

.Pretrained at scale (Wikipedia + BooksCorpus), then adapted to downstream tasks

WHAT IT TEACHES	HOW IT WORKS	OBJECTIVE
Bidirectional contextual semantics	Mask subset of tokens and predict originals	MLM (Masked Language Modeling)
Inter-sentence coherence signals	Predict whether sentence B follows sentence A	NSP (Next Sentence Prediction)

.MLM and NSP were jointly used in original BERT training setup



[CLS] .is prepended and its final embedding is used as a sequence-level summary

[SEP] .separates sentence segments and marks boundaries

[CLS] .In classification tasks, [CLS] passes through a lightweight prediction head

[CLS] .Token design helps unify single- and pair-sentence tasks

- .AraBERT follows BERT-base style architecture with Arabic-specific preprocessing
- .Segmentation and Arabic-tailored tokenization improve vocabulary coverage
- .Domain/language adaptation quality can matter more than raw parameter count

ADAPTATION BENEFIT	COMPONENT
Better handling of prefixes/suffixes and morphology	Farasa-style segmentation
Lower fragmentation and stronger lexical coverage	Arabic-focused vocabulary
Improved contextual understanding in target language	Arabic corpus curation



.T5 reframes every NLP task as text input -> text output

.Task instruction is included as a prefix (e.g., "summarize:", "translate:")

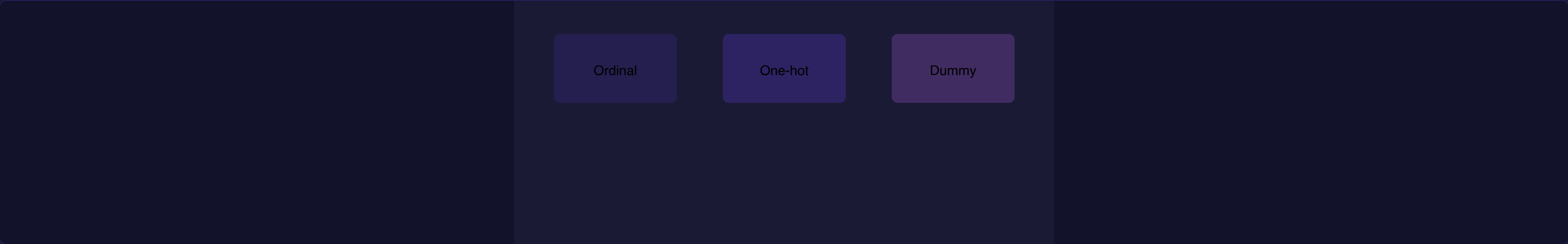
.One shared architecture/training recipe serves multiple tasks

.This design strongly influenced modern prompt-based LLM usage

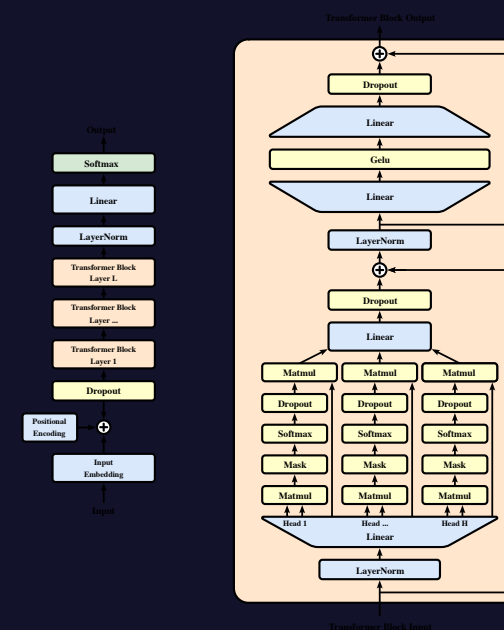
T5 Pretraining Objective: Span Corruption

- .Remove contiguous text spans from input and replace with sentinel tokens
- .Train model to reconstruct missing spans autoregressively
- .Encourages stronger generative behavior than token-only masking

TYPICAL STRENGTH	CORRUPTION STYLE	MODEL
Language understanding tasks	Random masked individual tokens	BERT
Generation + multitask transfer	Contiguous span corruption	T5



USED IN	VISIBILITY	ATTENTION TYPE
Encoder representations	All tokens attend to all tokens	Fully visible
Autoregressive generation	Token attends to past only	Causal
Seq2Seq decoding in T5	Full source access + causal target decoding	Prefix-causal hybrid



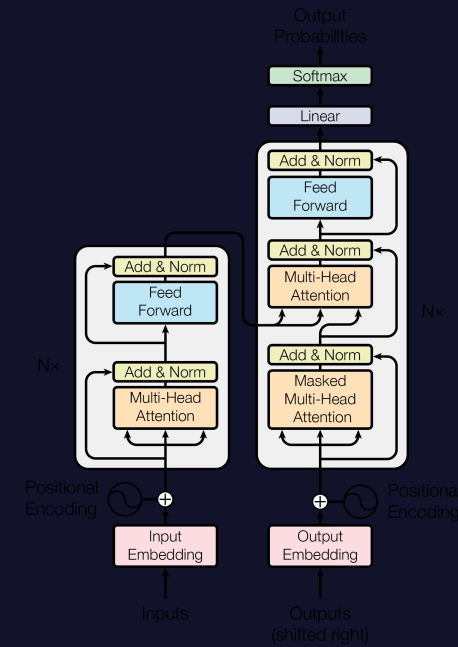
.GPT predicts next token from left context only (causal modeling)

.Uses masked self-attention in decoder stack for no-lookahead generation

.Excellent for open-ended completion, instruction following, and synthesis

KEY THEME	APPROX. PARAMETERS	GENERATION
Proof of transfer-learning viability	117M	GPT-1
Large-scale web pretraining for generation	1.5B	GPT-2
Few-shot in-context capabilities at scale	175B	GPT-3

.Model utility scales with data, architecture, and training compute quality



.Attention solves fixed-vector bottlenecks in sequence transduction

.At each step, model dynamically focuses on most relevant source positions

.Entire mechanism is differentiable and learned end-to-end

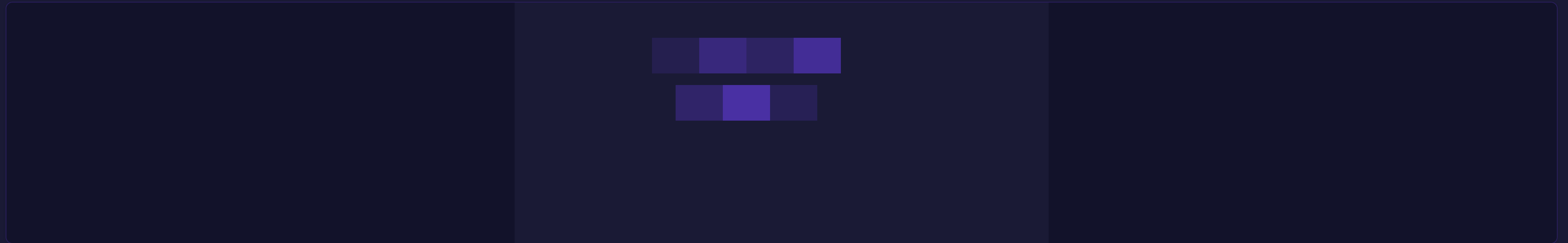


$$\text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)V = \text{Attention}(Q, K, V)$$

- .Compute relevance scores between query state and source states
- .Apply softmax to obtain attention weights (probability distribution)
- .Return weighted sum of value/source states as contextual output

TYPICAL PLACEMENT	SCORING FUNCTION	VARIANT
Attention after decoder state update	Bilinear/dot-style	Luong
Attention integrated before decoder update	MLP/additive score	Bahdanau

.Both approximate alignment; design choice affects speed and accuracy tradeoffs

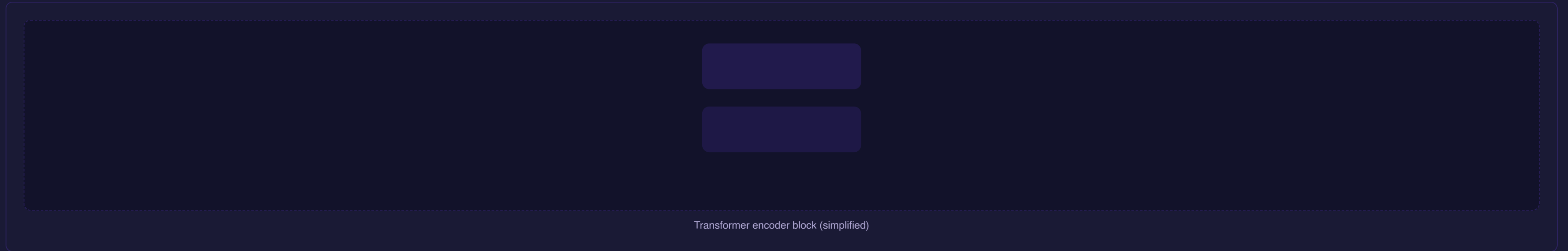


.Query asks for relevant context; Key indexes available context; Value carries content

.Self-attention lets each token aggregate information from other tokens in sequence

.Multi-head attention captures different linguistic relations in parallel

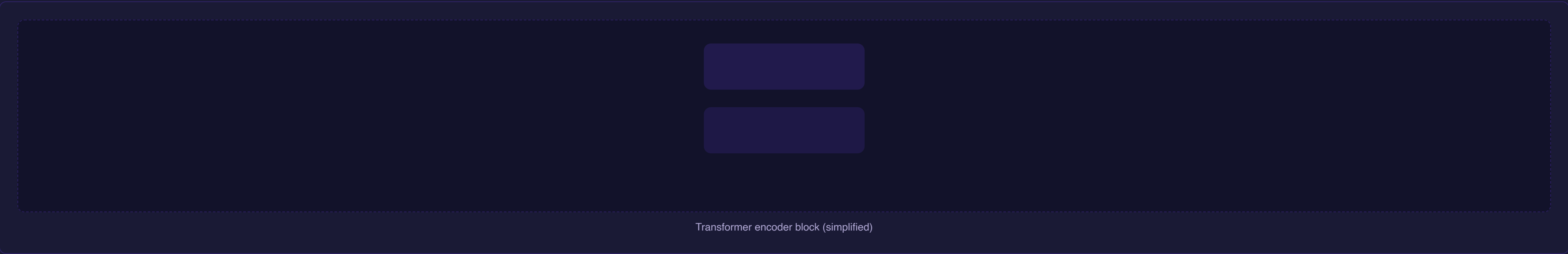
.Different heads can specialize in syntax, agreement, locality, or semantics



.Decoder cannot access future tokens during generation

.Causal mask enforces left-to-right consistency and prevents information leakage

.Enables parallel training over full target sequence while preserving autoregressive objective



PRACTICAL IMPACT	COMPLEXITY	BOTTLENECK
Time/memory explode as context grows	$O(L^2)$	Attention map computation
High VRAM demand for deep long-context models	$O(N \cdot L \cdot d_{model})$	Activation storage (training)

Long-context efficiency is now a central LLM engineering topic

- .Performance improves with data, model size, and compute, but with diminishing returns
- .Compute-optimal training balances parameter count and token budget
- .Chinchilla-style insight: many large models are under-trained relative to their size

IMPLICATION	PRINCIPLE
Tune model size and data volume jointly	Fixed compute budget
Small fraction of tokens may dominate useful learning	Data-quality filtering
Roughly ~20 training tokens per parameter	Chinchilla ratio (rule of thumb)

.Choose architecture by task type: encoder-only, encoder-decoder, or decoder-only

.Prefer transfer learning and fine-tuning over training from scratch when possible

.Use attention-aware designs and decoding strategy (greedy/beam) based on product constraints

.Plan for memory/compute early: quantization, context limits, and deployment targets

.For domain-heavy use cases, adapt with specialized corpora and evaluation protocols

.Strong GenAI systems come from balanced choices across data, architecture, compute, and evaluation

Section Takeaways

.Encoder models understand; decoder models generate; seq2seq models both .1

.Attention lets models focus on relevant context per token .2

.Scaling laws trade compute for capability — plan responsibly .3

?**Think:** Which GenAI risks matter most for your organization's data

RAG

Retrieval-Augmented Generation

RAG evaluation

Augmented prompts

Chunking & retrieval

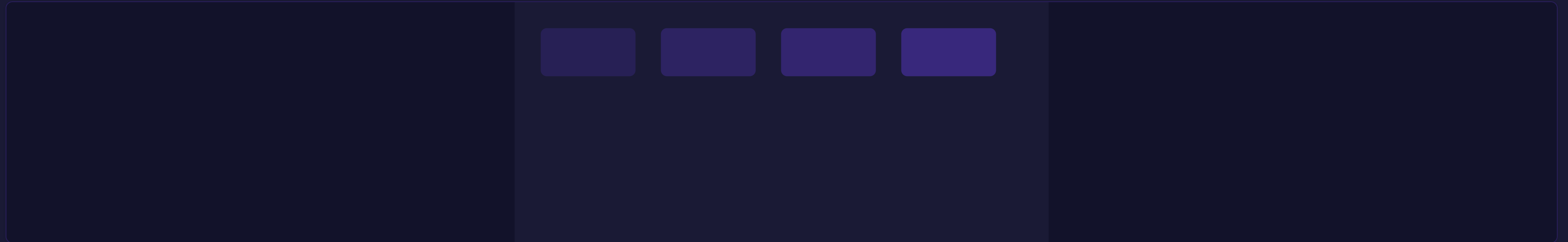
Vector stores

slides · ~22 min 14 ▶

?Where Are We in the Bootcamp

.Ground LLM answers in your documents with retrieval-augmented generation

- S1 Foundations & Preprocessing •
- S2 Regression Models •
- S3 Classification Basics •
- S4 Naive Bayes & Trees •
- S5 SVM & Kernels •
- S6 Clustering & PCA •
- S7 Deep Learning •
- S8 NLP Fundamentals •
- S9 Tokenization •
- S10 Text Analysis & NER •
- S11 Language Modeling •
- S12 Embeddings & RNNs •
- S13 Seq2Seq & NMT •
- S14 Generative AI •
- S15 RAG Systems •
- S16 MLOps •



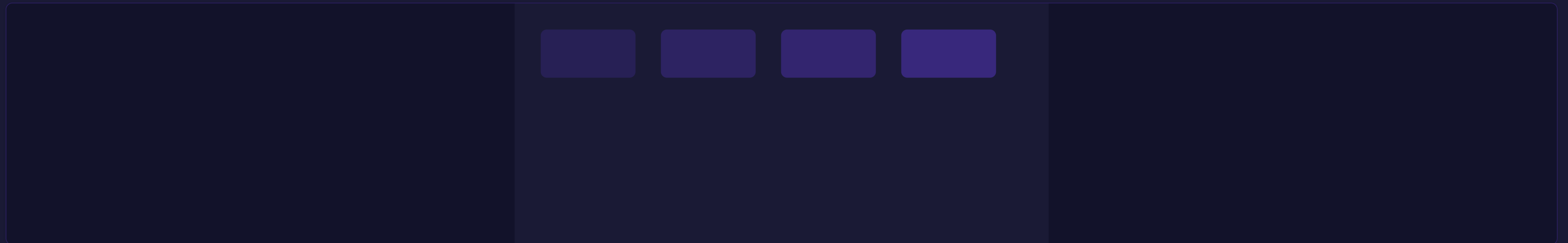
.LLMs alone lack private, up-to-date, or domain-specific knowledge

.RAG retrieves relevant documents at query time and injects them into the prompt

.Reduces hallucination risk and lets you cite sources in enterprise settings



Index → Retrieve → Augment → Generate



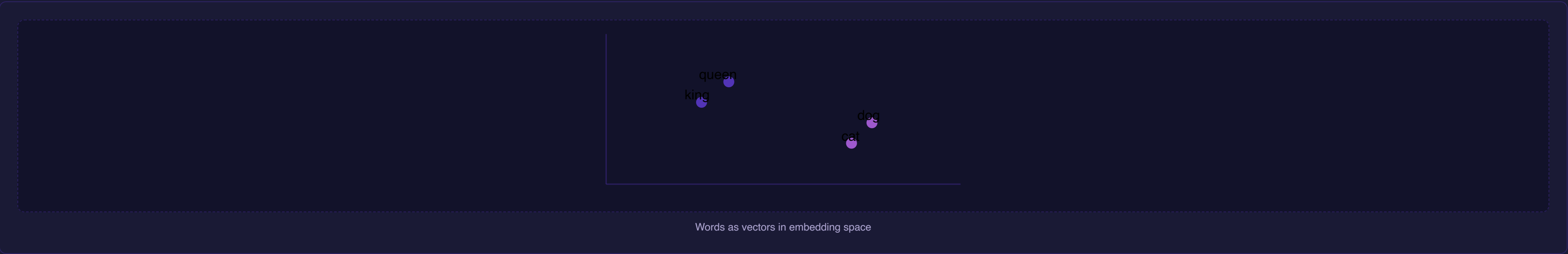
.Offline: chunk documents → embed → store in a vector index

.Online: embed user query → retrieve top-k chunks → build augmented prompt

.LLM generates answer conditioned on retrieved context

.Think of RAG as search + synthesis, not search alone

- .Fixed-size windows (e.g., 512 tokens) — simple but may split semantics
- .Semantic / paragraph chunks — better coherence, variable length
- .Overlap between chunks improves recall for boundary facts



EXAMPLES	ROLE	COMPONENT
OpenAI, Cohere, sentence-transformers	Maps text to dense vectors	Embedding model
FAISS, Pinecone, Chroma, pgvector	Similarity search at scale	Vector DB
JSON fields on each chunk	Filter by tenant, date, ACL	Metadata

?Recall@k: did the right chunk appear in top results

.Hybrid search (BM25 + vectors) often beats either alone

.Re-rankers can sharpen top results before the LLM sees them

.A robust template states rules, lists sources, then asks the question



.Instruct the model to answer only from provided context

.Include chunk boundaries and source IDs for traceability

.Handle empty retrieval with a safe fallback message



.Even with context, models may invent facts — evaluate faithfulness

.Ask the model to quote supporting spans when possible

.Human review for high-stakes domains (legal, medical, finance)



	WHAT IT MEASURES	METRIC
	Correct chunk in top-k	Retrieval recall
	Response addresses the user question	Answer relevance
	Claims supported by retrieved text	Faithfulness
	End-to-end SLA and token usage	Latency / cost

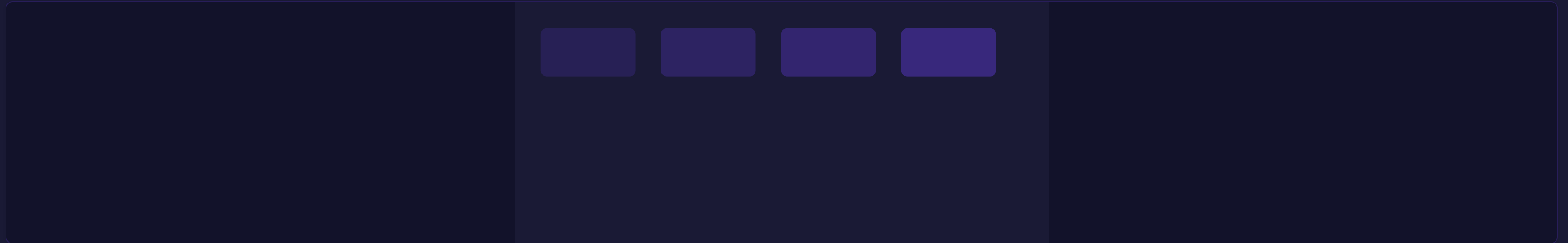
.Chunks too large — dilute relevance; too small — lose context

.Stale index — documents updated but embeddings not refreshed

.ACL leaks — retrieving chunks the user should not see



TRADEOFF	BEST WHEN	APPROACH
Retrieval quality is critical	Knowledge changes often; need citations	RAG
Expensive refresh cycle	Style/format/domain phrasing	Fine-tuning
Higher ops complexity	Regulated domain + brand voice	Both



.Embed the user query and every document chunk

.Retrieve the top chunk via similarity search

.Inject retrieved text into the LLM prompt before generation

.Versioned ingestion pipeline with chunk + embed + index jobs

.Observability: log queries, retrieved IDs, latency, user feedback

.Governance: PII scrubbing, retention policy, and access-controlled indexes

Section Takeaways

.RAG = retrieve relevant chunks → augment prompt → generate answer .1

.Chunking strategy and embeddings quality dominate retrieval recall .2

.Evaluate faithfulness — models can still hallucinate with context .3

?**Think:** What document boundary would cause harmful retrieval for your domain

MLOps

Production ML Systems

Production serving

Monitoring & drift

Model registry

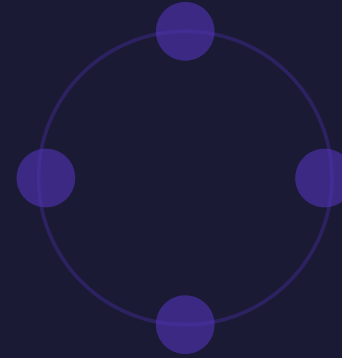
CI/CD for ML

slides · ~24 min 15 ▶

?Where Are We in the Bootcamp

.Ship models reliably: versioning, monitoring, and automated pipelines

- S1 Foundations & Preprocessing •
- S2 Regression Models •
- S3 Classification Basics •
- S4 Naive Bayes & Trees •
- S5 SVM & Kernels •
- S6 Clustering & PCA •
- S7 Deep Learning •
- S8 NLP Fundamentals •
- S9 Tokenization •
- S10 Text Analysis & NER •
- S11 Language Modeling •
- S12 Embeddings & RNNs •
- S13 Seq2Seq & NMT •
- S14 Generative AI •
- S15 RAG Systems •
- S16 MLOps •



.MLOps applies DevOps discipline to data, models, and experiments

.Goal: reproducible training, safe deployment, and continuous monitoring

.Bridges data science, engineering, and operations teams

.Data pipelines — ingestion, validation, feature stores

.Training pipelines — experiment tracking, model registry

.Serving — APIs, batch scoring, edge deployment

.Monitoring — data drift, concept drift, performance decay

WHY VERSION IT	ARTIFACT
Recreate training exactly	Code + config
Audit and debug drift	Training data snapshot
Rollback and compare	Model weights
Same dependencies in train and serve	Environment

.Log parameters, metrics, artifacts, and git SHA per run

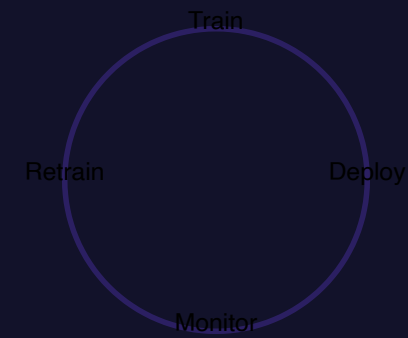
.Compare runs side-by-side before promoting a model

.Tools: MLflow, Weights & Biases, Neptune, cloud-native trackers

.Stages: staging → production → archived

.Promotion gates: metric thresholds, bias checks, sign-off

.Never deploy straight from a laptop without registry metadata



MLOps continuous delivery loop

.CI: unit tests + data/schema validation + training smoke tests

.CD: deploy container or endpoint when registry stage changes

.Feature pipelines may need separate release cadence from models

EXAMPLE SIGNAL	DEFINITION	TYPE
New user demographics	Input distribution changes	Data drift
Fraud patterns evolve	$P(y x)$ changes	Concept drift
Stricter QA on positives	Labeling policy changes	Label drift

.Track latency, error rate, throughput at the serving layer

.Track prediction distribution and feature statistics over time

.Alert on SLA breaches and statistical tests vs training baseline

.Centralize features for training and serving to prevent train-serve skew

.Same feature definitions online and offline

.Point-in-time correctness for historical training sets

.Reuse across teams — reduces duplicated ETL

.Batch scoring for reports and nightly jobs

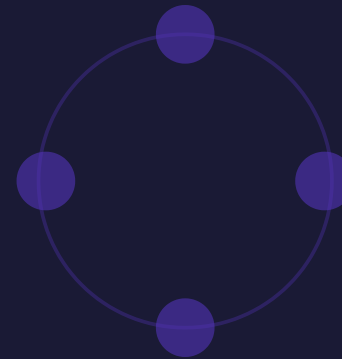
.Real-time REST/gRPC for interactive products

.Shadow / canary deploys to compare new model safely

.Log hyperparameters and evaluation metrics per training run

.Tag runs with git SHA and data version for auditability

.Push artifacts to MLflow, W&B, or your cloud tracker in production



.Pick one model from the bootcamp and draw its production path

.List 3 metrics you would monitor week one after launch

.Identify the single biggest reproducibility gap in your current workflow

Section Takeaways

.MLOps connects code, data, and models with reproducible pipelines .1

.Monitor drift — data and concept drift degrade silent production models .2

.CI/CD for ML includes data validation, not just unit tests .3

?**Think:** Which production incident would your current process miss

Key Takeaways

.Good ML pipelines start with strong preprocessing and proper validation

.Model choice depends on data structure, interpretability needs, and scale

.For classification, evaluate with confusion matrix, precision, recall, and F1

.For unsupervised tasks, combine clustering diagnostics with PCA-based interpretation

.For deep learning, balance architecture power with regularization and validation discipline

.For NLP, robust preprocessing and tokenization choices strongly shape model outcomes

.Tokenizer design directly impacts OOV handling, sequence length, and model quality in NLP

.For language modeling, probability estimation quality and perplexity tracking are critical

.For sequence tasks, contextual representations and attention mechanisms are key drivers of quality

.For MT/NLG, combine BLEU and ROUGE with qualitative semantic validation

.For modern GenAI, balance architecture choice, scaling strategy, and compute efficiency

.RAG grounds LLMs in your documents; MLOps ships and monitors models reliably

.Use cross-validation and hyperparameter tuning for robust generalization

.Thank you. Questions and discussion are welcome